E-Pharmacy Main Project Code

Indexed technical document with structured headings and readable code layout

Generated: 2026-02-24 13:15:08
Total files included: 51

Scope excludes `.env*`, `node_modules`, build outputs, and non-core artifacts.

Index

Source Files

[1] README.md FILE

````
# E-Pharmacy

A dynamic web application built with Next.js (React) for the frontend and MongoDB for the
backend. This platform allows patients to upload prescriptions, and pharmacists can view and
fulfill them from nearby medical stores. Features an advanced pharmacy finder with Google Maps
integration and intelligent ranking system.

## Features

- **Patient Upload**: Patients can upload prescription images along with their details.
- **Pharmacist Dashboard**: Pharmacists can view pending prescriptions in their area and
assign them for fulfillment.
- **Advanced Pharmacy Finder**: Interactive Google Maps integration with location-based search
and intelligent ranking.
- **Service Partner Ranking**: Pharmacies using our e-pharmacy service are ranked higher in
search results.
- **Prescription Upload Support**: Clear tagging shows which pharmacies support prescription
uploads.
- **Location-based Matching**: Prescriptions are filtered by location to connect patients with
nearby pharmacists.
- **Geospatial Search**: Find pharmacies within a specified radius using GPS coordinates.

## Tech Stack

- **Frontend**: Next.js, React, Tailwind CSS, Google Maps JavaScript API
- **Backend**: Next.js API Routes, MongoDB with Mongoose
- **Database**: MongoDB with geospatial indexing
- **Maps**: Google Maps Platform

## Getting Started

### Prerequisites

- Node.js (v18 or later)
- MongoDB (local or MongoDB Atlas)
- Google Maps API Key

### Installation

1. Clone the repository:
   ```bash
   git clone <repository-url>
   cd e-pharmacy
   ```

2. Install dependencies:
   ```bash
   npm install
   ```

3. Set up environment variables:
   Create a `.env.local` file in the root directory and add your configuration:
   ```
   # MongoDB Atlas connection strings for separate databases
   PATIENT_MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/patients
   PHARMACIST_MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/pharmacists
   MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/epharmacy

   # JWT Secret for authentication
   JWT_SECRET=your-super-secret-jwt-key-here-make-it-long-and-random

   # Google Maps API Key (get from Google Cloud Console)
   NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=your-google-maps-api-key-here
   ```

4. Seed sample pharmacy data:
   ```bash
   curl -X POST http://localhost:3000/api/pharmacies/seed
   ```

5. Run the development server:
   ```bash
   npm run dev
   ```

6. Open [http://localhost:3000](http://localhost:3000) in your browser.
````

````markdown
## Usage

- **Home Page**: Navigate to upload prescriptions or access the pharmacist dashboard.
- **Upload Prescription**: Fill in patient details and upload a prescription image.
- **Pharmacist Dashboard**: Enter your location to view pending prescriptions and assign them.
- **Pharmacy Finder**: Use the interactive map to find nearby pharmacies. Service partners are marked with stars and ranked higher. Pharmacies supporting prescription uploads are clearly tagged.

## API Endpoints

### Prescriptions
- `POST /api/prescriptions` - Upload a new prescription
- `GET /api/prescriptions` - Fetch prescriptions (with optional location and status filters)
- `PUT /api/prescriptions/[id]` - Update prescription status

### Users
- `POST /api/users` - Create a new user (patient or pharmacist)
- `GET /api/users` - Fetch users by role

### Pharmacies
- `GET /api/pharmacies` - Search pharmacies with geospatial queries and ranking
  - Query parameters: `lat`, `lng`, `radius`, `q` (search query)
- `POST /api/pharmacies` - Register a new pharmacy
- `POST /api/pharmacies/seed` - Seed database with sample pharmacy data

### Authentication
- `POST /api/auth/login` - User login
- `GET /api/auth/me` - Get current user info
- `POST /api/auth/logout` - User logout

## Pharmacy Ranking System

Pharmacies are intelligently ranked based on multiple criteria:

1. **Service Partners**: Pharmacies using our e-pharmacy service (⭐ Service Partner) are ranked highest
2. **Subscription Tier**: Premium > Yearly > Monthly > Free
3. **Rating**: Higher rated pharmacies appear first
4. **Prescription Upload Support**: Tagged pharmacies (📄 Upload Support) support digital prescription uploads

## Google Maps Setup

1. Go to [Google Cloud Console](https://console.cloud.google.com/)
2. Create a new project or select existing one
3. Enable the Maps JavaScript API
4. Create credentials (API Key)
5. Add your API key to `.env.local` as `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`
6. Optionally restrict the API key to your domain for security

## Build and Deploy

To build the project for production:

```bash
npm run build
```

To start the production server:

```bash
npm start
```

Deploy on platforms like Vercel, Netlify, or any Node.js hosting service. Ensure to set the `MONGODB_URI` environment variable in your deployment environment.

## Contributing

Contributions are welcome! Please open an issue or submit a pull request.

## License

This project is licensed under the MIT License.
# E-Pharmacy
# E-Pharmacy
# E-Pharmacy
````

[2] app/api/auth/change-password/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { comparePassword, getUserFromToken, hashPassword } from '@/lib/auth';

export async function POST(request: NextRequest) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user) {
      return NextResponse.json({ error: 'Invalid session' }, { status: 401 });
    }

    const { currentPassword, newPassword, confirmPassword } = await request.json();

    if (!currentPassword || !newPassword || !confirmPassword) {
      return NextResponse.json({ error: 'All password fields are required' }, { status: 400
});
    }
    if (newPassword.length < 8) {
      return NextResponse.json({ error: 'New password must be at least 8 characters' }, {
status: 400 });
    }
    if (newPassword !== confirmPassword) {
      return NextResponse.json({ error: 'New password and confirm password do not match' }, {
status: 400 });
    }
    if (newPassword === currentPassword) {
      return NextResponse.json({ error: 'New password must be different from current password'
}, { status: 400 });
    }

    const existingPassword = typeof (user as { password?: unknown }).password === 'string'
      ? ((user as { password: string }).password)
      : '';

    if (!existingPassword) {
      return NextResponse.json({ error: 'Password is not set for this account' }, { status:
400 });
    }

    const isCurrentPasswordValid = await comparePassword(currentPassword, existingPassword);
    if (!isCurrentPasswordValid) {
      return NextResponse.json({ error: 'Current password is incorrect' }, { status: 400 });
    }

    const hashedPassword = await hashPassword(newPassword);
    (user as { password: string }).password = hashedPassword;
    await user.save();

    return NextResponse.json({ message: 'Password changed successfully' });
  } catch (error) {
    console.error('Change password error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

[3] app/api/auth/login/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { getUserModel } from '@/models/User';
import { comparePassword, generateToken } from '@/lib/auth';

export async function POST(request: NextRequest) {
  try {
    const { email, password, role } = await request.json();

    if (!email || !password || !role) {
      return NextResponse.json({ error: 'Email, password, and role are required' }, { status:
400 });
    }

    if (!['patient', 'pharmacist'].includes(role)) {
      return NextResponse.json({ error: 'Invalid role specified' }, { status: 400 });
    }

    const UserModel = await getUserModel(role as 'patient' | 'pharmacist');
    const user = await UserModel.findOne({ email });

    if (!user) {
```

```
      return NextResponse.json({ error: 'Invalid credentials' }, { status: 401 });
    }

    const isValidPassword = await comparePassword(password, user.password);
    if (!isValidPassword) {
      return NextResponse.json({ error: 'Invalid credentials' }, { status: 401 });
    }

    const token = generateToken({
      id: user._id.toString(),
      email: user.email,
      role: user.role,
      name: user.name,
    });

    const response = NextResponse.json({
      message: 'Login successful',
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
      },
    });

    // Set HTTP-only cookie
    response.cookies.set('auth-token', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 60 * 60 * 24 * 7, // 7 days
    });

    return response;
  } catch (error) {
    console.error('Login error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

[4] app/api/auth/logout/route.ts FILE

```
import { NextResponse } from 'next/server';

export async function POST() {
  const response = NextResponse.json({ message: 'Logout successful' });

  // Clear the auth token cookie
  response.cookies.set('auth-token', '', {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 0,
  });

  return response;
}
```

[5] app/api/auth/me/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { getUserFromToken } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

export async function GET(request: NextRequest) {
  try {
    const isOptional = request.nextUrl.searchParams.get('optional') === '1';
    const token = request.cookies.get('auth-token')?.value;

    if (!token) {
      if (isOptional) {
        return NextResponse.json({ user: null });
      }
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
    }

    const user = await getUserFromToken(token);

    if (!user) {
```

```
      if (isOptional) {
        return NextResponse.json({ user: null });
      }
      return NextResponse.json({ error: 'Invalid token' }, { status: 401 });
    }

    const safeUser = typeof (user as { toObject?: () => unknown }).toObject === 'function'
      ? ((user as { toObject: () => Record<string, unknown> }).toObject())
      : (user as unknown as Record<string, unknown>);

    const id = safeUser._id ? String(safeUser._id) : '';
    const name = typeof safeUser.name === 'string' ? safeUser.name : '';
    const email = typeof safeUser.email === 'string' ? safeUser.email : '';
    const role = typeof safeUser.role === 'string' ? safeUser.role : '';
    const location = typeof safeUser.location === 'string' ? safeUser.location : '';
    const subscriptionType =
      typeof safeUser.subscriptionType === 'string' ? safeUser.subscriptionType : null;

    if (!id || !email || !role) {
      if (isOptional) {
        return NextResponse.json({ user: null });
      }
      return NextResponse.json({ error: 'Invalid user session' }, { status: 401 });
    }

    let pharmacyId: string | null = null;
    if (role === 'pharmacist' && id) {
      const PharmacyModel = await getPharmacyModel();
      const pharmacy = await PharmacyModel.findOne({ pharmacistId: id }).select('_id').lean<{
_id: unknown } | null>();
      if (pharmacy?._id) {
        pharmacyId = String(pharmacy._id);
      }
    }

    return NextResponse.json({
      user: {
        id,
        name,
        email,
        role,
        location,
        subscriptionType,
        pharmacyId,
      },
    });
  } catch (error) {
    console.error('Auth check error:', error);
    if (request.nextUrl.searchParams.get('optional') === '1') {
      return NextResponse.json({ user: null });
    }
    return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
  }
}
```

[6] app/api/google/nearby-pharmacies/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';

type GooglePlace = {
  id: string;
  displayName?: { text?: string };
  formattedAddress?: string;
  location?: { latitude?: number; longitude?: number };
  rating?: number;
  userRatingCount?: number;
  nationalPhoneNumber?: string;
  internationalPhoneNumber?: string;
  websiteUri?: string;
};

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const latParam = searchParams.get('lat');
    const lngParam = searchParams.get('lng');
    const cityParam = searchParams.get('city');
    const searchQueryParam = searchParams.get('q');
    const radiusParam = searchParams.get('radius') || '10000';
    const limitParam = searchParams.get('limit') || '60';
    const lat = Number(latParam);
    const lng = Number(lngParam);
    const radius = Math.min(Number(radiusParam), 50000);
```

```
    const requestedLimit = Math.max(1, Math.min(Number(limitParam), 60));

    const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
    if (!apiKey) {
      return NextResponse.json({ error: 'Google Maps API key is not configured' }, { status:
500 });
    }

    const headers = {
      'Content-Type': 'application/json',
      'X-Goog-Api-Key': apiKey,
      'X-Goog-FieldMask':

'places.id,places.displayName,places.formattedAddress,places.location,places.rating,places.use
rRatingCount,places.nationalPhoneNumber,places.internationalPhoneNumber,places.websiteUri,next
PageToken',
    };

    const allPlaces: GooglePlace[] = [];
    let nextPageToken: string | undefined;

    for (let page = 0; page < 3 && allPlaces.length < requestedLimit; page++) {
      const pageSize = Math.min(20, requestedLimit - allPlaces.length);
      let response: Response;

      if (searchQueryParam && searchQueryParam.trim()) {
        response = await fetch('https://places.googleapis.com/v1/places:searchText', {
          method: 'POST',
          headers,
          body: JSON.stringify({
            textQuery: `pharmacy ${searchQueryParam.trim()} india`,
            maxResultCount: pageSize,
            pageToken: nextPageToken,
          }),
        });
      } else if (cityParam && cityParam.trim()) {
        response = await fetch('https://places.googleapis.com/v1/places:searchText', {
          method: 'POST',
          headers,
          body: JSON.stringify({
            textQuery: `pharmacy in ${cityParam.trim()}`,
            maxResultCount: pageSize,
            pageToken: nextPageToken,
          }),
        });
      } else {
        if (!latParam || !lngParam || Number.isNaN(lat) || Number.isNaN(lng) ||
Number.isNaN(radius)) {
          return NextResponse.json(
            { error: 'Provide either city or valid lat/lng coordinates' },
            { status: 400 }
          );
        }

        response = await fetch('https://places.googleapis.com/v1/places:searchNearby', {
          method: 'POST',
          headers,
          body: JSON.stringify({
            includedTypes: ['pharmacy'],
            maxResultCount: pageSize,
            pageToken: nextPageToken,
            locationRestriction: {
              circle: {
                center: {
                  latitude: lat,
                  longitude: lng,
                },
                radius,
              },
            },
          }),
        });
      }

      if (!response.ok) {
        const errorBody = await response.text();
        return NextResponse.json(
          { error: 'Google Places request failed', details: errorBody },
          { status: response.status }
        );
      }

      const data = (await response.json()) as { places?: GooglePlace[]; nextPageToken?: string
};
      const places = data.places || [];
      allPlaces.push(...places);
      nextPageToken = data.nextPageToken;
```

```
      if (!nextPageToken || places.length === 0) {
        break;
      }
    }

    const normalized = allPlaces
      .filter((place) => place.location?.latitude && place.location?.longitude)
      .map((place) => ({
        placeId: place.id,
        name: place.displayName?.text || 'Unknown Pharmacy',
        address: place.formattedAddress || '',
        coordinates: {
          lat: place.location?.latitude || 0,
          lng: place.location?.longitude || 0,
        },
        rating: place.rating || 0,
        reviewCount: place.userRatingCount || 0,
        phone: place.internationalPhoneNumber || place.nationalPhoneNumber || '',
        website: place.websiteUri || '',
      }));

    return NextResponse.json(normalized.slice(0, requestedLimit));
  } catch (error) {
    console.error('Nearby Google pharmacies error:', error);
    return NextResponse.json({ error: 'Failed to fetch nearby pharmacies' }, { status: 500 });
  }
}
```

[7] app/api/google/pincode-city/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';

type GeocodeResult = {
  formatted_address?: string;
  geometry?: {
    location?: {
      lat?: number;
      lng?: number;
    };
  };
  address_components?: Array<{
    long_name?: string;
    types?: string[];
  }>;
};

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const pincode = (searchParams.get('pincode') || '').trim();

    if (!/^[1-9][0-9]{5}$/.test(pincode)) {
      return NextResponse.json({ error: 'Valid Indian PIN code is required' }, { status: 400
});
    }

    const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
    if (!apiKey) {
      return NextResponse.json({ error: 'Google Maps API key is not configured' }, { status:
500 });
    }

    const response = await fetch(
      `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURIComponent(
        `${pincode}, India`
      )}&components=country:IN&key=${apiKey}`
    );

    if (!response.ok) {
      return NextResponse.json({ error: 'Geocode request failed' }, { status: response.status
});
    }

    const payload = (await response.json()) as {
      results?: GeocodeResult[];
      status?: string;
      error_message?: string;
    };

    if (payload.status !== 'OK' || !payload.results?.length) {
      return NextResponse.json(
        { error: payload.error_message || 'Could not resolve city from PIN code' },
        { status: 404 }
```

```
      );
    }

    const components = payload.results[0].address_components || [];
    const cityComponent = components.find(
      (component) =>
        component.types?.includes('locality') ||
        component.types?.includes('postal_town') ||
        component.types?.includes('administrative_area_level_2') ||
        component.types?.includes('administrative_area_level_3')
    );

    const city = cityComponent?.long_name || '';
    if (!city) {
      return NextResponse.json({ error: 'City was not found for this PIN code' }, { status:
404 });
    }

    const lat = payload.results[0].geometry?.location?.lat;
    const lng = payload.results[0].geometry?.location?.lng;
    if (typeof lat !== 'number' || typeof lng !== 'number') {
      return NextResponse.json({ error: 'Coordinates were not found for this PIN code' }, {
status: 404 });
    }

    return NextResponse.json({
      city,
      formattedAddress: payload.results[0].formatted_address || '',
      coordinates: { lat, lng },
    });
  } catch (error) {
    console.error('Pincode city lookup error:', error);
    return NextResponse.json({ error: 'Failed to resolve city from PIN code' }, { status: 500
});
  }
}
```

[8] app/api/google/reverse-geocode/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';

type GeocodeResult = {
  formatted_address?: string;
  address_components?: Array<{
    long_name?: string;
    short_name?: string;
    types?: string[];
  }>;
};

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const latParam = searchParams.get('lat');
    const lngParam = searchParams.get('lng');

    if (!latParam || !lngParam) {
      return NextResponse.json({ error: 'lat and lng are required' }, { status: 400 });
    }

    const lat = Number(latParam);
    const lng = Number(lngParam);
    if (Number.isNaN(lat) || Number.isNaN(lng)) {
      return NextResponse.json({ error: 'Invalid coordinates' }, { status: 400 });
    }

    const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
    if (!apiKey) {
      return NextResponse.json({ error: 'Google Maps API key is not configured' }, { status:
500 });
    }

    const response = await fetch(
      `https://maps.googleapis.com/maps/api/geocode/json?latlng=${lat},${lng}&key=${apiKey}`
    );

    if (!response.ok) {
      return NextResponse.json({ error: 'Reverse geocode request failed' }, { status:
response.status });
    }

    const payload = (await response.json()) as {
      results?: GeocodeResult[];
```

```
      status?: string;
      error_message?: string;
    };

    if (payload.status !== 'OK' || !payload.results?.length) {
      return NextResponse.json(
        { error: payload.error_message || 'Could not resolve city from coordinates' },
        { status: 404 }
      );
    }

    const components = payload.results[0].address_components || [];
    const cityComponent = components.find(
      (component) =>
        component.types?.includes('locality') ||
        component.types?.includes('postal_town') ||
        component.types?.includes('administrative_area_level_2')
    );

    const city = cityComponent?.long_name || '';
    const formattedAddress =
      typeof payload.results[0] === 'object' &&
      payload.results[0] &&
      'formatted_address' in payload.results[0]
        ? String((payload.results[0] as { formatted_address?: unknown }).formatted_address ||
'')
        : '';
    if (!city) {
      return NextResponse.json({ error: 'City was not found in geocode response' }, { status:
404 });
    }

    return NextResponse.json({ city, formattedAddress });
  } catch (error) {
    console.error('Reverse geocode error:', error);
    return NextResponse.json({ error: 'Failed to reverse geocode location' }, { status: 500
});
  }
}
```

[9] app/api/notifications/stream/route.ts FILE

```
import { NextRequest } from 'next/server';
import dbConnect from '@/lib/mongodb';
import { getUserFromToken } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

export const runtime = 'nodejs';

// Store active connections for SSE
type ActiveConnection = {
  pharmacistId: string;
  controller: ReadableStreamDefaultController;
};

const activeConnections = new Map<string, ActiveConnection>();

export async function GET(request: NextRequest) {
  await dbConnect();

  try {
    // Verify pharmacist authentication
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return new Response('Unauthorized', { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user || user.role !== 'pharmacist') {
      return new Response('Unauthorized', { status: 401 });
    }

    // Create SSE response
    const stream = new ReadableStream({
      start(controller) {
        // Send initial connection message
        const encoder = new TextEncoder();
        controller.enqueue(encoder.encode('data: {"type": "connected", "message": "Connected
to notification stream"}\n\n'));

        // Store the connection
        const connectionId = `${user._id}-${Date.now()}`;
        activeConnections.set(connectionId, {
```

```
        pharmacistId: user._id.toString(),
        controller,
      });

      // Clean up on disconnect
      request.signal.addEventListener('abort', () => {
        activeConnections.delete(connectionId);
      });
    },
    cancel() {
      // Connection closed
    }
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive',
      'Access-Control-Allow-Origin': '*',
      'Access-Control-Allow-Headers': 'Cache-Control',
    },
  });
} catch (error) {
  console.error('Notification stream error:', error);
  return new Response('Internal Server Error', { status: 500 });
}
}

// Function to send notifications to pharmacists
export async function sendNotificationToPharmacists(prescription: {
  _id: string;
  patientName: string;
  location: string;
  createdAt: Date;
  status: string;
  pharmacistIds?: string[];
}) {
  const encoder = new TextEncoder();
  const selectedPharmacyIds = prescription.pharmacistIds || [];
  const targetPharmacistIds = new Set<string>();

  if (selectedPharmacyIds.length > 0) {
    try {
      const PharmacyModel = await getPharmacyModel();
      const matchedPharmacies = await PharmacyModel.find(
        { _id: { $in: selectedPharmacyIds } },
        { pharmacistId: 1 }
      ).lean<{ pharmacistId?: { toString: () => string } }[]>();

      for (const pharmacy of matchedPharmacies) {
        if (pharmacy.pharmacistId) {
          targetPharmacistIds.add(pharmacy.pharmacistId.toString());
        }
      }
    } catch (error) {
      console.error('Failed to resolve target pharmacists for notification:', error);
    }
  }

  const notification = {
    type: 'new_prescription',
    prescription: {
      id: prescription._id,
      patientName: prescription.patientName,
      location: prescription.location,
      createdAt: prescription.createdAt,
      status: prescription.status,
      selectedPharmacyIds,
    },
  };

  const data = `data: ${JSON.stringify(notification)}\n\n`;

  for (const [connectionId, connection] of activeConnections) {
    const shouldSend =
      targetPharmacistIds.size === 0 || targetPharmacistIds.has(connection.pharmacistId);
    if (!shouldSend) {
      continue;
    }
    try {
      connection.controller.enqueue(encoder.encode(data));
    } catch (error) {
      // Remove broken connections
      activeConnections.delete(connectionId);
      console.error('Failed to deliver notification to active connection:', error);
    }
```

```
    }
  }
```

[10] app/api/patient/history/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import getPrescriptionModel from '@/models/Prescription';
import getPharmacyModel from '@/models/Pharmacy';
import { getUserFromToken } from '@/lib/auth';

type PharmacyStatusEntry = {
  pharmacyId?: unknown;
  status?: string;
  availabilityResponse?: string | null;
  pharmacistMessage?: string;
  assignedAt?: string | null;
  completedAt?: string | null;
};

type PrescriptionRecord = Record<string, unknown> & {
  pharmacyStatuses?: PharmacyStatusEntry[];
  toObject?: () => Record<string, unknown> & { pharmacyStatuses?: PharmacyStatusEntry[] };
};

export async function GET(request: NextRequest) {
  try {
    // Get token from cookie
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user || user.role !== 'patient') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    // Get prescriptions for this patient from main database
    const Prescription = await getPrescriptionModel();
    const prescriptions = await Prescription.find({
      patientEmail: user.email
    }).sort({ createdAt: -1 });
    const typedPrescriptions = prescriptions as PrescriptionRecord[];

    const pharmacyIds = Array.from(
      new Set(
        typedPrescriptions.flatMap((prescription) =>
          Array.isArray(prescription.pharmacyStatuses)
            ? prescription.pharmacyStatuses
                .map((entry: { pharmacyId?: unknown }) => String(entry?.pharmacyId || ''))
                .filter(Boolean)
            : []
        )
      )
    );

    let pharmacyById = new Map<string, { _id: unknown; name: string; address: string;
location: string }>();
    if (pharmacyIds.length > 0) {
      const Pharmacy = await getPharmacyModel();
      const pharmacies = await Pharmacy.find({ _id: { $in: pharmacyIds } })
        .select('_id name address location')
        .lean();

      pharmacyById = new Map(
        pharmacies.map((pharmacy: { _id: unknown; name: string; address: string; location:
string }) => [
          String(pharmacy._id),
          pharmacy,
        ])
      );
    }

    const enrichedPrescriptions = typedPrescriptions.map((prescription) => {
      const base = prescription?.toObject ? prescription.toObject() : prescription;
      const pharmacyDetails = (Array.isArray(base.pharmacyStatuses) ? base.pharmacyStatuses :
[]).map(
        (entry: PharmacyStatusEntry) => {
          const pharmacyId = String(entry?.pharmacyId || '');
          const pharmacy = pharmacyById.get(pharmacyId);
          return {
            pharmacyId,
            name: pharmacy?.name || 'Pharmacy name unavailable',
```

```
          address: pharmacy?.address || '',
          location: pharmacy?.location || '',
          status: entry?.status || 'pending',
          availabilityResponse: entry?.availabilityResponse || null,
          pharmacistMessage: entry?.pharmacistMessage || '',
          assignedAt: entry?.assignedAt || null,
          completedAt: entry?.completedAt || null,
        };
      }
    );

    return {
      ...base,
      pharmacyDetails,
    };
  });

  return NextResponse.json(enrichedPrescriptions);
} catch (error) {
  console.error('Patient history error:', error);
  return NextResponse.json({ error: 'Failed to fetch prescription history' }, { status: 500
});
  }
}
```

[11] app/api/pharmacies/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import mongoose from 'mongoose';
import getPharmacyModel from '@/models/Pharmacy';
import getPharmacistUserModel from '@/models/PharmacistUser';

const capitalizeFirstCharacter = (value: string) => {
  const trimmed = value.trim();
  if (!trimmed) return trimmed;
  return `${trimmed.charAt(0).toUpperCase()}${trimmed.slice(1)}`;
};

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const lat = searchParams.get('lat');
    const lng = searchParams.get('lng');
    const radius = searchParams.get('radius') || '5000'; // Default 5km radius
    const query = searchParams.get('q'); // Search query for pharmacy name/address
    const location = searchParams.get('location'); // Location-based search for upload page
    const limit = searchParams.get('limit') ? parseInt(searchParams.get('limit')!) :
undefined;
    const onlySubscribed = searchParams.get('onlySubscribed') === '1';

    const PharmacyModel = await getPharmacyModel();

    type PharmacyResult = {
      _id: mongoose.Types.ObjectId;
      pharmacistId?: mongoose.Types.ObjectId;
      coordinates?: { lat?: number; lng?: number };
      isUsingService?: boolean;
      subscriptionType?: 'monthly' | 'yearly' | 'premium' | null;
      rating?: number;
      reviewCount?: number;
      createdAt?: Date;
      [key: string]: unknown;
    };

    const getPriority = (pharmacy: PharmacyResult) => {
      // Highest priority: subscribed/active service pharmacies.
      if (pharmacy.isUsingService || pharmacy.subscriptionType) return 3;
      // Next: pharmacies registered with us.
      if (pharmacy.pharmacistId) return 2;
      // Lowest: generic listings.
      return 1;
    };

    const sortPharmacies = (list: PharmacyResult[]) =>
      list.sort((a, b) => {
        const priorityDiff = getPriority(b) - getPriority(a);
        if (priorityDiff !== 0) return priorityDiff;

        const ratingDiff = (b.rating || 0) - (a.rating || 0);
        if (ratingDiff !== 0) return ratingDiff;

        const reviewDiff = (b.reviewCount || 0) - (a.reviewCount || 0);
        if (reviewDiff !== 0) return reviewDiff;
```

```
      return new Date(b.createdAt || 0).getTime() - new Date(a.createdAt || 0).getTime();
    });

  const calculateDistanceMeters = (
    sourceLat: number,
    sourceLng: number,
    targetLat: number,
    targetLng: number
  ) => {
    const toRad = (value: number) => (value * Math.PI) / 180;
    const earthRadiusM = 6371000;
    const dLat = toRad(targetLat - sourceLat);
    const dLng = toRad(targetLng - sourceLng);
    const a =
      Math.sin(dLat / 2) * Math.sin(dLat / 2) +
      Math.cos(toRad(sourceLat)) *
        Math.cos(toRad(targetLat)) *
        Math.sin(dLng / 2) *
        Math.sin(dLng / 2);
    const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return earthRadiusM * c;
  };

  let pharmacies: PharmacyResult[];

  if (lat && lng) {
    const sourceLat = parseFloat(lat);
    const sourceLng = parseFloat(lng);
    const maxDistance = parseInt(radius, 10);
    if (Number.isNaN(sourceLat) || Number.isNaN(sourceLng) || Number.isNaN(maxDistance)) {
      return NextResponse.json({ error: 'Invalid lat/lng/radius' }, { status: 400 });
    }

    const allPharmacies = await PharmacyModel.find({}).lean<PharmacyResult[]>();

    const withDistance = allPharmacies
      .map((pharmacy) => {
        const targetLat = pharmacy.coordinates?.lat;
        const targetLng = pharmacy.coordinates?.lng;
        if (typeof targetLat !== 'number' || typeof targetLng !== 'number') {
          return null;
        }
        const distanceMeters = calculateDistanceMeters(sourceLat, sourceLng, targetLat,
targetLng);
        return { ...pharmacy, distanceMeters };
      })
      .filter(
        (pharmacy): pharmacy is PharmacyResult & { distanceMeters: number } => !!pharmacy
      )
      .sort((a, b) => {
        const distanceDiff = a.distanceMeters - b.distanceMeters;
        if (distanceDiff !== 0) return distanceDiff;

        const priorityDiff = getPriority(b) - getPriority(a);
        if (priorityDiff !== 0) return priorityDiff;

        return (b.rating || 0) - (a.rating || 0);
      });

    const withinRadius = withDistance.filter((pharmacy) => pharmacy.distanceMeters <=
maxDistance);
    const nearestFallback = withinRadius.length > 0 ? withinRadius : withDistance.slice(0,
50);

    pharmacies = nearestFallback.map(
      (pharmacy) =>
        Object.fromEntries(
          Object.entries(pharmacy).filter(([key]) => key !== 'distanceMeters')
        ) as PharmacyResult
    );

  } else if (query) {
    // Text-based search
    pharmacies = await PharmacyModel.find({
      $or: [
        { name: { $regex: query, $options: 'i' } },
        { address: { $regex: query, $options: 'i' } },
        { location: { $regex: query, $options: 'i' } }
      ]
    }).lean<PharmacyResult[]>();
    pharmacies = sortPharmacies(pharmacies);
  } else if (location) {
    // Location-based search for upload page
    pharmacies = await PharmacyModel.find({
      location: { $regex: location, $options: 'i' }
    }).lean<PharmacyResult[]>();
```

```
      pharmacies = sortPharmacies(pharmacies);
    } else {
      // Return all pharmacies sorted by service usage and rating
      pharmacies = await PharmacyModel.find({}).lean<PharmacyResult[]>();
      pharmacies = sortPharmacies(pharmacies);
    }

    // Filter pharmacies to only include those with valid pharmacist accounts
    const PharmacistUserModel = await getPharmacistUserModel();
    const validPharmacistIds = await PharmacistUserModel.find({}, '_id').lean<{ _id:
mongoose.Types.ObjectId }[]>();
    const validPharmacistIdSet = new Set(validPharmacistIds.map((p: { _id:
mongoose.Types.ObjectId }) => p._id.toString()));

    const hasLinkedPharmacists = validPharmacistIdSet.size > 0;

    const filteredPharmacies = pharmacies.filter((pharmacy) => {
      if (hasLinkedPharmacists) {
        const hasValidPharmacist =
          pharmacy.pharmacistId && validPharmacistIdSet.has(pharmacy.pharmacistId.toString());
        if (!hasValidPharmacist) return false;
      }

      if (!onlySubscribed) return true;
      return Boolean(pharmacy.isUsingService || pharmacy.subscriptionType);
    });

    // Apply limit if specified
    const result = limit ? filteredPharmacies.slice(0, limit) : filteredPharmacies;

    return NextResponse.json(result);
  } catch (error) {
    console.error('Pharmacy search error:', error);
    return NextResponse.json({ error: 'Failed to search pharmacies' }, { status: 500 });
  }
}

export async function POST(request: NextRequest) {
  try {
    const pharmacyData = (await request.json()) as Record<string, unknown>;
    if (typeof pharmacyData.name === 'string') {
      pharmacyData.name = capitalizeFirstCharacter(pharmacyData.name);
    }
    const PharmacyModel = await getPharmacyModel();

    const pharmacy = new PharmacyModel(pharmacyData);
    await pharmacy.save();

    return NextResponse.json({
      message: 'Pharmacy registered successfully',
      id: pharmacy._id
    }, { status: 201 });
  } catch (error: unknown) {
    console.error('Pharmacy registration error:', error);
    if (error && typeof error === 'object' && 'code' in error && error.code === 11000) {
      return NextResponse.json({ error: 'Pharmacy already exists' }, { status: 400 });
    }
    return NextResponse.json({ error: 'Failed to register pharmacy' }, { status: 500 });
  }
}
```

[12] app/api/pharmacies/seed-test/route.ts FILE

```
import { NextResponse } from 'next/server';
import { hashPassword } from '@/lib/auth';
import getPharmacistUserModel from '@/models/PharmacistUser';
import getPharmacyModel from '@/models/Pharmacy';

export async function POST() {
  try {
    const PharmacistUserModel = await getPharmacistUserModel();
    const PharmacyModel = await getPharmacyModel();

    const testEmail = 'india.pharmacy@test.local';
    const testPassword = 'India@12345';

    let pharmacist = await PharmacistUserModel.findOne({ email: testEmail });

    if (!pharmacist) {
      const hashed = await hashPassword(testPassword);
      pharmacist = await PharmacistUserModel.create({
        name: 'India Pharmacy',
        email: testEmail,
```

```
        password: hashed,
        role: 'pharmacist',
        phone: '+91-9999999999',
        address: 'Connaught Place, New Delhi, India',
        location: 'New Delhi',
        subscriptionType: 'monthly',
        subscriptionStart: new Date(),
        subscriptionEnd: (() => {
          const d = new Date();
          d.setMonth(d.getMonth() + 1);
          return d;
        })(),
      });
    } else {
      pharmacist.subscriptionType = 'monthly';
      pharmacist.subscriptionStart = new Date();
      const end = new Date();
      end.setMonth(end.getMonth() + 1);
      pharmacist.subscriptionEnd = end;
      pharmacist.location = pharmacist.location || 'New Delhi';
      await pharmacist.save();
    }

    let pharmacy = await PharmacyModel.findOne({ pharmacistId: pharmacist._id });

    if (!pharmacy) {
      pharmacy = await PharmacyModel.create({
        pharmacistId: pharmacist._id,
        name: 'India Pharmacy',
        email: testEmail,
        phone: '+91-9999999999',
        address: 'Connaught Place, New Delhi, India',
        location: 'New Delhi',
        coordinates: { lat: 28.6139, lng: 77.209 },
        supportsPrescriptionUpload: true,
        isUsingService: true,
        subscriptionType: 'monthly',
        rating: 4.7,
        reviewCount: 120,
        services: ['Prescription Fulfillment', 'Home Delivery'],
      });
    } else {
      pharmacy.name = 'India Pharmacy';
      pharmacy.email = testEmail;
      pharmacy.phone = '+91-9999999999';
      pharmacy.address = pharmacy.address || 'Connaught Place, New Delhi, India';
      pharmacy.location = pharmacy.location || 'New Delhi';
      pharmacy.coordinates = pharmacy.coordinates || { lat: 28.6139, lng: 77.209 };
      pharmacy.supportsPrescriptionUpload = true;
      pharmacy.isUsingService = true;
      pharmacy.subscriptionType = 'monthly';
      if (!pharmacy.rating || pharmacy.rating < 1) pharmacy.rating = 4.7;
      if (!pharmacy.reviewCount || pharmacy.reviewCount < 1) pharmacy.reviewCount = 120;
      await pharmacy.save();
    }

    return NextResponse.json({
      message: 'Test pharmacist and pharmacy are ready.',
      credentials: {
        role: 'pharmacist',
        email: testEmail,
        password: testPassword,
      },
      pharmacistId: pharmacist._id,
      pharmacyId: pharmacy._id,
    });
  } catch (error) {
    console.error('Seed test pharmacist error:', error);
    return NextResponse.json({ error: 'Failed to create test pharmacist' }, { status: 500 });
  }
}
```

[13] app/api/pharmacies/seed/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { seedPharmacies } from '@/lib/seedPharmacies';

export async function POST(request: NextRequest) {
  try {
    await seedPharmacies();
    return NextResponse.json({ message: 'Pharmacies seeded successfully' });
  } catch (error) {
    console.error('Error seeding pharmacies:', error);
```

```
    return NextResponse.json({ error: 'Failed to seed pharmacies' }, { status: 500 });
  }
}
```

[14] app/api/prescriptions/[id]/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import getPrescriptionModel from '@/models/Prescription';
import { getUserFromToken } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

export async function PUT(request: NextRequest, { params }: { params: Promise<{ id: string }>
}) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user || !['pharmacist', 'patient'].includes(user.role)) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { id } = await params;
    const body = await request.json() as {
      status?: 'assigned' | 'request_fulfillment' | 'confirm_fulfillment';
      pharmacyId?: string;
      availabilityResponse?: 'not_available' | 'same_medicine_available' |
'same_salt_different_company';
      customMessage?: string;
    };
    const { status, pharmacyId, availabilityResponse, customMessage } = body;

    const Prescription = await getPrescriptionModel();
    const prescription = await Prescription.findById(id);

    if (!prescription) {
      return NextResponse.json({ error: 'Prescription not found' }, { status: 404 });
    }

    if (!Array.isArray(prescription.pharmacyStatuses)) {
      prescription.pharmacyStatuses = [];
    }

    if (user.role === 'patient') {
      if (prescription.patientEmail !== user.email) {
        return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
      }

      if (status !== 'confirm_fulfillment' || !pharmacyId) {
        return NextResponse.json({ error: 'Invalid patient confirmation request' }, { status:
400 });
      }

      const matchingStatus = prescription.pharmacyStatuses.find(
        (entry: { pharmacyId?: unknown }) => String(entry?.pharmacyId) === String(pharmacyId)
      );

      if (!matchingStatus) {
        return NextResponse.json({ error: 'Selected pharmacy not found for this prescription'
}, { status: 404 });
      }

      if (matchingStatus.status !== 'fulfillment_requested') {
        return NextResponse.json({ error: 'No pending fulfillment confirmation request from
this pharmacy' }, { status: 400 });
      }

      matchingStatus.status = 'fulfilled';
      matchingStatus.completedAt = new Date();

      const statuses = prescription.pharmacyStatuses.map((entry: { status?: string }) =>
entry?.status || 'pending');
      const hasFulfilled = statuses.includes('fulfilled');
      const hasOpen = statuses.some((value: string) => !['fulfilled',
'rejected'].includes(value));
      if (hasFulfilled && !hasOpen) {
        prescription.status = 'fulfilled';
      }

      await prescription.save();
```

```
      return NextResponse.json({
        message: 'Fulfillment confirmed successfully',
        prescription,
      });
    }

    if (!status || !['assigned', 'request_fulfillment'].includes(status)) {
      return NextResponse.json({ error: 'Invalid status' }, { status: 400 });
    }

    if (status === 'assigned') {
      const validAvailabilityResponses = new Set([
        'not_available',
        'same_medicine_available',
        'same_salt_different_company',
      ]);
      if (!availabilityResponse || !validAvailabilityResponses.has(availabilityResponse)) {
        return NextResponse.json(
          { error: 'Select a valid medicine availability response' },
          { status: 400 }
        );
      }
      if (typeof customMessage === 'string' && customMessage.length > 500) {
        return NextResponse.json(
          { error: 'Custom message must be 500 characters or fewer' },
          { status: 400 }
        );
      }
    }

    const PharmacyModel = await getPharmacyModel();
    const pharmacy = await PharmacyModel.findOne({ pharmacistId: user._id }).select('_id');
    if (!pharmacy?._id) {
      return NextResponse.json({ error: 'Pharmacy profile not found for this pharmacist' }, {
status: 404 });
    }

    const pharmacyIdString = pharmacy?._id ? String(pharmacy._id) : '';

    if (pharmacyIdString) {
      const matchingStatus = prescription.pharmacyStatuses.find(
        (entry: { pharmacyId?: unknown }) => String(entry?.pharmacyId) === pharmacyIdString
      );
      if (status === 'assigned') {
        if (matchingStatus) {
          matchingStatus.status = 'accepted';
          matchingStatus.availabilityResponse = availabilityResponse;
          matchingStatus.pharmacistMessage = (customMessage || '').trim();
          matchingStatus.assignedAt = new Date();
        } else {
          prescription.pharmacyStatuses.push({
            pharmacyId: pharmacy._id,
            status: 'accepted',
            availabilityResponse,
            pharmacistMessage: (customMessage || '').trim(),
            assignedAt: new Date(),
          });
        }
      } else if (status === 'request_fulfillment') {
        if (prescription.status !== 'assigned') {
          return NextResponse.json(
            { error: 'Prescription must be accepted before requesting patient fulfillment' },
            { status: 400 }
          );
        }

        if (matchingStatus?.status === 'fulfillment_requested') {
          return NextResponse.json({
            message: 'Patient confirmation already requested',
            prescription,
          });
        }
        const resolvedAvailability =
          (availabilityResponse as string | undefined) ||
          (matchingStatus?.availabilityResponse as string | undefined) ||
          'same_medicine_available';
        const resolvedMessage =
          typeof customMessage === 'string'
            ? customMessage.trim()
            : (matchingStatus?.pharmacistMessage as string | undefined) || '';

        // Write fulfillment request directly to MongoDB to avoid stale in-memory enum issues.
        if (matchingStatus) {
          await Prescription.updateOne(
            { _id: id, 'pharmacyStatuses.pharmacyId': pharmacy._id },
            {
              $set: {
```

```
            'pharmacyStatuses.$.status': 'fulfillment_requested',
            'pharmacyStatuses.$.availabilityResponse': resolvedAvailability,
            'pharmacyStatuses.$.pharmacistMessage': resolvedMessage,
            status: 'assigned',
          },
        }
      );
    } else {
      await Prescription.updateOne(
        { _id: id },
        {
          $push: {
            pharmacyStatuses: {
              pharmacyId: pharmacy._id,
              status: 'fulfillment_requested',
              availabilityResponse: resolvedAvailability,
              pharmacistMessage: resolvedMessage,
              assignedAt: new Date(),
            },
          },
          $set: { status: 'assigned' },
        }
      );
    }

    const updatedPrescription = await Prescription.findById(id);
    return NextResponse.json({
      message: 'Patient confirmation requested for fulfillment',
      prescription: updatedPrescription || prescription,
    });
  }
}

if (status === 'assigned') {
  prescription.status = 'assigned';
}
await prescription.save();

return NextResponse.json({
  message:
    status === 'request_fulfillment'
      ? 'Patient confirmation requested for fulfillment'
      : 'Prescription accepted successfully',
  prescription
});
} catch (error) {
  console.error('Prescription update error:', error);
  return NextResponse.json({ error: 'Failed to update prescription' }, { status: 500 });
}
}

export async function DELETE(request: NextRequest, { params }: { params: Promise<{ id: string
}> }) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user || user.role !== 'patient') {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { id } = await params;
    const Prescription = await getPrescriptionModel();

    const deletedPrescription = await Prescription.findOneAndDelete({
      _id: id,
      patientEmail: user.email,
    });

    if (!deletedPrescription) {
      return NextResponse.json({ error: 'Prescription not found' }, { status: 404 });
    }

    return NextResponse.json({ message: 'Prescription removed successfully' });
  } catch (error) {
    console.error('Prescription delete error:', error);
    return NextResponse.json({ error: 'Failed to remove prescription' }, { status: 500 });
  }
}
```

[15] app/api/prescriptions/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import getPrescriptionModel from '@/models/Prescription';
import { sendNotificationToPharmacists } from '@/app/api/notifications/stream/route';

export async function POST(request: NextRequest) {
  try {
    const Prescription = await getPrescriptionModel();
    const { patientName, patientEmail, patientPhone, patientAddress, prescriptionImages,
notes, location, selectedPharmacyIds } = await request.json();

    if (!prescriptionImages || prescriptionImages.length === 0) {
      return NextResponse.json({ error: 'At least one prescription image is required' }, {
status: 400 });
    }

    if (!selectedPharmacyIds || selectedPharmacyIds.length === 0) {
      return NextResponse.json({ error: 'At least one pharmacy must be selected' }, { status:
400 });
    }

    // Create pharmacy status entries for each selected pharmacy
    const pharmacyStatuses = selectedPharmacyIds.map((pharmacyId: string) => ({
      pharmacyId,
      status: 'pending',
      assignedAt: new Date()
    }));

    const prescription = new Prescription({
      patientName,
      patientEmail,
      patientPhone,
      patientAddress,
      prescriptionImages,
      notes,
      location,
      pharmacistIds: selectedPharmacyIds,
      pharmacyStatuses,
      status: 'assigned' // Overall status is assigned when pharmacies are selected
    });

    await prescription.save();

    // Send real-time notification to selected pharmacists
    try {
      await sendNotificationToPharmacists(prescription);
    } catch (error) {
      console.error('Failed to send notification:', error);
      // Don't fail the upload if notification fails
    }

    return NextResponse.json({ message: 'Prescription uploaded successfully', id:
prescription._id }, { status: 201 });
  } catch (error) {
    console.error('Prescription upload error:', error);
    return NextResponse.json({ error: 'Failed to upload prescription' }, { status: 500 });
  }
}

export async function GET(request: NextRequest) {
  try {
    const Prescription = await getPrescriptionModel();
    const { searchParams } = new URL(request.url);
    const location = searchParams.get('location');
    const status = searchParams.get('status') || 'pending';

    const query: Record<string, unknown> = {};
    if (status !== 'all') {
      query.status = status;
    }
    if (location && location.trim()) {
      query.location = { $regex: location, $options: 'i' };
    }

    const prescriptions = await Prescription.find(query).sort({ createdAt: -1 });

    return NextResponse.json(prescriptions);
  } catch (error) {
    return NextResponse.json({ error: 'Failed to fetch prescriptions' }, { status: 500 });
  }
}
```

[16] app/api/profile/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { getUserFromToken } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

const getSafeString = (value: unknown) => (typeof value === 'string' ? value.trim() : '');
const capitalizeFirstCharacter = (value: string) => {
  const trimmed = value.trim();
  if (!trimmed) return trimmed;
  return `${trimmed.charAt(0).toUpperCase()}${trimmed.slice(1)}`;
};

export async function GET(request: NextRequest) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user) {
      return NextResponse.json({ error: 'Invalid session' }, { status: 401 });
    }

    const safeUser = typeof (user as { toObject?: () => unknown }).toObject === 'function'
      ? ((user as { toObject: () => Record<string, unknown> }).toObject())
      : (user as unknown as Record<string, unknown>);

    const role = getSafeString(safeUser.role);
    let subscriptionType = getSafeString(safeUser.subscriptionType);
    const subscriptionStart = safeUser.subscriptionStart ? String(safeUser.subscriptionStart)
: '';
    const subscriptionEnd = safeUser.subscriptionEnd ? String(safeUser.subscriptionEnd) : '';

    if (role === 'pharmacist' && (!subscriptionType || !subscriptionStart ||
!subscriptionEnd)) {
      try {
        const PharmacyModel = await getPharmacyModel();
        const pharmacy = await PharmacyModel.findOne({ pharmacistId: safeUser._id })
          .select('subscriptionType')
          .lean<{ subscriptionType?: unknown } | null>();
        if (!subscriptionType) {
          subscriptionType = getSafeString(pharmacy?.subscriptionType);
        }
      } catch {
        // Keep profile GET resilient if pharmacy lookup fails.
      }
    }

    return NextResponse.json({
      profile: {
        id: safeUser._id ? String(safeUser._id) : '',
        name: getSafeString(safeUser.name),
        email: getSafeString(safeUser.email),
        role,
        phone: getSafeString(safeUser.phone),
        address: getSafeString(safeUser.address),
        location: getSafeString(safeUser.location),
        subscriptionType,
        subscriptionStart,
        subscriptionEnd,
      },
    });
  } catch (error) {
    console.error('Profile GET error:', error);
    return NextResponse.json({ error: 'Failed to load profile' }, { status: 500 });
  }
}

export async function PUT(request: NextRequest) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user) {
      return NextResponse.json({ error: 'Invalid session' }, { status: 401 });
    }

    const body = await request.json();
    const name = getSafeString(body?.name);
    const phone = getSafeString(body?.phone);
    const address = getSafeString(body?.address);
    const location = getSafeString(body?.location);
```

```
    if (!name) {
      return NextResponse.json({ error: 'Name is required' }, { status: 400 });
    }

    const role = getSafeString((user as { role?: unknown }).role);
    const normalizedName = role === 'pharmacist' ? capitalizeFirstCharacter(name) : name;

    (user as { name: string }).name = normalizedName;
    (user as { phone?: string }).phone = phone;
    (user as { address?: string }).address = address;

    if (role === 'pharmacist') {
      (user as { location?: string }).location = location;
    }

    await user.save();

    if (role === 'pharmacist') {
      const PharmacyModel = await getPharmacyModel();
      const pharmacistId = (user as { _id: unknown })._id;

      const pharmacyUpdates: Record<string, string> = {
        name: normalizedName,
        email: getSafeString((user as { email?: unknown }).email),
      };
      if (phone) pharmacyUpdates.phone = phone;
      if (address) pharmacyUpdates.address = address;
      if (location) pharmacyUpdates.location = location;

      await PharmacyModel.findOneAndUpdate({ pharmacistId }, pharmacyUpdates, { new: true });
    }

    return NextResponse.json({
      message: 'Profile updated successfully',
      profile: {
        id: (user as { _id: { toString: () => string } })._id.toString(),
        name: normalizedName,
        email: getSafeString((user as { email?: unknown }).email),
        role,
        phone,
        address,
        location: role === 'pharmacist' ? location : '',
        subscriptionType: getSafeString((user as { subscriptionType?: unknown
}).subscriptionType),
        subscriptionStart: (user as { subscriptionStart?: unknown }).subscriptionStart
          ? String((user as { subscriptionStart?: unknown }).subscriptionStart)
          : '',
        subscriptionEnd: (user as { subscriptionEnd?: unknown }).subscriptionEnd
          ? String((user as { subscriptionEnd?: unknown }).subscriptionEnd)
          : '',
      },
    });
  } catch (error) {
    console.error('Profile update error:', error);
    return NextResponse.json({ error: 'Failed to update profile' }, { status: 500 });
  }
}
```

[17] app/api/subscription/activate/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { getUserFromToken } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

const VALID_PLANS = new Set(['monthly', 'yearly', 'premium']);

export async function POST(request: NextRequest) {
  try {
    const token = request.cookies.get('auth-token')?.value;
    if (!token) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const user = await getUserFromToken(token);
    if (!user || user.role !== 'pharmacist') {
      return NextResponse.json({ error: 'Only pharmacists can activate a plan' }, { status:
403 });
    }

    const body = (await request.json()) as { planId?: string };
    const planId = typeof body?.planId === 'string' ? body.planId : '';
    if (!VALID_PLANS.has(planId)) {
      return NextResponse.json({ error: 'Invalid plan selected' }, { status: 400 });
```

```
    }

    const now = new Date();
    const end = new Date(now);
    if (planId === 'monthly') {
      end.setMonth(end.getMonth() + 1);
    } else {
      end.setFullYear(end.getFullYear() + 1);
    }

    (user as { subscriptionType?: string }).subscriptionType = planId;
    (user as { subscriptionStart?: Date }).subscriptionStart = now;
    (user as { subscriptionEnd?: Date }).subscriptionEnd = end;
    await user.save();

    const PharmacyModel = await getPharmacyModel();
    await PharmacyModel.findOneAndUpdate(
      { pharmacistId: user._id },
      {
        subscriptionType: planId,
        isUsingService: true,
        supportsPrescriptionUpload: true,
      },
      { new: true }
    );

    return NextResponse.json({
      message: 'Plan activated successfully',
      subscriptionType: planId,
      subscriptionStart: now.toISOString(),
      subscriptionEnd: end.toISOString(),
    });
  } catch (error) {
    console.error('Subscription activation error:', error);
    return NextResponse.json({ error: 'Failed to activate plan' }, { status: 500 });
  }
}
```

[18] app/api/users/route.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';
import { getUserModel } from '@/models/User';
import { hashPassword } from '@/lib/auth';
import getPharmacyModel from '@/models/Pharmacy';

type GeocodeResult = {
  formatted_address?: string;
  geometry?: {
    location?: {
      lat?: number;
      lng?: number;
    };
  };
  address_components?: Array<{
    long_name?: string;
    short_name?: string;
    types?: string[];
  }>;
};

const INDIAN_PIN_REGEX = /^[1-9][0-9]{5}$/;
const capitalizeFirstCharacter = (value: string) => {
  const trimmed = value.trim();
  if (!trimmed) return trimmed;
  return `${trimmed.charAt(0).toUpperCase()}${trimmed.slice(1)}`;
};

type CreateUserData = {
  name: string;
  email: string;
  password: string;
  role: 'patient' | 'pharmacist';
  phone?: string;
  address?: string;
  location?: string;
  subscriptionType?: string;
  subscriptionStart?: Date;
  subscriptionEnd?: Date;
};

type Coordinates = {
  lat: number;
  lng: number;
```

```
};

type GeocodedIndiaResult = {
  city: string;
  formattedAddress: string;
  coordinates: Coordinates;
};

const geocodeIndianLocation = async (rawLocation: string): Promise<GeocodedIndiaResult | null>
=> {
  const location = rawLocation.trim();
  if (!location) return null;

  const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
  if (!apiKey) return null;

  const query = INDIAN_PIN_REGEX.test(location) ? `${location}, India` : location;
  const response = await fetch(
    `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURIComponent(
      query
    )}&components=country:IN&key=${apiKey}`
  );

  if (!response.ok) return null;
  const payload = (await response.json()) as { results?: GeocodeResult[]; status?: string };
  if (payload.status !== 'OK' || !payload.results?.length) return null;

  const firstResult = payload.results[0];
  const components = firstResult.address_components || [];
  const country = components.find((component) => component.types?.includes('country'));
  const isIndia = country?.short_name === 'IN' || country?.long_name?.toLowerCase() ===
'india';
  if (!isIndia) return null;

  const cityComponent = components.find(
    (component) =>
      component.types?.includes('locality') ||
      component.types?.includes('postal_town') ||
      component.types?.includes('administrative_area_level_2') ||
      component.types?.includes('administrative_area_level_3')
  );

  const lat = firstResult.geometry?.location?.lat;
  const lng = firstResult.geometry?.location?.lng;
  if (typeof lat !== 'number' || typeof lng !== 'number') return null;

  return {
    city: cityComponent?.long_name?.trim() || location,
    formattedAddress: firstResult.formatted_address?.trim() || '',
    coordinates: { lat, lng },
  };
};

const reverseGeocodeIndianCoordinates = async (
  coordinates: Coordinates
): Promise<{ city: string; formattedAddress: string } | null> => {
  const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
  if (!apiKey) return null;

  const response = await fetch(
    `https://maps.googleapis.com/maps/api/geocode/json?
latlng=${coordinates.lat},${coordinates.lng}&key=${apiKey}`
  );

  if (!response.ok) return null;
  const payload = (await response.json()) as { results?: GeocodeResult[]; status?: string };
  if (payload.status !== 'OK' || !payload.results?.length) return null;

  const firstResult = payload.results[0];
  const components = firstResult.address_components || [];
  const country = components.find((component) => component.types?.includes('country'));
  const isIndia = country?.short_name === 'IN' || country?.long_name?.toLowerCase() ===
'india';
  if (!isIndia) return null;

  const cityComponent = components.find(
    (component) =>
      component.types?.includes('locality') ||
      component.types?.includes('postal_town') ||
      component.types?.includes('administrative_area_level_2') ||
      component.types?.includes('administrative_area_level_3')
  );

  const city = cityComponent?.long_name?.trim() || '';
  if (!city) return null;

  return {
```

```
    city,
    formattedAddress: firstResult.formatted_address?.trim() || '',
  };
};

export async function POST(request: NextRequest) {
  try {
    const {
      name,
      email,
      password,
      role,
      phone,
      address,
      location,
      subscriptionType,
      pharmacyCoordinates,
    } = await request.json();

    if (!name || !email || !password || !role) {
      return NextResponse.json({ error: 'Name, email, password, and role are required' }, {
status: 400 });
    }

    if (!['patient', 'pharmacist'].includes(role)) {
      return NextResponse.json({ error: 'Invalid role specified' }, { status: 400 });
    }

    const normalizedName =
      role === 'pharmacist' ? capitalizeFirstCharacter(name) : name;

    const hashedPassword = await hashPassword(password);

    const UserModel = await getUserModel(role as 'patient' | 'pharmacist');

    // Prepare user data based on role
    const userData: CreateUserData = {
      name: normalizedName,
      email,
      password: hashedPassword,
      role: role as 'patient' | 'pharmacist',
      phone,
      address,
    };
    let pharmacyCoordinatesForCreate: Coordinates = { lat: 28.6139, lng: 77.209 };

    // Add role-specific fields
    if (role === 'pharmacist') {
      let pharmacyLocation = '';
      let pharmacyAddress = typeof address === 'string' ? address.trim() : '';
      const hasCoordinates =
        pharmacyCoordinates &&
        typeof pharmacyCoordinates === 'object' &&
        typeof (pharmacyCoordinates as { lat?: unknown }).lat === 'number' &&
        typeof (pharmacyCoordinates as { lng?: unknown }).lng === 'number';

      if (hasCoordinates) {
        pharmacyCoordinatesForCreate = {
          lat: (pharmacyCoordinates as { lat: number }).lat,
          lng: (pharmacyCoordinates as { lng: number }).lng,
        };
        const reverseData = await
reverseGeocodeIndianCoordinates(pharmacyCoordinatesForCreate);
        if (!reverseData) {
          return NextResponse.json(
            { error: 'Map-selected pharmacy location must be inside India' },
            { status: 400 }
          );
        }
        pharmacyLocation = reverseData.city;
        if (!pharmacyAddress) {
          pharmacyAddress = reverseData.formattedAddress;
        }
      } else {
        if (!location || typeof location !== 'string' || !location.trim()) {
          return NextResponse.json(
            { error: 'Service location is required for pharmacists' },
            { status: 400 }
          );
        }
        const geocoded = await geocodeIndianLocation(location);
        if (!geocoded) {
          return NextResponse.json(
            { error: 'Only valid Indian locations are allowed for pharmacist registration' },
            { status: 400 }
          );
        }
```

```
        pharmacyLocation = geocoded.city;
        pharmacyCoordinatesForCreate = geocoded.coordinates;
      }

      if (!pharmacyAddress) {
        return NextResponse.json(
          { error: 'Pharmacy address is required. Add it manually or pick from map.' },
          { status: 400 }
        );
      }

      userData.location = pharmacyLocation;
      userData.address = pharmacyAddress;
      if (subscriptionType) {
        userData.subscriptionType = subscriptionType;
        userData.subscriptionStart = new Date();
        const subscriptionEnd = new Date();
        if (subscriptionType === 'monthly') {
          subscriptionEnd.setMonth(subscriptionEnd.getMonth() + 1);
        } else if (subscriptionType === 'yearly' || subscriptionType === 'premium') {
          subscriptionEnd.setFullYear(subscriptionEnd.getFullYear() + 1);
        }
        userData.subscriptionEnd = subscriptionEnd;
      }
    }

    const user = new UserModel(userData);
    await user.save();

    // If registering as pharmacist, also create pharmacy entry
    if (role === 'pharmacist') {
      try {
        const PharmacyModel = await getPharmacyModel();
        const pharmacyData = {
          pharmacistId: user._id,
          name: normalizedName,
          email,
          phone,
          address: userData.address,
          location: userData.location,
          coordinates: pharmacyCoordinatesForCreate,
          supportsPrescriptionUpload: subscriptionType ? true : false,
          isUsingService: subscriptionType ? true : false,
          subscriptionType,
          rating: 0,
          reviewCount: 0,
          services: []
        };

        const pharmacy = new PharmacyModel(pharmacyData);
        await pharmacy.save();
      } catch (pharmacyError) {
        console.error('Error creating pharmacy entry:', pharmacyError);
        // Don't fail the registration if pharmacy creation fails
      }
    }

    return NextResponse.json({ message: 'User created successfully', id: user._id }, { status:
201 });
  } catch (error: unknown) {
    const errorCode =
      typeof error === 'object' && error !== null && 'code' in error
        ? (error as { code?: unknown }).code
        : undefined;

    if (errorCode === 11000) {
      return NextResponse.json({ error: 'Email already exists' }, { status: 400 });
    }
    console.error('User creation error:', error);
    return NextResponse.json({ error: 'Failed to create user' }, { status: 500 });
  }
}

export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url);
    const role = searchParams.get('role');
    const location = searchParams.get('location');

    if (!role) {
      return NextResponse.json({ error: 'Role parameter is required' }, { status: 400 });
    }

    if (!['patient', 'pharmacist'].includes(role)) {
      return NextResponse.json({ error: 'Invalid role specified' }, { status: 400 });
    }
```

```
    const UserModel = await getUserModel(role as 'patient' | 'pharmacist');

    const query: Record<string, unknown> = {};
    if (location) query.location = { $regex: location, $options: 'i' };

    const users = await UserModel.find(query);

    return NextResponse.json(users);
  } catch (error) {
    console.error('User fetch error:', error);
    return NextResponse.json({ error: 'Failed to fetch users' }, { status: 500 });
  }
}
```

[19] app/dashboard/page.tsx FILE

```
'use client';

import NotificationComponent from '@/components/NotificationComponent';
import { useRouter } from 'next/navigation';
import { useCallback, useEffect, useMemo, useState } from 'react';

interface Prescription {
  _id: string;
  patientName: string;
  patientEmail: string;
  patientPhone: string;
  patientAddress: string;
  prescriptionImages: string[];
  notes?: string;
  status: string;
  location: string;
  createdAt: string;
  pharmacyStatuses?: PharmacyStatus[];
}

interface PharmacyStatus {
  pharmacyId?: string;
  status: 'pending' | 'accepted' | 'rejected' | 'fulfilled' | 'fulfillment_requested';
  availabilityResponse?: 'not_available' | 'same_medicine_available' |
'same_salt_different_company' | null;
  pharmacistMessage?: string;
  assignedAt?: string;
  completedAt?: string;
}

const STORAGE_NOTIFICATIONS_KEY = 'pharmacy_notifications';
const STORAGE_UNREAD_KEY = 'pharmacy_notifications_unread';
const AVAILABILITY_OPTIONS = [
  { value: 'not_available', label: 'Medicine not available' },
  { value: 'same_medicine_available', label: 'Same medicine available' },
  { value: 'same_salt_different_company', label: 'Same salt, different company available' },
] as const;
type AvailabilityOptionValue = typeof AVAILABILITY_OPTIONS[number]['value'];
const AVAILABILITY_LABELS: Record<AvailabilityOptionValue, string> = {
  not_available: 'Medicine not available',
  same_medicine_available: 'Same medicine available',
  same_salt_different_company: 'Same salt, different company available',
};

export default function DashboardPage() {
  const router = useRouter();
  const [prescriptions, setPrescriptions] = useState<Prescription[]>([]);
  const [location, setLocation] = useState('');
  const [pharmacyId, setPharmacyId] = useState('');
  const [message, setMessage] = useState('');
  const [acceptingPrescription, setAcceptingPrescription] = useState<Prescription | null>
(null);
  const [availabilityResponse, setAvailabilityResponse] = useState<AvailabilityOptionValue>
('same_medicine_available');
  const [customMessage, setCustomMessage] = useState('');
  const [isSubmittingAccept, setIsSubmittingAccept] = useState(false);

  const fetchPrescriptions = useCallback(async (targetLocation: string) => {
    if (!targetLocation.trim()) {
      setPrescriptions([]);
      return;
    }
    try {
      const response = await fetch(`/api/prescriptions?
location=${targetLocation}&status=all`);
      const data = await response.json();
      setPrescriptions(data);
```

```
    } catch {
      console.error('Failed to fetch prescriptions');
    }
  }, []);

  useEffect(() => {
    const loadPharmacistLocation = async () => {
      try {
        const response = await fetch('/api/auth/me');
        if (!response.ok) {
          router.push('/login');
          return;
        }
        const data = await response.json();
        if (data?.user?.role !== 'pharmacist') {
          router.push('/upload');
          return;
        }
        const pharmacistLocation = data?.user?.location || '';
        const pharmacistPharmacyId = data?.user?.pharmacyId || '';
        setPharmacyId(pharmacistPharmacyId);
        if (pharmacistLocation) {
          setLocation(pharmacistLocation);
          fetchPrescriptions(pharmacistLocation);
        }
      } catch (error) {
        console.error('Failed to load pharmacist profile:', error);
        router.push('/login');
      }
    };

    loadPharmacistLocation();
  }, [fetchPrescriptions, router]);

  const handleNotificationRefresh = useCallback(() => {
    fetchPrescriptions(location);
  }, [fetchPrescriptions, location]);

  const updatePrescriptionStatus = async (
    id: string,
    status: 'assigned' | 'request_fulfillment',
    options?: { availabilityResponse?: AvailabilityOptionValue; customMessage?: string }
  ) => {
    try {
      const response = await fetch(`/api/prescriptions/${id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          status,
          availabilityResponse: options?.availabilityResponse,
          customMessage: options?.customMessage,
        }),
      });
      const payload = await response.json().catch(() => ({}));
      if (response.ok) {
        const targetPrescription = prescriptions.find((item) => item._id === id);
        if (targetPrescription) {
          try {
            const nextStatus = status === 'request_fulfillment' ? 'fulfillment_requested' :
'accepted';
            const notification = {
              prescription: {
                id: targetPrescription._id,
                patientName: targetPrescription.patientName,
                location: targetPrescription.location,
                createdAt: new Date().toISOString(),
                status: nextStatus,
              },
            };
            const raw = localStorage.getItem(STORAGE_NOTIFICATIONS_KEY);
            const existing = raw ? (JSON.parse(raw) as typeof notification[]) : [];
            const merged = [notification, ...existing].slice(0, 100);
            localStorage.setItem(STORAGE_NOTIFICATIONS_KEY, JSON.stringify(merged));
            const unreadCount = Number(localStorage.getItem(STORAGE_UNREAD_KEY) || '0');
            localStorage.setItem(STORAGE_UNREAD_KEY, String(unreadCount + 1));
            window.dispatchEvent(new Event('notification-updated'));
          } catch {
            // Do not block status update flow if local notifications fail.
          }
        }
        setMessage(
          status === 'request_fulfillment'
            ? 'Patient confirmation requested for fulfillment.'
            : `Prescription accepted: ${AVAILABILITY_LABELS[(options?.availabilityResponse ||
'same_medicine_available') as AvailabilityOptionValue]}`
        );
        fetchPrescriptions(location);
```

```
        return true;
      } else {
        setMessage(payload?.error || 'Failed to update prescription.');
      }
    } catch {
      setMessage('Error updating prescription.');
    }
    return false;
  };

  const openAcceptModal = (prescription: Prescription) => {
    setAvailabilityResponse('same_medicine_available');
    setCustomMessage('');
    setAcceptingPrescription(prescription);
  };

  const closeAcceptModal = () => {
    if (isSubmittingAccept) return;
    setAcceptingPrescription(null);
  };

  const submitAcceptDecision = async () => {
    if (!acceptingPrescription) return;
    setIsSubmittingAccept(true);
    const success = await updatePrescriptionStatus(acceptingPrescription._id, 'assigned', {
      availabilityResponse,
      customMessage,
    });
    setIsSubmittingAccept(false);
    if (success) {
      setAcceptingPrescription(null);
    }
  };

  const analytics = useMemo(() => {
    const getOwnStatus = (item: Prescription) =>
      item.pharmacyStatuses?.find((entry) => String(entry.pharmacyId) === pharmacyId)?.status;

    const ownPrescriptions = prescriptions.filter((item) =>
      item.pharmacyStatuses?.some((entry) => String(entry.pharmacyId) === pharmacyId)
    );

    const totalRequests = ownPrescriptions.length;
    const completedPrescriptions = ownPrescriptions.filter((item) => getOwnStatus(item) ===
'fulfilled').length;
    const activePrescriptions = ownPrescriptions.filter((item) => getOwnStatus(item) !==
'fulfilled').length;
    const uniquePatients = new Set(
      ownPrescriptions
        .filter((item) => {
          const ownStatus = getOwnStatus(item);
          return ownStatus === 'accepted' || ownStatus === 'fulfillment_requested' ||
ownStatus === 'fulfilled';
        })
        .map((item) => item.patientEmail?.toLowerCase() || item.patientName.toLowerCase())
    ).size;
    const today = new Date();
    const todayRequests = ownPrescriptions.filter((item) => {
      const created = new Date(item.createdAt);
      return (
        created.getDate() === today.getDate() &&
        created.getMonth() === today.getMonth() &&
        created.getFullYear() === today.getFullYear()
      );
    }).length;

    return {
      totalRequests,
      completedPrescriptions,
      activePrescriptions,
      uniquePatients,
      todayRequests,
    };
  }, [prescriptions, pharmacyId]);

  const ownPrescriptions = useMemo(
    () =>
      prescriptions
        .filter((item) => item.pharmacyStatuses?.some((entry) => String(entry.pharmacyId) ===
pharmacyId))
        .sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()),
    [prescriptions, pharmacyId]
  );

  const recentActivity = useMemo(
    () =>
      ownPrescriptions.slice(0, 8),
```

```
    [ownPrescriptions]
  );

  const getOwnPharmacyStatus = (prescription: Prescription) =>
    prescription.pharmacyStatuses?.find((entry) => String(entry.pharmacyId) ===
pharmacyId)?.status || 'pending';
  const getOwnPharmacyStatusEntry = (prescription: Prescription) =>
    prescription.pharmacyStatuses?.find((entry) => String(entry.pharmacyId) === pharmacyId);

  const formatOwnStatus = (status: string) =>
    status.charAt(0).toUpperCase() + status.slice(1).replace('_', ' ');

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-7xl">
        <div className="mb-6 rounded-3xl border border-white/10 bg-slate-900/80 p-8">
          <p className="inline-flex rounded-full border border-emerald-300/30 bg-emerald-
300/10 px-3 py-1 text-xs font-semibold uppercase tracking-[0.14em] text-emerald-100">
            Pharmacist Workspace
          </p>
          <h1 className="mt-4 text-3xl font-bold text-white sm:text-4xl">Dashboard</h1>
          <p className="mt-2 text-slate-300">
            Enter your location to monitor pending prescriptions and assign requests in real
time.
          </p>
        </div>

        <div className="rounded-3xl border border-white/10 bg-slate-900/80 p-6">
          <NotificationComponent
            currentLocation={location}
            onNewPrescription={handleNotificationRefresh}
          />

          <div className="mt-5">
            <label className="mb-2 block text-sm font-medium text-slate-200">Location
(City/ZIP)</label>
            <input
              type="text"
              placeholder="Enter your service location"
              value={location}
              onChange={(e) => {
                const value = e.target.value;
                setLocation(value);
                fetchPrescriptions(value);
              }}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
            />
          </div>

          {message && <p className="mt-4 rounded-xl bg-white/10 px-4 py-3 text-sm text-slate-
100">{message}</p>}

          <div className="mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-5">
            <div className="rounded-2xl border border-cyan-300/30 bg-cyan-300/10 p-4">
              <p className="text-xs uppercase tracking-[0.1em] text-cyan-100">Total
Activity</p>
              <p className="mt-2 text-3xl font-bold text-white">{analytics.totalRequests}</p>
            </div>
            <div className="rounded-2xl border border-emerald-300/30 bg-emerald-300/10 p-4">
              <p className="text-xs uppercase tracking-[0.1em] text-emerald-100">Completed</p>
              <p className="mt-2 text-3xl font-bold text-white">
{analytics.completedPrescriptions}</p>
            </div>
            <div className="rounded-2xl border border-amber-300/30 bg-amber-300/10 p-4">
              <p className="text-xs uppercase tracking-[0.1em] text-amber-100">Open</p>
              <p className="mt-2 text-3xl font-bold text-white">
{analytics.activePrescriptions}</p>
            </div>
            <div className="rounded-2xl border border-fuchsia-300/30 bg-fuchsia-300/10 p-4">
              <p className="text-xs uppercase tracking-[0.1em] text-fuchsia-100">Patients
Visited</p>
              <p className="mt-2 text-3xl font-bold text-white">{analytics.uniquePatients}</p>
            </div>
            <div className="rounded-2xl border border-indigo-300/30 bg-indigo-300/10 p-4">
              <p className="text-xs uppercase tracking-[0.1em] text-indigo-100">Today</p>
              <p className="mt-2 text-3xl font-bold text-white">{analytics.todayRequests}</p>
            </div>
          </div>

          <div className="mt-6 rounded-2xl border border-white/10 bg-slate-950/60 p-4">
            <h2 className="text-lg font-semibold text-white">Recent Activity</h2>
            {recentActivity.length === 0 ? (
              <p className="mt-2 text-sm text-slate-300">No activity yet for this location.
</p>
            ) : (
              <div className="mt-3 space-y-2">
```

```
                {recentActivity.map((item) => (
                  <div key={item._id} className="flex items-center justify-between rounded-xl
border border-white/10 bg-slate-900/70 p-3">
                    <div>
                      <p className="text-sm text-white">{item.patientName}</p>
                      <p className="text-xs text-slate-300">{item.location}</p>
                    </div>
                    <div className="text-right">
                      <p className="text-xs uppercase text-slate-200">
{formatOwnStatus(getOwnPharmacyStatus(item))}</p>
                      <p className="text-xs text-slate-400">{new
Date(item.createdAt).toLocaleString()}</p>
                    </div>
                  </div>
                ))}
              </div>
            )}
          </div>

          <div className="mt-6 grid grid-cols-1 gap-4 lg:grid-cols-2 xl:grid-cols-3">
            {ownPrescriptions.map((prescription) => (
              <article key={prescription._id} className="rounded-2xl border border-white/10
bg-slate-950/70 p-5">
                <h2 className="text-lg font-semibold text-white">{prescription.patientName}
</h2>
                <div className="mt-3 space-y-1 text-sm text-slate-200">
                  <p><strong>Email:</strong> {prescription.patientEmail}</p>
                  <p><strong>Phone:</strong> {prescription.patientPhone || 'Not provided'}</p>
                  <p><strong>Address:</strong> {prescription.patientAddress || 'Not provided'}
</p>
                  <p><strong>Location:</strong> {prescription.location}</p>
                  <p><strong>Status:</strong>
{formatOwnStatus(getOwnPharmacyStatus(prescription))}</p>
                  {getOwnPharmacyStatus(prescription) === 'accepted' && (
                    <p>
                      <strong>Availability:</strong>{' '}
                      {(() => {
                        const ownEntry = prescription.pharmacyStatuses?.find(
                          (entry) => String(entry.pharmacyId) === pharmacyId
                        );
                        const key = ownEntry?.availabilityResponse as AvailabilityOptionValue
| undefined;
                        return key ? AVAILABILITY_LABELS[key] : 'Not shared';
                      })()}
                    </p>
                  )}
                  <p><strong>Date:</strong> {new
Date(prescription.createdAt).toLocaleDateString()}</p>
                </div>

                {prescription.notes && (
                  <div className="mt-4 rounded-xl bg-cyan-300/10 px-3 py-2 text-sm text-cyan-
100">
                    <strong>Notes:</strong> {prescription.notes}
                  </div>
                )}

                {prescription.prescriptionImages.length > 0 && (
                  <div className="mt-4 grid grid-cols-2 gap-3">
                    {prescription.prescriptionImages.map((image, index) => (
                      <button
                        type="button"
                        key={`${prescription._id}-${index}`}
                        onClick={() => window.open(image, '_blank')}
                        className="group relative overflow-hidden rounded-xl border border-
white/10"
                      >
                        <img
                          src={image}
                          alt={`Prescription ${index + 1}`}
                          className="h-28 w-full object-cover transition group-hover:scale-
105"
                        />
                        <span className="absolute bottom-2 right-2 rounded-md bg-black/60 px-2
py-1 text-xs text-white">
                          {index + 1}
                        </span>
                      </button>
                    ))}
                  </div>
                )}

                <button
                  onClick={() => {
                    if (getOwnPharmacyStatus(prescription) === 'pending') {
                      openAcceptModal(prescription);
                      return;
```

```
                    }
                    const ownEntry = getOwnPharmacyStatusEntry(prescription);
                    updatePrescriptionStatus(prescription._id, 'request_fulfillment', {
                      availabilityResponse:
                        (ownEntry?.availabilityResponse as AvailabilityOptionValue |
undefined) ||
                        'same_medicine_available',
                      customMessage: ownEntry?.pharmacistMessage || '',
                    });
                  }}
                  disabled={getOwnPharmacyStatus(prescription) === 'fulfilled' ||
getOwnPharmacyStatus(prescription) === 'fulfillment_requested'}
                  className="mt-4 w-full rounded-xl bg-emerald-300 px-4 py-3 font-semibold
text-slate-900 transition hover:bg-emerald-200 disabled:cursor-not-allowed disabled:opacity-
60"
                >
                  {getOwnPharmacyStatus(prescription) === 'fulfilled'
                    ? 'Completed'
                    : getOwnPharmacyStatus(prescription) === 'fulfillment_requested'
                      ? 'Confirmation Requested'
                      : getOwnPharmacyStatus(prescription) === 'accepted'
                      ? 'Request Patient Confirmation'
                      : 'Accept Prescription'}
                </button>
              </article>
            ))}
          </div>
        </div>
      </div>

      {acceptingPrescription && (
        <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 p-4">
          <div className="w-full max-w-lg rounded-2xl border border-white/10 bg-slate-900 p-
6">
            <h2 className="text-xl font-semibold text-white">Accept Prescription</h2>
            <p className="mt-1 text-sm text-slate-300">
              Patient: {acceptingPrescription.patientName}
            </p>

            <label className="mt-5 block text-sm font-medium text-slate-200">Medicine
availability</label>
            <select
              value={availabilityResponse}
              onChange={(e) => setAvailabilityResponse(e.target.value as
AvailabilityOptionValue)}
              className="mt-2 w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
            >
              {AVAILABILITY_OPTIONS.map((option) => (
                <option key={option.value} value={option.value}>
                  {option.label}
                </option>
              ))}
            </select>

            <label className="mt-4 block text-sm font-medium text-slate-200">
              Custom message (optional)
            </label>
            <textarea
              value={customMessage}
              onChange={(e) => setCustomMessage(e.target.value)}
              maxLength={500}
              rows={4}
              placeholder="Write a custom message for the patient..."
              className="mt-2 w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
            />
            <p className="mt-1 text-xs text-slate-400">{customMessage.length}/500</p>

            <div className="mt-5 flex gap-3">
              <button
                type="button"
                onClick={closeAcceptModal}
                disabled={isSubmittingAccept}
                className="flex-1 rounded-xl border border-white/20 px-4 py-2.5 text-sm font-
semibold text-slate-200 transition hover:bg-white/10 disabled:cursor-not-allowed
disabled:opacity-60"
              >
                Cancel
              </button>
              <button
                type="button"
                onClick={submitAcceptDecision}
                disabled={isSubmittingAccept}
                className="flex-1 rounded-xl bg-emerald-300 px-4 py-2.5 text-sm font-semibold
text-slate-900 transition hover:bg-emerald-200 disabled:cursor-not-allowed disabled:opacity-
60"
```

```
              >
                {isSubmittingAccept ? 'Saving...' : 'Accept & Send'}
              </button>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
```

[20] app/favicon.ico FILE


```
    ( F    ( n 00  (–      ¾F(    $]ºº]$  òÿÿÿÿò
8àÿÿÿÿÿÿÿÿà8 âÿÿÿÿÿÿÿÿÿÿâ iÿÿÿÿÿÿÿÿÿÿÿÿi#ôÿÿOOOÿ®®®ÿ«««ÿ«««ÿ«««ÿ«««ÿ-
ÿgggÿÿÿÿô#Yÿÿÿÿ  ÿíííÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿýýý555ÿÿÿÿY»ÿÿÿÿkkkÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿ»»ÿÿÿÿ
ÿÎÎÎÿÿÿÿÿÿÿÿÿÿäääÿ  ÿÿÿÿÿ»YÿÿÿÿÿJJJÿýýýÿÿÿÿÿÿkkkÿÿÿÿÿÿY#ôÿÿÿÿ  ÿ¶¶¶ÿÔÔÔÿ
ÿÿÿÿÿô#iÿÿÿÿÿ111ÿDDDÿÿÿÿÿÿi âÿÿÿÿÿÿÿÿÿÿâ 8àÿÿÿÿÿÿÿÿà8  òÿÿÿÿò  $]ºº]$( @   ,U  èè  U, *
ÙùÿÿÿÿÿÿùÙ
*  óÿÿÿÿÿÿÿÿÿÿÿÿó  QáÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿáQrûÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿûrrÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿrOûÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿûO äÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿä  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  (ôÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿô'  ÿÿÿÿÿÿ888ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ     ÿ
   ÿ     ÿ___ÿÿÿÿÿÿÿ  Ôÿÿÿÿÿÿ  ÿîîîÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿSSSÿÿÿ
ÿÿÿÿÔ +úÿÿÿÿÿÿÿhhhÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ®®®ÿÿÿÿÿÿÿÿú+TÿÿÿÿÿÿÿÿÿËËËÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿôôôÿ,,,ÿÿÿÿÿÿÿÿT  ÿÿÿÿÿÿÿÿGGGÿýýýÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿ  éÿÿÿÿÿÿÿÿÿÿ-
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿäääÿ  ÿÿÿÿÿÿÿÿÿéèÿÿÿÿÿÿÿÿÿÿ+++ÿóóóÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿjjjÿÿÿÿÿÿÿÿÿÿÿÿé  ÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÌÌÌÿÿÿÿÿÿÿÿÿÿÿÿ  Tÿÿÿÿÿÿÿÿÿÿ  ÿäää
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿýýýIIIÿÿÿÿÿÿÿÿÿÿÿÿT+úÿÿÿÿÿÿÿÿÿÿÿhhhÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ¯¯¯ÿ  ÿÿÿÿÿÿÿÿÿÿÿú+ Ôÿÿÿÿ
ÿÿÿÿÿÿÿÿËËËÿÿÿÿÿÿÿÿÿÿôôôÿ,,,ÿÿÿÿÿÿÿÿÿÿÿÿÔ  ÿÿÿÿÿÿÿÿÿÿÿÿGGGÿýýýÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿ  'ôÿÿÿÿÿÿ
ÿÿÿÿÿ±±
±ÿìììÿ  ÿÿÿÿÿÿÿÿÿÿÿÿõ(  ÿÿÿÿÿÿÿÿÿÿÿ333ÿ___ÿÿÿÿÿÿÿÿÿÿÿÿ   äÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿä Oûÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿûOrÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿrrûÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿûrQáÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿáQ  óÿÿÿÿÿÿÿÿ
ÿÿÿó  *  ÙùÿÿÿÿÿÿùÙ  * ,U  èè  U, (0`  –   (Lj³ø÷³jK(
 V ØøÿÿÿÿÿÿÿÿøØ U %  áÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿá  &   ïÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿi   QÜÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÜR
  þÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿþ   -
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  ¸ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ¸  ®ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿ®  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  PýÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿýO ßÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿß  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  #ñÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿñ#  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ   äÿÿÿÿÿÿÿÿÿ$$$ÿhhhÿeeeÿ
eeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿeeeÿPPPÿ  ÿÿÿÿÿÿÿÿ
ä Uÿÿÿÿÿÿÿÿÿÿ  ÿëëëÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿsssÿÿÿÿÿÿÿÿÿÿÿUiÿÿÿÿÿÿÿÿÿÿÿeeeÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÌÌÌÿÿÿÿÿÿÿÿÿÿÿÿi
Úÿÿÿÿÿÿÿÿÿÿÿ  ÿÉÉÉÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿýýý
HHHÿÿÿÿÿÿÿÿÿÿÿÿÚ
(ùÿÿÿÿÿÿÿÿÿÿÿEEEÿüüüÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ®®®ÿ
ÿÿÿÿÿÿÿÿÿÿÿÿø(Kÿÿÿÿÿÿÿÿÿÿÿÿÿªªªÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿôôôÿ,,,ÿÿÿÿÿÿÿÿÿÿÿÿÿLjÿÿÿÿÿÿÿÿÿÿÿÿÿ)))ÿòòòÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿj
´ÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿäääÿ  ÿÿÿÿÿÿÿÿÿ
ÿÿÿÿ³øÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  ÿáááÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿiiiÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿøøÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿeeeÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿËËËÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ø³ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  ÿÉÉÉÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿýýýHHHÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
´jÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿEEEÿüüüÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ®®®ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿjLÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿªªªÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿôôôÿ,,,ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿK(øÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ)))
ÿòòòÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿù(
Úÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ     ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿäääÿ  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÚ
iÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  ÿáááÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿiiiÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿiUÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿeeeÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÌÌÌÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿU äÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  ÿÉÉÉÿÿÿÿÿÿÿÿÿÿÿÿÿÿýýýHHHÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿä   ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿEEEÿüüüÿÿÿÿ®®®ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ  #ñÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ¬¬¬ÿûûÿ,,,
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿñ#  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ222ÿ}}}
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ   ßÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿß Oýÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
ÿÿÿÿÿÿP  ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ   ®ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ® ¸ÿÿÿÿÿÿÿ
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ¸  -
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ   þÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿþ   RÜÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÜQ   ï
ÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿi   &  áÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿá  % U Øøÿÿÿÿÿÿÿÿ øØ V         (Kj³÷ø³jL(        PNG


IHDR  \r¨f sRGB®Î é8eXIfMM*  i          D"8s IDATx í]      ° Õ  n  ]< QVA  èÄh$
ÊN    13*  q°Âdª Ä  ©Ii     D  L2  ª(Î (Ô  2ÖÄ  Ñ G
  Áq_@å±  à      xÈ  ï»Ð  ¾  {o÷½½  ÓýÿUß{}»O  å;çÿûôùÏ9ÝÂ  d    ®(Dg  Ð 8     è ô Nº ]€î@   h x¥ ?v
ÀNà3à=`;ð6ð.ð&° àuâ à   ±   6   P  ©Ð½  Á@ÿÃ  RÓPùi Z qÊ^DNãà   wp¼
Ø¼ ÐXÐhÐ   Hg@ ÀÌ
: Ùâ |ð5` p"@ À'¼É²   s{
ëpü*ð2ÀÞ     Ä  dÒ¯     È  |(0 ø
0à  >Kò
³ x X ¬ 6  IJÈ¤C|?$KE  N Ø}Ï  |  Âòµá à  ö h $         2   Ù|/§Â  .N z  #¼  ÃW  eÀ
à5€ã
          Ü¶  úà;Ây  ¾  ñgàs ©   h ^ IÄÈDL(¢; ¸ 8  Ò  Hjg€c H|x  1 ÈR"   a€ïôÓ    GÁÙ@  è9`/`%0 è
HÄ@jð½~,° ÛK
  , t). Î èI      D è  T¦ O û )~º ° V ì  u $b   èª
ÐU%¥7       ¨  ù_É$b   8AçxÀ    ß  øJö3`    510 wQñ? ¤øvrðÑá:ü2þKÄ@  ¤øv*
```

```
÷ þ 1` £¯ ÉYÐ ãâ " Dª0b@¤ÆYö 9¬¦ Àðý¾[tá F1À ôp`kÐ\UÓ` ÜRø·A#W81
 e`)RÛZM Õö÷ [uæ F0ð rq.ÀÕ ¥Ú#^ð=C"Ä 9P'«R~f½ ï
pn€z dC" Òe ¦§?³\K€÷½¨@&$b }jzú3Ûµöx/{ èÄ 1 Ra€#ÿ|÷çÆ UK=&Ä^€æ TM¹n 2À9ÿ5)?
s¦ Àû{0'ó D Dà ßo[kMÕoK0¿xÿ ÜTd _@] b r æ ÆGÃÄ¶À´;« ¯ò D DàêÜ1ÀgaR Û`ÚÜ'` 0 Â
 >\ ë/ù ñ½ f Ð ù øñ ÷ Å ß!fn÷Z¢|b ÜðU .t® å à¤ ÑÀÚr ª9 Å+ ý ±î ý b
r nE D k =æé8à ÀÙü! b R Clã P Þ EÙ`¼Ü K '~ß@ Ë}*°!`ø@Á¢6Lô
 ;À $b@DÃÀ?# ÈgÞFý
à µ V 1 výÇ ;ÃEsôYQ¹ ÇÌ=É® ö4¿ ßb@TÂÀn þ! ¹ò3 q¾ 0^ÊV ãc µ1ð Ü¶
´¶[ýï ë ÀM¹=8I Ý Õ1@ Ú @Cu· · `N¸oÀãW JÄ ÆWúÑâÇe÷±I ¯n¦¹Nàmà¯ ¸Æ Ü´ô_d (Ä4`EÜ
IÂ°¯ "ÌµË 1 *3ð+\þEå Ñ\Mâ ÀÍ)g rÁ ¦
» è¿ 8 >õ¿ pô?vIÒ°0 Ç ~ü! b øÅ $'Ó%"I¿ ¿ RýÀi 1 0° ?S~& Ô Ór ä Æ ¤{
n ô_ ÄÀ ÈÌLà? ÌT eäÔÆ À7ùC" rÎÀOQ~"qIË° 0 8 ?$b §Ü rÿ#@x_â v¼JÌ £/¹ é3ð'dá/ ÈÖ÷W[¤
¤ ço'Nü
ðló ú- 2ÎÛüå@jÊO~Ôî 0 À2` HÄ@Æ Ø ò Ð+ ª¤Ý p OB®ÞuOè¿ È( lãS Ô ü¦9Èô½ò ~ðc ß :x/
èXd .ðù °Æ ò dEÉVày@F$H2Â ïü+M* i Ül 8O@F$H2Á å®24& r´
POÀ´Ö¢üÔÂ€ûä7NùY S
ó¶ ø ÀY 1` ðã;ÿJS3n² g[ ' @W@" la`32Ên?'ûHB2p
î h Ä m mu ×j@F@ Ü V ²Z!¦¿xIä ÉýH® y Ñ±)ü ù>À åZ!6 º à aÎ`àþ ¦ÌdDV$9f èù
pM¿6»I¨!LG:\LdrwPyË~ýPá§%Ôæ L3 Æá7çTKÑÄA m mo|³6© 3Ð é- ÒhJ3¿¾? 67
á¶ yr¶ " û þ àgÈÎ 4. $±1À÷ý_ ¼ [* ¸& ¸¤ S/õd q´ Ìiãä ÁCÞ âh 3¿¾> 6Å¶%± ¶Èã\ #´RZq¦
ð=lK|Å XÆÀXäWS ej5/iýÐÓ$± :äýv@½ Î8Ð×
 d 1( z 2~Fà)´ùö3¢ôÎ ÁÞÀl ®C¿ ëùâè#´ í= ².\Lt? % Ñ N $9b %Ê: àþ 2 ùä u
É1 |- l dòÁ÷ t$b À@?ü¥·@ »FàcÔïÏ ^ D d [9ý à z Ùà Ï:
HÄ@
¾ P2 v )~ ¦®@ àüï z5°Ç| úÒ ¿R«Öµª à Á|`# W 39Ø ºá< î"- ±0Àï \ < ì dÒå€uÀoGL z 1 Gp ° à e å
€¯d .øj
HÄ@jôFÊ3€ @ c { s<ÿùJ& É@ ÷ëöb ¸Ù Àw ²§ © nÁµ àvð ²û<
 , M; û*p>p!0 h Hü à{= »öü x à ]IÄÈDLh èÚâ<'i h 8Ç@ V Á#ïã Jÿ° àf ûI ÈÄHn
¨ûã W } àNÆt[u ò$ ¢° § å þ
@ 2 ]&) # 3 , =%¯ TôÕÀkÀ&à I ÷I¶ ®Ó³ð Ù[8 è
´L ]È ]tï Tõg õÁ6à–@b 2 U OVºã:

A?€¯
} .iî | àxC Â÷rvßw ;ÎÀ#ê >éi8 _b8 2 WP ìÑõè { ' n¯á Ó8ðz; Æ¤yÝ s Ã@¥¦ ¼ Pi· o |
ÂSùi h$ 3ðÿ@ß¹ jì áIEND®B`
```


[21] app/globals.css FILE

```css
@import "tailwindcss";

:root {
  --background: #020617;
  --foreground: #e2e8f0;
}

[data-theme='light'] {
  --background: #f8fafc;
  --foreground: #0f172a;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

body {
  min-height: 100vh;
  background: radial-gradient(circle at top, #0f172a 0%, #020617 45%, #000000 100%);
  color: var(--foreground);
  font-family: var(--font-geist-sans), "Segoe UI", sans-serif;
  text-rendering: optimizeLegibility;
}

[data-theme='light'] body {
  background: radial-gradient(circle at top, #e2e8f0 0%, #f8fafc 48%, #ffffff 100%);
}

.glass-panel {
  background: linear-gradient(
    140deg,
    rgba(15, 23, 42, 0.82) 0%,
    rgba(30, 41, 59, 0.58) 55%,
    rgba(51, 65, 85, 0.42) 100%
  );
  box-shadow: 0 16px 40px rgba(15, 23, 42, 0.35);
  backdrop-filter: blur(10px);
}

[data-theme='light'] .glass-panel {
  background: linear-gradient(
    140deg,
```

```
    rgba(255, 255, 255, 0.96) 0%,
    rgba(248, 250, 252, 0.94) 60%,
    rgba(241, 245, 249, 0.9) 100%
  );
  box-shadow: 0 14px 34px rgba(15, 23, 42, 0.08);
}

[data-theme='light'] .bg-slate-950\/95,
[data-theme='light'] .bg-slate-950\/85,
[data-theme='light'] .bg-slate-950\/80,
[data-theme='light'] .bg-slate-950\/70,
[data-theme='light'] .bg-slate-950\/60,
[data-theme='light'] .bg-slate-950\/50,
[data-theme='light'] .bg-slate-950,
[data-theme='light'] .bg-slate-900\/80,
[data-theme='light'] .bg-slate-900\/70,
[data-theme='light'] .bg-slate-900\/60,
[data-theme='light'] .bg-slate-900\/50,
[data-theme='light'] .bg-slate-900\/85,
[data-theme='light'] .bg-slate-900 {
  background-color: rgba(255, 255, 255, 0.94) !important;
}

[data-theme='light'] .bg-white\/10,
[data-theme='light'] .bg-white\/15,
[data-theme='light'] .bg-white\/5,
[data-theme='light'] .bg-white\/\[0\.04\] {
  background-color: rgba(15, 23, 42, 0.05) !important;
}

[data-theme='light'] .bg-slate-700\/70 {
  background-color: rgba(148, 163, 184, 0.2) !important;
}

[data-theme='light'] .bg-cyan-300\/10,
[data-theme='light'] .bg-cyan-300\/20 {
  background-color: rgba(14, 116, 144, 0.1) !important;
}

[data-theme='light'] .bg-emerald-300\/10,
[data-theme='light'] .bg-emerald-300\/20 {
  background-color: rgba(16, 185, 129, 0.12) !important;
}

[data-theme='light'] .bg-rose-300\/10 {
  background-color: rgba(244, 63, 94, 0.1) !important;
}

[data-theme='light'] .bg-indigo-300\/10 {
  background-color: rgba(99, 102, 241, 0.1) !important;
}

[data-theme='light'] .border-white\/10,
[data-theme='light'] .border-white\/15,
[data-theme='light'] .border-white\/20,
[data-theme='light'] .border-white\/30 {
  border-color: rgba(148, 163, 184, 0.5) !important;
}

[data-theme='light'] .border-cyan-300\/30,
[data-theme='light'] .border-cyan-300\/40 {
  border-color: rgba(14, 116, 144, 0.45) !important;
}

[data-theme='light'] .border-emerald-300\/30 {
  border-color: rgba(5, 150, 105, 0.45) !important;
}

[data-theme='light'] .border-rose-300\/30 {
  border-color: rgba(190, 24, 93, 0.4) !important;
}

[data-theme='light'] .border-slate-700,
[data-theme='light'] .border-slate-600 {
  border-color: rgba(100, 116, 139, 0.45) !important;
}

[data-theme='light'] .text-white,
[data-theme='light'] .text-slate-100 {
  color: #0f172a !important;
}

[data-theme='light'] .text-slate-200,
[data-theme='light'] .text-slate-300 {
  color: #334155 !important;
}
```

```
[data-theme='light'] .text-slate-400,
[data-theme='light'] .text-slate-500 {
  color: #64748b !important;
}

[data-theme='light'] .text-cyan-100,
[data-theme='light'] .text-cyan-200,
[data-theme='light'] .text-sky-100 {
  color: #0e7490 !important;
}

[data-theme='light'] .text-emerald-100 {
  color: #047857 !important;
}

[data-theme='light'] .text-amber-100,
[data-theme='light'] .text-amber-300,
[data-theme='light'] .text-yellow-400,
[data-theme='light'] .text-orange-100 {
  color: #92400e !important;
}

[data-theme='light'] .text-rose-100 {
  color: #be123c !important;
}

[data-theme='light'] .text-indigo-100,
[data-theme='light'] .text-violet-100 {
  color: #4338ca !important;
}

[data-theme='light'] .text-fuchsia-100,
[data-theme='light'] .text-purple-600 {
  color: #a21caf !important;
}

[data-theme='light'] .text-gray-400,
[data-theme='light'] .text-gray-500,
[data-theme='light'] .text-gray-600,
[data-theme='light'] .text-gray-700,
[data-theme='light'] .text-gray-800,
[data-theme='light'] .text-gray-900 {
  color: #374151 !important;
}

[data-theme='light'] .placeholder\:text-slate-500::placeholder {
  color: #64748b !important;
}

[data-theme='light'] .hover\:bg-white\/10:hover,
[data-theme='light'] .hover\:bg-white\/5:hover {
  background-color: rgba(15, 23, 42, 0.08) !important;
}

[data-theme='light'] .hover\:text-white:hover {
  color: #0f172a !important;
}

[data-theme='light'] .bg-black\/60 {
  background-color: rgba(15, 23, 42, 0.58) !important;
}

.text-balance {
  text-wrap: balance;
}
```

[22] app/history/page.tsx FILE

```
'use client';

import { useRouter } from 'next/navigation';
import { useEffect, useState } from 'react';

interface Prescription {
  _id: string;
  prescriptionImages: string[];
  notes: string;
  status: 'pending' | 'assigned' | 'fulfilled';
  location: string;
  createdAt: string;
  pharmacyDetails?: PharmacyDetail[];
}
```

```
interface PharmacyDetail {
  pharmacyId: string;
  name: string;
  address: string;
  location: string;
  status: 'pending' | 'accepted' | 'rejected' | 'fulfilled' | 'fulfillment_requested';
  availabilityResponse?: 'not_available' | 'same_medicine_available' |
'same_salt_different_company' | null;
  pharmacistMessage?: string;
  assignedAt?: string | null;
  completedAt?: string | null;
}

const STORAGE_PATIENT_NOTIFICATIONS_KEY = 'patient_notifications';
const STORAGE_PATIENT_UNREAD_KEY = 'patient_notifications_unread';

export default function HistoryPage() {
  const [prescriptions, setPrescriptions] = useState<Prescription[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [successMessage, setSuccessMessage] = useState('');
  const [deletingId, setDeletingId] = useState<string | null>(null);
  const [confirmingKey, setConfirmingKey] = useState<string | null>(null);
  const router = useRouter();

  useEffect(() => {
    fetchHistory();
  }, []);

  const fetchHistory = async () => {
    try {
      const authResponse = await fetch('/api/auth/me');
      if (!authResponse.ok) {
        router.push('/login');
        return;
      }

      const userData = await authResponse.json();
      if (userData.user.role !== 'patient') {
        router.push('/login');
        return;
      }

      const response = await fetch('/api/patient/history');

      if (response.ok) {
        const data = await response.json();
        setPrescriptions(data);
      } else {
        setError('Failed to load prescription history');
      }
    } catch {
      setError('Failed to load prescription history');
      router.push('/login');
    } finally {
      setLoading(false);
    }
  };

  const getStatusColor = (status: string) => {
    switch (status) {
      case 'pending':
        return 'bg-amber-300/20 text-amber-100 border-amber-300/30';
      case 'assigned':
        return 'bg-cyan-300/20 text-cyan-100 border-cyan-300/30';
      case 'fulfilled':
        return 'bg-emerald-300/20 text-emerald-100 border-emerald-300/30';
      default:
        return 'bg-white/10 text-slate-200 border-white/20';
    }
  };

  const getPharmacyStatusColor = (status: string) => {
    switch (status) {
      case 'accepted':
        return 'bg-cyan-300/20 text-cyan-100 border-cyan-300/30';
      case 'rejected':
        return 'bg-rose-300/20 text-rose-100 border-rose-300/30';
      case 'fulfilled':
        return 'bg-emerald-300/20 text-emerald-100 border-emerald-300/30';
      case 'fulfillment_requested':
        return 'bg-indigo-300/20 text-indigo-100 border-indigo-300/30';
      default:
        return 'bg-amber-300/20 text-amber-100 border-amber-300/30';
    }
  };
```

```
  const getAvailabilityLabel = (value: PharmacyDetail['availabilityResponse']) => {
    switch (value) {
      case 'not_available':
        return 'Medicine not available';
      case 'same_medicine_available':
        return 'Same medicine available';
      case 'same_salt_different_company':
        return 'Same salt, different company available';
      default:
        return '';
    }
  };

  const formatDate = (dateString: string) =>
    new Date(dateString).toLocaleDateString('en-US', {
      year: 'numeric',
      month: 'long',
      day: 'numeric',
      hour: '2-digit',
      minute: '2-digit',
    });

  const removePrescription = async (id: string) => {
    const confirmDelete = window.confirm('Are you sure you want to remove this
prescription?');
    if (!confirmDelete) return;

    try {
      setDeletingId(id);
      setError('');
      const response = await fetch(`/api/prescriptions/${id}`, { method: 'DELETE' });
      const payload = await response.json().catch(() => ({}));

      if (!response.ok) {
        setError(payload?.error || 'Failed to remove prescription');
        return;
      }

      setPrescriptions((prev) => prev.filter((item) => item._id !== id));
    } catch {
      setError('Failed to remove prescription');
    } finally {
      setDeletingId(null);
    }
  };

  const confirmFulfillment = async (prescriptionId: string, pharmacyId: string) => {
    const key = `${prescriptionId}-${pharmacyId}`;
    try {
      setConfirmingKey(key);
      setError('');
      setSuccessMessage('');

      const response = await fetch(`/api/prescriptions/${prescriptionId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ status: 'confirm_fulfillment', pharmacyId }),
      });
      const payload = await response.json().catch(() => ({}));

      if (!response.ok) {
        setError(payload?.error || 'Failed to confirm fulfillment');
        return;
      }

      try {
        const prescription = prescriptions.find((item) => item._id === prescriptionId);
        const pharmacy = prescription?.pharmacyDetails?.find((item) => item.pharmacyId ===
pharmacyId);
        const notification = {
          id: `patient-confirm-${prescriptionId}-${pharmacyId}-${Date.now()}`,
          title: 'Fulfillment confirmed',
          message: `${pharmacy?.name || 'Pharmacy'} marked prescription as fulfilled.`,
          createdAt: new Date().toISOString(),
        };
        const raw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
        const existing = raw ? (JSON.parse(raw) as typeof notification[]) : [];
        const merged = [notification, ...existing].slice(0, 100);
        localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(merged));
        const unreadCount = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) || '0');
        localStorage.setItem(STORAGE_PATIENT_UNREAD_KEY, String(unreadCount + 1));
        window.dispatchEvent(new Event('notification-updated'));
      } catch {
        // Keep fulfillment action successful even if local notification storage fails.
      }

      setSuccessMessage('Fulfillment confirmed successfully.');
```

```
      await fetchHistory();
    } catch {
      setError('Failed to confirm fulfillment');
    } finally {
      setConfirmingKey(null);
    }
  };

  if (loading) {
    return (
      <div className="flex min-h-screen items-center justify-center">
        <div className="text-center">
          <div className="mx-auto h-12 w-12 animate-spin rounded-full border-b-2 border-cyan-
300" />
          <p className="mt-4 text-slate-300">Loading your prescription history...</p>
        </div>
      </div>
    );
  }

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-7xl rounded-3xl border border-white/10 bg-slate-900/80 p-6
sm:p-8">
        <h1 className="text-3xl font-bold text-white">My Prescription History</h1>
        <p className="mt-2 text-slate-300">Track every upload with status and timeline
details.</p>

        {error && <div className="mt-6 rounded-xl border border-rose-300/30 bg-rose-300/15 px-
4 py-3 text-rose-100">{error}</div>}
        {successMessage && (
          <div className="mt-6 rounded-xl border border-emerald-300/30 bg-emerald-300/15 px-4
py-3 text-emerald-100">
            {successMessage}
          </div>
        )}

        {prescriptions.length === 0 ? (
          <div className="mt-8 rounded-2xl border border-white/10 bg-slate-950/70 px-6 py-10
text-center">
            <h2 className="text-xl font-semibold text-white">No prescriptions yet</h2>
            <p className="mt-2 text-slate-300">You haven&apos;t uploaded any prescriptions
yet.</p>
            <button
              onClick={() => router.push('/upload')}
              className="mt-5 rounded-xl bg-cyan-300 px-5 py-3 font-semibold text-slate-900
transition hover:bg-cyan-200"
            >
              Upload Your First Prescription
            </button>
          </div>
        ) : (
          <div className="mt-8 space-y-6">
            {prescriptions.map((prescription) => (
              <article key={prescription._id} className="rounded-2xl border border-white/10
bg-slate-950/70 p-5 sm:p-6">
                <div className="flex flex-col gap-3 sm:flex-row sm:items-start sm:justify-
between">
                  <div>
                    <h3 className="text-lg font-semibold text-white">Prescription #
{prescription._id.slice(-8)}</h3>
                    <p className="text-sm text-slate-400">Uploaded on
{formatDate(prescription.createdAt)}</p>
                  </div>
                  <div className="flex items-center gap-2 self-start sm:self-auto">
                    <span className={`rounded-full border px-3 py-1 text-xs font-semibold
${getStatusColor(prescription.status)}`}>
                      {prescription.status.charAt(0).toUpperCase() +
prescription.status.slice(1)}
                    </span>
                    <button
                      type="button"
                      onClick={() => removePrescription(prescription._id)}
                      disabled={deletingId === prescription._id}
                      className="rounded-full border border-rose-300/40 bg-rose-300/15 px-3
py-1 text-xs font-semibold text-rose-100 transition hover:bg-rose-300/25 disabled:cursor-not-
allowed disabled:opacity-60"
                    >
                      {deletingId === prescription._id ? 'Removing...' : 'Remove'}
                    </button>
                  </div>
                </div>

                <div className="mt-5 grid grid-cols-1 gap-5 lg:grid-cols-12">
                  <div className="rounded-xl border border-white/10 bg-slate-900/50 p-4 text-
sm text-slate-200 lg:col-span-7">
                    <p className="text-sm font-semibold text-white">Prescription Details</p>
```

```
                <div className="mt-3 space-y-2">
                  <p><strong>Location:</strong> {prescription.location}</p>
                  <p><strong>Images:</strong> {prescription.prescriptionImages.length}
uploaded</p>
                  {prescription.notes && <p><strong>Notes:</strong> {prescription.notes}
</p>}
                </div>

                {prescription.prescriptionImages.length > 0 && (
                  <div className="mt-4 grid grid-cols-1 gap-3 sm:grid-cols-2">
                    {prescription.prescriptionImages.map((image, index) => (
                      <button
                        type="button"
                        key={`${prescription._id}-${index}`}
                        onClick={() => window.open(image, '_blank')}
                        className="group relative overflow-hidden rounded-xl border
border-white/10"
                      >
                        <img
                          src={image}
                          alt={`Prescription ${index + 1}`}
                          className="h-36 w-full object-cover transition group-
hover:scale-105"
                        />
                        <span className="absolute bottom-2 right-2 rounded-md bg-black/60
px-2 py-1 text-xs text-white">
                          {index + 1}
                        </span>
                      </button>
                    ))}
                  </div>
                )}
              </div>

              <div className="rounded-xl border border-white/10 bg-slate-900/50 p-4 text-
sm text-slate-200 lg:col-span-5">
                <p className="mb-2 text-sm font-semibold text-white">Pharmacy Details</p>
                {prescription.pharmacyDetails && prescription.pharmacyDetails.length > 0 ?
(
                  <div className="grid grid-cols-1 gap-3">
                    {prescription.pharmacyDetails.map((pharmacy) => (
                      <div key={`${prescription._id}-${pharmacy.pharmacyId}`}
className="rounded-xl border border-white/10 bg-slate-900/70 p-3">
                        <div className="flex items-center justify-between gap-2">
                          <p className="font-semibold text-white">{pharmacy.name}</p>
                          <span className={`rounded-full border px-2.5 py-0.5 text-xs
font-semibold ${getPharmacyStatusColor(pharmacy.status)}`}>
                            {pharmacy.status.charAt(0).toUpperCase() +
pharmacy.status.slice(1).replace('_', ' ')}
                          </span>
                        </div>
                        {pharmacy.status === 'fulfillment_requested' && (
                          <div className="mt-2 rounded-lg border border-indigo-300/30 bg-
indigo-300/10 p-2.5">
                            <p className="text-xs text-indigo-100">
                              This pharmacy requested your fulfillment confirmation.
                            </p>
                            {pharmacy.availabilityResponse && (
                              <p className="mt-1 text-xs text-indigo-100">
                                <strong>Availability:</strong>
{getAvailabilityLabel(pharmacy.availabilityResponse)}
                              </p>
                            )}
                            {pharmacy.pharmacistMessage && (
                              <p className="mt-1 text-xs text-indigo-100">
                                <strong>Message:</strong> {pharmacy.pharmacistMessage}
                              </p>
                            )}
                            <button
                              type="button"
                              onClick={() => confirmFulfillment(prescription._id,
pharmacy.pharmacyId)}
                              disabled={confirmingKey ===
`${prescription._id}-${pharmacy.pharmacyId}`}
                              className="mt-2 rounded-lg bg-emerald-300 px-3 py-1.5 text-
xs font-semibold text-slate-900 transition hover:bg-emerald-200 disabled:cursor-not-allowed
disabled:opacity-60"
                            >
                              {confirmingKey ===
`${prescription._id}-${pharmacy.pharmacyId}` ? 'Confirming...' : 'Confirm Fulfilled'}
                            </button>
                          </div>
                        )}
                        {pharmacy.availabilityResponse && (
                          <p className="mt-2 text-xs text-cyan-100">
                            <strong>Availability:</strong>
{getAvailabilityLabel(pharmacy.availabilityResponse)}
```

```
                              </p>
                            )}
                            {pharmacy.pharmacistMessage && (
                              <p className="mt-1 text-xs text-slate-300">
                                <strong>Message:</strong> {pharmacy.pharmacistMessage}
                              </p>
                            )}
                            {pharmacy.address && <p className="mt-1 text-slate-300">
{pharmacy.address}</p>}
                            {pharmacy.location && <p className="text-slate-400">Area:
{pharmacy.location}</p>}
                          </div>
                        ))}
                      </div>
                    ) : (
                      <p className="text-slate-400">No pharmacy details available</p>
                    )}
                  </div>
                </div>
              </article>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}
```

[23] app/layout.tsx FILE

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import Navbar from "@/components/Navbar";
import Footer from "@/components/Footer";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "E-Pharmacy - Mediator Between Patients and Pharmacies",
  description: "Connect patients with nearby pharmacies for seamless prescription
fulfillment",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body
        suppressHydrationWarning
        className={`${geistSans.variable} ${geistMono.variable} antialiased`}
      >
        <Navbar />
        {children}
        <Footer />
      </body>
    </html>
  );
}
```

[24] app/login/page.tsx FILE

```
'use client';

import Link from 'next/link';
import { useState } from 'react';

export default function LoginPage() {
```

```
  const [formData, setFormData] = useState({
    email: '',
    password: '',
    role: 'patient',
  });
  const [message, setMessage] = useState('');
  const [isLoading, setIsLoading] = useState(false);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsLoading(true);
    setMessage('');

    try {
      const response = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });

      const result = await response.json();

      if (response.ok) {
        setMessage('Login successful! Redirecting...');
        const redirectPath = formData.role === 'patient' ? '/upload' : '/dashboard';
        window.location.assign(redirectPath);
        return;
      } else {
        setMessage(result.error || 'Login failed');
      }
    } catch {
      setMessage('An error occurred. Please try again.');
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className="min-h-screen px-4 py-14 sm:px-6 lg:px-8">
      <div className="mx-auto grid w-full max-w-5xl gap-8 lg:grid-cols-2">
        <section className="glass-panel rounded-3xl border border-white/10 p-8">
          <p className="mb-4 inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10
px-3 py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
            Welcome Back
          </p>
          <h1 className="text-3xl font-bold text-white sm:text-4xl">Sign in to continue care
coordination</h1>
          <p className="mt-4 text-slate-300">
            Access your account to upload prescriptions, track status, or manage incoming
requests from nearby patients.
          </p>
          <div className="mt-8 space-y-3 text-sm text-slate-200">
            <p>Patient flow: Upload, track, and review history.</p>
            <p>Pharmacist flow: Receive alerts and assign requests quickly.</p>
            <p>Single platform designed for speed and local coverage.</p>
          </div>
        </section>

        <section className="rounded-3xl border border-white/10 bg-slate-900/85 p-8 shadow-
2xl">
          <h2 className="text-2xl font-bold text-white">Login to E-Pharmacy</h2>
          <form onSubmit={handleSubmit} className="mt-6 space-y-5">
            <div>
              <label className="mb-2 block text-sm font-medium text-slate-200">I am a</label>
              <select
                name="role"
                value={formData.role}
                onChange={handleChange}
                className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
              >
                <option value="patient">Patient</option>
                <option value="pharmacist">Pharmacist</option>
              </select>
            </div>
            <div>
              <label className="mb-2 block text-sm font-medium text-slate-200">Email</label>
              <input
                type="email"
                name="email"
                value={formData.email}
                onChange={handleChange}
                required
```

```
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your email"
            />
          </div>
          <div>
            <label className="mb-2 block text-sm font-medium text-slate-
200">Password</label>
            <input
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
              required
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your password"
            />
          </div>
          <button
            type="submit"
            disabled={isLoading}
            className="w-full rounded-xl bg-cyan-300 px-4 py-3 font-semibold text-slate-900
transition hover:bg-cyan-200 disabled:cursor-not-allowed disabled:opacity-60"
          >
            {isLoading ? 'Logging in...' : 'Login'}
          </button>
        </form>
        {message && (
          <p
            className={`mt-4 rounded-xl px-4 py-3 text-center text-sm ${
              message.includes('successful')
                ? 'bg-emerald-300/15 text-emerald-100'
                : 'bg-rose-300/15 text-rose-100'
            }`}
          >
            {message}
          </p>
        )}
        <p className="mt-6 text-sm text-slate-300">
          Don&apos;t have an account?{' '}
          <Link href="/register" className="font-semibold text-cyan-200 hover:text-cyan-
100">
            Register here
          </Link>
        </p>
      </section>
    </div>
  </div>
  );
}
```

[25] app/notifications/page.tsx FILE

```
'use client';

import NotificationComponent from '@/components/NotificationComponent';
import { useRouter } from 'next/navigation';
import { useEffect, useState } from 'react';

export default function NotificationsPage() {
  const [location, setLocation] = useState('');
  const [isCheckingAuth, setIsCheckingAuth] = useState(true);
  const router = useRouter();

  useEffect(() => {
    const checkAuth = async () => {
      try {
        const response = await fetch('/api/auth/me');
        if (!response.ok) {
          router.push('/login');
          return;
        }

        const data = await response.json();
        if (data?.user?.role !== 'pharmacist') {
          router.push('/');
          return;
        }

        setLocation(data?.user?.location || '');
      } catch (error) {
        console.error('Failed to verify pharmacist session:', error);
```

```
        router.push('/login');
      } finally {
        setIsCheckingAuth(false);
      }
    };

    checkAuth();
  }, [router]);

  if (isCheckingAuth) {
    return (
      <div className="flex min-h-screen items-center justify-center">
        <div className="text-slate-300">Loading notifications...</div>
      </div>
    );
  }

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-4xl">
        <div className="mb-6 rounded-3xl border border-white/10 bg-slate-900/80 p-8">
          <p className="inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10 px-3
py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
            Live Feed
          </p>
          <h1 className="mt-4 text-3xl font-bold text-white sm:text-4xl">Notifications</h1>
          <p className="mt-2 text-slate-300">
            Real-time prescription alerts from patients that match your pharmacy profile.
          </p>
        </div>

        <div className="rounded-3xl border border-white/10 bg-slate-900/80 p-6">
          <div className="mb-5">
            <label className="mb-2 block text-sm font-medium text-slate-200">Filter by
location (City/ZIP)</label>
            <input
              type="text"
              value={location}
              onChange={(e) => setLocation(e.target.value)}
              placeholder="Enter your service location"
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
            />
          </div>

          <NotificationComponent
            currentLocation={location}
            maxItems={20}
            title="Incoming Prescription Alerts"
          />
        </div>
      </div>
    </div>
  );
}
```

[26] app/page.tsx FILE

```
import Link from "next/link";

const impactStats = [
  { label: "Average match time", value: "7 min" },
  { label: "Partner pharmacies", value: "240+" },
  { label: "Cities covered", value: "38" },
  { label: "Patient satisfaction", value: "97%" },
];

const workflowSteps = [
  {
    title: "Upload your prescription",
    description:
      "Share a clear photo and basic details. We verify format and route it securely.",
  },
  {
    title: "Nearby pharmacies get notified",
    description:
      "Subscribed pharmacies in your area can review and respond in real time.",
  },
  {
    title: "Confirm and collect",
    description:
      "Compare response speed and availability, then choose where to fulfill.",
  },
```

```
];

const highlightCards = [
  {
    title: "Faster fulfillment",
    description:
      "Reduce wait times with location-aware matching and instant notification flow.",
  },
  {
    title: "Built for trust",
    description:
      "Role-based access, session protection, and audit-ready prescription history.",
  },
  {
    title: "Pharmacy growth",
    description:
      "Help local pharmacies discover nearby demand and manage requests efficiently.",
  },
];

const faqItems = [
  {
    question: "Who can use E-Pharmacy?",
    answer:
      "Patients can upload prescriptions and browse nearby options. Pharmacies can subscribe
to receive matching requests.",
  },
  {
    question: "Does it replace my doctor or pharmacy?",
    answer:
      "No. E-Pharmacy is a coordination platform between patients and licensed pharmacies.",
  },
  {
    question: "How quickly do pharmacies respond?",
    answer:
      "Response time depends on your area, but most active requests receive initial visibility
within minutes.",
  },
  {
    question: "Can I track past requests?",
    answer:
      "Yes. Patients can view history and status updates, while pharmacists can manage live
demand from the dashboard.",
  },
];

const plans = [
  {
    id: "monthly",
    name: "Monthly Plan",
    price: "₹299/month",
    features: [
      "Access to prescriptions",
      "Basic dashboard",
      "Email support",
    ],
    ctaLabel: "Select Plan",
    href: "/subscribe",
    popular: false,
  },
  {
    id: "yearly",
    name: "Yearly Plan",
    price: "₹999/year",
    features: [
      "All monthly features",
      "Priority matching",
      "Phone support",
      "Save 20%",
    ],
    ctaLabel: "Select Plan",
    href: "/subscribe",
    popular: true,
  },
  {
    id: "premium",
    name: "Premium Plan",
    price: "₹2999/year",
    features: [
      "All yearly features",
      "Advanced analytics",
      "Dedicated account manager",
      "Custom integrations",
    ],
    ctaLabel: "Select Plan",
    href: "/subscribe",
    popular: false,
```

```
  },
];

export default function HomePage() {
  return (
    <div className="relative overflow-hidden bg-slate-950 text-slate-100">
      <div className="pointer-events-none absolute inset-0 bg-[radial-
gradient(circle_at_12%_18%,rgba(45,212,191,0.28),transparent_34%),radial-
gradient(circle_at_80%_22%,rgba(251,146,60,0.2),transparent_30%),radial-
gradient(circle_at_52%_84%,rgba(56,189,248,0.18),transparent_30%)]" />

      <main className="relative mx-auto max-w-7xl px-4 pb-24 pt-14 sm:px-6 lg:px-8">
        <section className="grid items-center gap-10 pb-16 pt-8 lg:grid-cols-2">
          <div>
            <p className="mb-4 inline-flex rounded-full border border-cyan-300/30 bg-cyan-
300/10 px-4 py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
              Modern Prescription Coordination
            </p>
            <h1 className="text-balance text-4xl font-bold leading-tight sm:text-5xl lg:text-
6xl">
              Healthcare access should feel as responsive as messaging.
            </h1>
            <p className="mt-6 max-w-xl text-lg text-slate-300">
              E-Pharmacy connects patients and pharmacies with a workflow that is
              fast, local, and easier to manage from first upload to fulfillment.
            </p>
            <div className="mt-8 flex flex-wrap gap-3">
              <Link
                href="/upload"
                className="rounded-full bg-cyan-300 px-6 py-3 text-sm font-semibold text-
slate-900 transition hover:bg-cyan-200"
              >
                Upload Prescription
              </Link>
              <Link
                href="/pharmacies"
                className="rounded-full border border-slate-600 px-6 py-3 text-sm font-
semibold text-slate-100 transition hover:border-slate-400 hover:bg-slate-900"
              >
                Browse Pharmacies
              </Link>
              <Link
                href="/dashboard"
                className="rounded-full border border-emerald-300/40 bg-emerald-300/10 px-6
py-3 text-sm font-semibold text-emerald-100 transition hover:bg-emerald-300/20"
              >
                Open Dashboard
              </Link>
            </div>
          </div>

          <div className="glass-panel rounded-3xl border border-white/10 p-6 sm:p-8">
            <p className="mb-6 text-sm font-medium uppercase tracking-[0.15em] text-slate-
300">
              Why teams choose us
            </p>
            <div className="space-y-5">
              <article className="rounded-2xl border border-cyan-300/20 bg-cyan-300/10 p-5">
                <h2 className="text-lg font-semibold text-cyan-100">
                  Local-first matching
                </h2>
                <p className="mt-2 text-sm text-slate-200">
                  Pharmacy notifications prioritize nearby demand, improving response
                  time and convenience for patients.
                </p>
              </article>
              <article className="rounded-2xl border border-orange-300/20 bg-orange-300/10 p-
5">
                <h2 className="text-lg font-semibold text-orange-100">
                  Better operations for pharmacists
                </h2>
                <p className="mt-2 text-sm text-slate-200">
                  Subscription-based access and dashboard tools keep incoming work
                  organized without switching platforms.
                </p>
              </article>
              <article className="rounded-2xl border border-emerald-300/20 bg-emerald-300/10
p-5">
                <h2 className="text-lg font-semibold text-emerald-100">
                  Clear patient journey
                </h2>
                <p className="mt-2 text-sm text-slate-200">
                  Upload, match, confirm, and track in one place with no complicated
                  onboarding.
                </p>
              </article>
            </div>
```

```
          </div>
        </section>

        <section className="mb-16 grid grid-cols-2 gap-4 sm:grid-cols-4">
          {impactStats.map((stat) => (
            <div
              key={stat.label}
              className="glass-panel rounded-2xl border border-white/10 p-5 text-center"
            >
              <p className="text-2xl font-bold text-white">{stat.value}</p>
              <p className="mt-1 text-xs uppercase tracking-[0.12em] text-slate-300">
                {stat.label}
              </p>
            </div>
          ))}
        </section>

        <section className="mb-16">
          <div className="mb-8 flex flex-wrap items-end justify-between gap-4">
            <h2 className="text-3xl font-bold text-white sm:text-4xl">
              How it works
            </h2>
            <Link
              href="/register"
              className="rounded-full border border-white/20 px-5 py-2 text-sm font-semibold
text-white transition hover:bg-white/10"
            >
              Create an account
            </Link>
          </div>
          <div className="grid gap-5 lg:grid-cols-3">
            {workflowSteps.map((step, index) => (
              <article
                key={step.title}
                className="glass-panel rounded-2xl border border-white/10 p-6"
              >
                <p className="mb-4 inline-flex h-8 w-8 items-center justify-center rounded-
full bg-white/15 text-sm font-bold text-cyan-100">
                  {index + 1}
                </p>
                <h3 className="text-xl font-semibold text-white">{step.title}</h3>
                <p className="mt-3 text-slate-300">{step.description}</p>
              </article>
            ))}
          </div>
        </section>

        <section className="mb-16 grid gap-5 lg:grid-cols-3">
          {highlightCards.map((card) => (
            <article
              key={card.title}
              className="rounded-2xl border border-slate-700 bg-slate-900/80 p-6"
            >
              <h3 className="text-xl font-semibold text-white">{card.title}</h3>
              <p className="mt-3 text-slate-300">{card.description}</p>
            </article>
          ))}
        </section>

        <section className="mb-16 rounded-3xl border border-white/10 bg-white/[0.04] p-6 sm:p-
8">
          <div className="grid gap-8 lg:grid-cols-2">
            <div>
              <h2 className="text-3xl font-bold text-white">Safety and privacy</h2>
              <p className="mt-4 text-slate-300">
                Sensitive data stays in controlled flows with role-specific access.
                We designed the experience so patients share only what is required
                while pharmacies see only the cases relevant to them.
              </p>
              <ul className="mt-5 space-y-3 text-sm text-slate-200">
                <li>Secure authentication sessions</li>
                <li>Prescription history for patient visibility</li>
                <li>Structured pharmacy subscription model</li>
              </ul>
            </div>
            <div className="rounded-2xl border border-emerald-300/20 bg-emerald-300/10 p-6">
              <p className="text-xs font-semibold uppercase tracking-[0.14em] text-emerald-
100">

                For pharmacy owners
              </p>
              <h3 className="mt-2 text-2xl font-bold text-white">
                Turn demand into measurable growth
              </h3>
              <p className="mt-3 text-slate-100">
                Subscription gives your team access to local prescription demand,
                helping you allocate staff and inventory with better confidence.
              </p>
```

```
              <Link
                href="/subscribe"
                className="mt-6 inline-block rounded-full bg-emerald-300 px-5 py-2 text-sm
font-semibold text-slate-900 transition hover:bg-emerald-200"
              >
                View subscription options
              </Link>
            </div>
          </div>
        </section>

        <section className="mb-16">
          <h2 className="mb-8 text-3xl font-bold text-white">What people say</h2>
          <div className="grid gap-5 lg:grid-cols-3">
            <blockquote className="glass-panel rounded-2xl border border-white/10 p-6">
              <p className="text-slate-100">
                &quot;I uploaded at night and had nearby responses fast the next morning.
                Way easier than calling multiple places.&quot;
              </p>
              <footer className="mt-4 text-sm text-slate-300">A patient in Dallas</footer>
            </blockquote>
            <blockquote className="glass-panel rounded-2xl border border-white/10 p-6">
              <p className="text-slate-100">
                &quot;The dashboard gives my staff a clearer queue. We now respond to
                demand we used to miss.&quot;
              </p>
              <footer className="mt-4 text-sm text-slate-300">
                Independent pharmacy manager
              </footer>
            </blockquote>
            <blockquote className="glass-panel rounded-2xl border border-white/10 p-6">
              <p className="text-slate-100">
                &quot;The process feels transparent. I can track status without guessing
                what happened after upload.&quot;
              </p>
              <footer className="mt-4 text-sm text-slate-300">Repeat patient user</footer>
            </blockquote>
          </div>
        </section>

        <section className="mb-16">
          <h2 className="mb-8 text-3xl font-bold text-white">Frequently asked questions</h2>
          <div className="grid gap-4 lg:grid-cols-2">
            {faqItems.map((item) => (
              <article
                key={item.question}
                className="rounded-2xl border border-slate-700 bg-slate-900/70 p-5"
              >
                <h3 className="text-lg font-semibold text-white">{item.question}</h3>
                <p className="mt-2 text-slate-300">{item.answer}</p>
              </article>
            ))}
          </div>
        </section>

        <section className="mb-16">
          <div className="mb-8 flex flex-wrap items-end justify-between gap-4">
            <div>
              <p className="text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
                Plans
              </p>
              <h2 className="mt-2 text-3xl font-bold text-white sm:text-4xl">
                Pharmacy subscription tiers
              </h2>
            </div>
            <Link
              href="/subscribe"
              className="rounded-full border border-white/20 px-5 py-2 text-sm font-semibold
text-white transition hover:bg-white/10"
            >
              Compare all plans
            </Link>
          </div>
          <div className="grid gap-5 lg:grid-cols-3">
            {plans.map((plan) => (
              <article
                key={plan.name}
                className={`rounded-2xl border p-6 ${
                  plan.popular
                    ? "border-cyan-200/40 bg-cyan-300/10"
                    : "border-white/10 bg-white/[0.04]"
                }`}
              >
                {plan.popular && (
                  <p className="mb-4 inline-flex rounded-full bg-emerald-300/20 px-3 py-1
text-xs font-semibold text-emerald-100">
                    Most Popular
```

```
                </p>
              )}
              <p className="text-sm font-semibold uppercase tracking-[0.12em] text-slate-
200">
                {plan.name}
              </p>
              <p className="mt-3 text-4xl font-bold text-cyan-200">{plan.price}</p>
              <ul className="mt-5 space-y-2 text-sm text-slate-200">
                {plan.features.map((feature) => (
                  <li key={feature}>• {feature}</li>
                ))}
              </ul>
              <Link
                href={plan.href}
                className={`mt-6 inline-block rounded-full px-5 py-2 text-sm font-semibold
transition ${
                  plan.popular
                    ? "bg-cyan-300 text-slate-900 hover:bg-cyan-200"
                    : "border border-white/20 text-white hover:bg-white/10"
                }`}
              >
                {plan.ctaLabel}
              </Link>
            </article>
          ))}
        </div>
      </section>

      <section className="rounded-3xl border border-cyan-200/30 bg-gradient-to-r from-cyan-
300/20 to-sky-300/10 p-7 text-center sm:p-10">
        <h2 className="text-3xl font-bold text-white">
          Start your first prescription request today
        </h2>
        <p className="mx-auto mt-3 max-w-2xl text-slate-100">
          Whether you are a patient looking for faster service or a pharmacy
          looking for qualified local demand, E-Pharmacy is ready.
        </p>
        <div className="mt-7 flex flex-wrap justify-center gap-3">
          <Link
            href="/upload"
            className="rounded-full bg-white px-6 py-3 text-sm font-semibold text-slate-900
transition hover:bg-slate-200"
          >
            Upload now
          </Link>
          <Link
            href="/subscribe"
            className="rounded-full border border-white/30 px-6 py-3 text-sm font-semibold
text-white transition hover:bg-white/10"
          >
            Become a pharmacy partner
          </Link>
        </div>
      </section>
    </main>
  </div>
 );
}
```

[27] app/pharmacies/page.tsx FILE

```
'use client';

import PharmacyMap from '@/components/PharmacyMap';
import { useCallback, useEffect, useMemo, useState } from 'react';

interface Pharmacy {
  _id: string;
  name: string;
  email: string;
  phone: string;
  address: string;
  location: string;
  coordinates: {
    lat: number;
    lng: number;
  };
  supportsPrescriptionUpload: boolean;
  isUsingService: boolean;
  subscriptionType: string | null;
  rating: number;
  reviewCount: number;
  operatingHours?: Record<string, unknown>;
```

```
  services?: string[];
  source: 'platform' | 'google';
  website?: string;
}

interface GoogleNearbyPharmacy {
  placeId: string;
  name: string;
  address: string;
  coordinates: {
    lat: number;
    lng: number;
  };
  rating: number;
  reviewCount: number;
  phone?: string;
  website?: string;
}

export default function PharmaciesPage() {
  const [platformPharmacies, setPlatformPharmacies] = useState<Pharmacy[]>([]);
  const [googlePharmacies, setGooglePharmacies] = useState<Pharmacy[]>([]);
  const [indiaSearchResults, setIndiaSearchResults] = useState<Pharmacy[]>([]);
  const [detectedCity, setDetectedCity] = useState('');
  const [searchQuery, setSearchQuery] = useState('');
  const [userLocation, setUserLocation] = useState<{ lat: number; lng: number } | null>(null);
  const [selectedPharmacy, setSelectedPharmacy] = useState<Pharmacy | null>(null);
  const [loading, setLoading] = useState(false);
  const [searchLoading, setSearchLoading] = useState(false);
  const [nearbyError, setNearbyError] = useState('');
  const [visibleCount, setVisibleCount] = useState(20);

  const fetchPlatformPharmacies = useCallback(async (lat?: number, lng?: number, city?:
string) => {
    try {
      let url = '/api/pharmacies';
      if (city) {
        url += `?location=${encodeURIComponent(city)}`;
      } else if (lat && lng) {
        url += `?lat=${lat}&lng=${lng}&radius=10000`;
      }
      const response = await fetch(url);
      const data = await response.json();
      const normalized: Pharmacy[] = (Array.isArray(data) ? data : []).map((pharmacy) => ({
        ...pharmacy,
        source: 'platform',
      }));
      setPlatformPharmacies(normalized);
      return normalized;
    } catch (error) {
      console.error('Failed to fetch platform pharmacies', error);
      return [] as Pharmacy[];
    }
  }, []);

  const resolveCityFromCoordinates = useCallback(async (lat: number, lng: number) => {
    try {
      const response = await fetch(`/api/google/reverse-geocode?lat=${lat}&lng=${lng}`);
      if (!response.ok) {
        return '';
      }
      const payload = await response.json();
      return payload?.city || '';
    } catch (error) {
      console.error('Failed to resolve city', error);
      return '';
    }
  }, []);

  const fetchNearbyPharmacies = useCallback(async (lat: number, lng: number) => {
    setLoading(true);
    setNearbyError('');

    const city = await resolveCityFromCoordinates(lat, lng);
    setDetectedCity(city);

    const latestPlatformPharmacies = city
      ? await fetchPlatformPharmacies(undefined, undefined, city)
      : await fetchPlatformPharmacies(lat, lng);

    try {
      const googleApiUrl = city
        ? `/api/google/nearby-pharmacies?city=${encodeURIComponent(city)}&limit=60`
        : `/api/google/nearby-pharmacies?lat=${lat}&lng=${lng}&radius=50000&limit=60`;
      const response = await fetch(googleApiUrl);
      if (!response.ok) {
        const payload = await response.json().catch(() => ({}));
```

```
        const details = payload?.details || payload?.error || 'Unable to fetch Google nearby
pharmacies';
        setNearbyError(details);
        setGooglePharmacies([]);
        return;
      }

      const data = (await response.json()) as GoogleNearbyPharmacy[];
      const normalizedGoogle: Pharmacy[] = data.map((item) => ({
        _id: `google-${item.placeId}`,
        name: item.name,
        email: '',
        phone: item.phone || '',
        address: item.address,
        location: item.address,
        coordinates: item.coordinates,
        supportsPrescriptionUpload: false,
        isUsingService: false,
        subscriptionType: null,
        rating: item.rating || 0,
        reviewCount: item.reviewCount || 0,
        services: [],
        source: 'google',
        website: item.website || '',
      }));

      const platformKeySet = new Set(
        latestPlatformPharmacies.map(
          (pharmacy) => `${pharmacy.name.toLowerCase()}|${pharmacy.address.toLowerCase()}`
        )
      );

      const dedupedGoogle = normalizedGoogle.filter((pharmacy) => {
        const key = `${pharmacy.name.toLowerCase()}|${pharmacy.address.toLowerCase()}`;
        return !platformKeySet.has(key);
      });

      setGooglePharmacies(dedupedGoogle);
    } catch (error) {
      console.error('Failed to fetch nearby pharmacies from Google', error);
      setNearbyError('Failed to load city-wide Google pharmacies.');
      setGooglePharmacies([]);
    } finally {
      setLoading(false);
    }
  }, [fetchPlatformPharmacies, resolveCityFromCoordinates]);

  useEffect(() => {
    fetchPlatformPharmacies();

    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(
        (position) => {
          const { latitude, longitude } = position.coords;
          setUserLocation({ lat: latitude, lng: longitude });
          fetchNearbyPharmacies(latitude, longitude);
        },
        () => {
          setNearbyError('Location access denied. Showing platform pharmacies only.');
        }
      );
    }
  }, [fetchNearbyPharmacies, fetchPlatformPharmacies]);

  const getCurrentLocation = () => {
    if (!navigator.geolocation) {
      setNearbyError('Geolocation is not supported by this browser.');
      return;
    }

    navigator.geolocation.getCurrentPosition(
      (position) => {
        const { latitude, longitude } = position.coords;
        setUserLocation({ lat: latitude, lng: longitude });
        fetchNearbyPharmacies(latitude, longitude);
      },
      () => {
        setNearbyError('Unable to get your location. Please allow location access.');
      }
    );
  };

  const allPharmacies = useMemo(() => {
    const merged = [...platformPharmacies, ...googlePharmacies];
    return merged.sort((a, b) => {
      const aPriority = a.source === 'platform' ? 2 : 1;
      const bPriority = b.source === 'platform' ? 2 : 1;
```

```
      if (aPriority !== bPriority) return bPriority - aPriority;
      if (a.rating !== b.rating) return b.rating - a.rating;
      return b.reviewCount - a.reviewCount;
    });
  }, [platformPharmacies, googlePharmacies]);

  useEffect(() => {
    const normalizedQuery = searchQuery.trim();
    if (normalizedQuery.length < 2) {
      setIndiaSearchResults([]);
      setSearchLoading(false);
      return;
    }

    let isActive = true;
    const timer = setTimeout(async () => {
      setSearchLoading(true);
      try {
        const [platformResponse, googleResponse] = await Promise.all([
          fetch(`/api/pharmacies?q=${encodeURIComponent(normalizedQuery)}&limit=100`),
          fetch(`/api/google/nearby-pharmacies?
q=${encodeURIComponent(normalizedQuery)}&limit=60`),
        ]);

        const platformData = platformResponse.ok ? await platformResponse.json() : [];
        const googleData = googleResponse.ok ? await googleResponse.json() : [];

        const normalizedPlatform: Pharmacy[] = (Array.isArray(platformData) ? platformData :
[]).map(
          (pharmacy) => ({
            ...pharmacy,
            source: 'platform',
          })
        );

        const normalizedGoogle: Pharmacy[] = (Array.isArray(googleData) ? googleData :
[]).map(
          (item: GoogleNearbyPharmacy) => ({
            _id: `google-${item.placeId}`,
            name: item.name,
            email: '',
            phone: item.phone || '',
            address: item.address,
            location: item.address,
            coordinates: item.coordinates,
            supportsPrescriptionUpload: false,
            isUsingService: false,
            subscriptionType: null,
            rating: item.rating || 0,
            reviewCount: item.reviewCount || 0,
            services: [],
            source: 'google',
            website: item.website || '',
          })
        );

        const platformKeySet = new Set(
          normalizedPlatform.map(
            (pharmacy) => `${pharmacy.name.toLowerCase()}|${pharmacy.address.toLowerCase()}`
          )
        );
        const dedupedGoogle = normalizedGoogle.filter((pharmacy) => {
          const key = `${pharmacy.name.toLowerCase()}|${pharmacy.address.toLowerCase()}`;
          return !platformKeySet.has(key);
        });

        const merged = [...normalizedPlatform, ...dedupedGoogle].sort((a, b) => {
          const aPriority = a.source === 'platform' ? 2 : 1;
          const bPriority = b.source === 'platform' ? 2 : 1;
          if (aPriority !== bPriority) return bPriority - aPriority;
          if (a.rating !== b.rating) return b.rating - a.rating;
          return b.reviewCount - a.reviewCount;
        });

        if (!isActive) return;
        setIndiaSearchResults(merged);
      } catch (error) {
        console.error('Failed to search pharmacies across India:', error);
        if (!isActive) return;
        setIndiaSearchResults([]);
      } finally {
        if (isActive) setSearchLoading(false);
      }
    }, 350);

    return () => {
      isActive = false;
```

```
      clearTimeout(timer);
    };
  }, [searchQuery]);

  const filteredPharmacies = useMemo(() => {
    const normalizedQuery = searchQuery.toLowerCase().trim();
    const isIndiaSearchMode = normalizedQuery.length >= 2;
    const base = isIndiaSearchMode ? indiaSearchResults : allPharmacies;
    if (!normalizedQuery) return base;
    if (isIndiaSearchMode) return base;

    return base.filter(
      (pharmacy) =>
        pharmacy.name.toLowerCase().includes(normalizedQuery) ||
        pharmacy.address.toLowerCase().includes(normalizedQuery) ||
        pharmacy.location.toLowerCase().includes(normalizedQuery)
    );
  }, [allPharmacies, indiaSearchResults, searchQuery]);

  useEffect(() => {
    setVisibleCount(20);
  }, [searchQuery, allPharmacies.length, indiaSearchResults.length]);

  const visiblePharmacies = useMemo(
    () => filteredPharmacies.slice(0, visibleCount),
    [filteredPharmacies, visibleCount]
  );

  const hasMore = visibleCount < filteredPharmacies.length;
  const isListLoading = loading || searchLoading;

  const handleListScroll = (event: React.UIEvent<HTMLDivElement>) => {
    if (!hasMore || isListLoading) return;

    const target = event.currentTarget;
    const reachedBottom =
      target.scrollTop + target.clientHeight >= target.scrollHeight - 80;

    if (reachedBottom) {
      setVisibleCount((count) => Math.min(count + 20, filteredPharmacies.length));
    }
  };

  const handlePharmacySelect = (pharmacy: Pharmacy) => {
    setSelectedPharmacy(pharmacy);
  };

  useEffect(() => {
    if (!selectedPharmacy) return;
    const stillVisible = visiblePharmacies.some((pharmacy) => pharmacy._id ===
selectedPharmacy._id);
    if (!stillVisible) {
      setSelectedPharmacy(null);
    }
  }, [selectedPharmacy, visiblePharmacies]);

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-7xl">
        <div className="mb-8 rounded-3xl border border-white/10 bg-slate-900/80 p-8">
          <p className="inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10 px-3
py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
            Discovery
          </p>
          <h1 className="mt-4 text-3xl font-bold text-white sm:text-4xl">Find Nearby
Pharmacies</h1>
          <p className="mt-2 max-w-2xl text-slate-300">
            Showing your complete city pharmacies: on-platform entries first, plus Google
pharmacies in your city.
          </p>
          <p className="mt-2 text-sm text-slate-400">
            Tip: type 2+ characters to search pharmacies across India by name.
          </p>
          {detectedCity && (
            <p className="mt-2 text-sm text-cyan-200">City mode: {detectedCity}</p>
          )}
        </div>

        <div className="grid grid-cols-1 gap-6 lg:grid-cols-3">
          <section className="rounded-3xl border border-white/10 bg-slate-900/80 p-6 lg:col-
span-1">
            <label className="mb-2 block text-sm font-medium text-slate-200">Search
Location</label>
            <input
              type="text"
              placeholder="Search pharmacy name, city, or address"
              value={searchQuery}
```

```
              onChange={(e) => setSearchQuery(e.target.value)}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
            />
            <button
              onClick={getCurrentLocation}
              className="mt-3 w-full rounded-xl border border-emerald-300/40 bg-emerald-300/15
px-4 py-3 text-sm font-semibold text-emerald-100 transition hover:bg-emerald-300/25"
            >
              Use My Current Location
            </button>
            {nearbyError && <p className="mt-3 text-xs text-amber-300">{nearbyError}</p>}

            <div
              className="mt-6 max-h-[32rem] space-y-3 overflow-y-auto pr-1"
              onScroll={handleListScroll}
            >
              <h2 className="text-sm font-semibold uppercase tracking-[0.1em] text-slate-300">
                Pharmacies ({filteredPharmacies.length})
              </h2>
              {isListLoading ? (
                <div className="py-8 text-center">
                  <div className="mx-auto h-8 w-8 animate-spin rounded-full border-b-2 border-
cyan-300" />
                  <p className="mt-2 text-sm text-slate-300">
                    {searchQuery.trim().length >= 2
                      ? 'Searching pharmacies across India...'
                      : 'Loading nearby pharmacies...'}
                  </p>
                  {detectedCity && <p className="mt-1 text-xs text-cyan-200">Searching whole
city: {detectedCity}</p>}
                </div>
              ) : (
                visiblePharmacies.map((pharmacy) => (
                  <article
                    key={pharmacy._id}
                    onClick={() => handlePharmacySelect(pharmacy)}
                    className={`cursor-pointer rounded-2xl border p-4 transition ${
                      selectedPharmacy?._id === pharmacy._id
                        ? 'border-cyan-300 bg-cyan-300/10'
                        : 'border-white/10 bg-slate-950/70 hover:border-white/25'
                    }`}
                  >
                    <h3 className="font-semibold text-white">{pharmacy.name}</h3>
                    <p className="mt-1 text-sm text-slate-300">{pharmacy.address}</p>
                    <div className="mt-3 flex flex-wrap gap-2">
                      {pharmacy.source === 'platform' && (
                        <span className="rounded-full bg-emerald-300/20 px-2 py-1 text-xs
text-emerald-100">
                          On Platform
                        </span>
                      )}
                      {pharmacy.subscriptionType && (
                        <span className="rounded-full bg-violet-300/20 px-2 py-1 text-xs text-
violet-100">
                          {pharmacy.subscriptionType} subscriber
                        </span>
                      )}
                      {pharmacy.isUsingService && (
                        <span className="rounded-full bg-cyan-300/20 px-2 py-1 text-xs text-
cyan-100">
                          Service Partner
                        </span>
                      )}
                    </div>
                    <p className="mt-2 text-xs text-slate-400">
                      Rating {pharmacy.rating} ({pharmacy.reviewCount} reviews)
                    </p>
                  </article>
                ))
              )}
              {!isListLoading && hasMore && (
                <p className="py-3 text-center text-xs text-slate-400">
                  Scroll down to load more pharmacies
                </p>
              )}
            </div>
          </section>

          <section className="rounded-3xl border border-white/10 bg-slate-900/80 p-3 lg:col-
span-2">
            <PharmacyMap
              pharmacies={visiblePharmacies}
              userLocation={userLocation || undefined}
              onPharmacySelect={handlePharmacySelect}
              selectedPharmacyId={selectedPharmacy?._id || null}
              heightClassName="h-[32rem]"
```

```
          />
        </section>
      </div>

      {selectedPharmacy && (
        <section className="mt-6 rounded-3xl border border-white/10 bg-slate-900/80 p-6">
          <h2 className="text-2xl font-bold text-white">{selectedPharmacy.name}</h2>
          <div className="mt-4 grid grid-cols-1 gap-6 md:grid-cols-2">
            <div>
              <h3 className="text-sm font-semibold uppercase tracking-[0.1em] text-slate-
300">Contact</h3>
              <div className="mt-2 space-y-1 text-slate-200">
                {selectedPharmacy.address && <p>{selectedPharmacy.address}</p>}
                {selectedPharmacy.phone && <p>Phone: {selectedPharmacy.phone}</p>}
                {selectedPharmacy.email && <p>Email: {selectedPharmacy.email}</p>}
                {selectedPharmacy.website && (
                  <p>
                    Website:{' '}
                    <a
                      href={selectedPharmacy.website}
                      target="_blank"
                      rel="noopener noreferrer"
                      className="text-cyan-200 underline hover:text-cyan-100"
                    >
                      {selectedPharmacy.website}
                    </a>
                  </p>
                )}
                {selectedPharmacy.location &&
                  selectedPharmacy.location.trim().toLowerCase() !==
                    (selectedPharmacy.address || '').trim().toLowerCase() && (
                    <p>Area: {selectedPharmacy.location}</p>
                  )}
              </div>
            </div>
            <div>
              <h3 className="text-sm font-semibold uppercase tracking-[0.1em] text-slate-
300">Services</h3>
              <div className="mt-2 flex flex-wrap gap-2">
                {selectedPharmacy.source === 'platform' && (
                  <span className="rounded-full bg-emerald-300/20 px-3 py-1 text-sm text-
emerald-100">
                    Platform Pharmacy
                  </span>
                )}
                {selectedPharmacy.isUsingService && (
                  <span className="rounded-full bg-cyan-300/20 px-3 py-1 text-sm text-cyan-
100">
                    Service Partner
                  </span>
                )}
                {selectedPharmacy.supportsPrescriptionUpload && (
                  <span className="rounded-full bg-sky-300/20 px-3 py-1 text-sm text-sky-
100">
                    Prescription Upload Supported
                  </span>
                )}
                {selectedPharmacy.subscriptionType && (
                  <span className="rounded-full bg-violet-300/20 px-3 py-1 text-sm text-
violet-100">
                    {selectedPharmacy.subscriptionType.charAt(0).toUpperCase() +
selectedPharmacy.subscriptionType.slice(1)} Plan
                  </span>
                )}
              </div>
              {selectedPharmacy.services && selectedPharmacy.services.length > 0 && (
                <div className="mt-3 flex flex-wrap gap-2">
                  {selectedPharmacy.services.map((service) => (
                    <span key={service} className="rounded-full border border-white/15 px-3
py-1 text-sm text-slate-200">
                      {service}
                    </span>
                  ))}
                </div>
              )}
            </div>
          </div>
        </section>
      )}
    </div>
  </div>
  );
}
```

[28] app/profile/page.tsx FILE

```
'use client';

import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { useEffect, useState } from 'react';

interface ProfileData {
  id: string;
  name: string;
  email: string;
  role: string;
  phone: string;
  address: string;
  location: string;
  subscriptionType: string;
  subscriptionStart: string;
  subscriptionEnd: string;
}

type EditableField = 'name' | 'phone' | 'address' | 'location';

export default function ProfilePage() {
  const router = useRouter();
  const [isLoading, setIsLoading] = useState(true);
  const [savingField, setSavingField] = useState<EditableField | null>(null);
  const [editingField, setEditingField] = useState<EditableField | null>(null);
  const [draftValue, setDraftValue] = useState('');
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
  const [profile, setProfile] = useState<ProfileData>({
    id: '',
    name: '',
    email: '',
    role: '',
    phone: '',
    address: '',
    location: '',
    subscriptionType: '',
    subscriptionStart: '',
    subscriptionEnd: '',
  });

  useEffect(() => {
    const loadProfile = async () => {
      try {
        const response = await fetch('/api/profile');
        if (!response.ok) {
          router.push('/login');
          return;
        }
        const data = await response.json();
        if (data?.profile) {
          setProfile({
            id: data.profile.id || '',
            name: data.profile.name || '',
            email: data.profile.email || '',
            role: data.profile.role || '',
            phone: data.profile.phone || '',
            address: data.profile.address || '',
            location: data.profile.location || '',
            subscriptionType: data.profile.subscriptionType || '',
            subscriptionStart: data.profile.subscriptionStart || '',
            subscriptionEnd: data.profile.subscriptionEnd || '',
          });
        }
      } catch {
        router.push('/login');
      } finally {
        setIsLoading(false);
      }
    };

    loadProfile();
  }, [router]);

  const startEditing = (field: EditableField) => {
    setEditingField(field);
    setDraftValue(profile[field] || '');
    setError('');
    setSuccess('');
  };

  const cancelEditing = () => {
```

```
    setEditingField(null);
    setDraftValue('');
  };

  const saveField = async (field: EditableField) => {
    setError('');
    setSuccess('');

    const nextValue = draftValue.trim();
    if (field === 'name' && !nextValue) {
      setError('Name is required.');
      return;
    }

    const nextProfile = { ...profile, [field]: nextValue };
    if (!nextProfile.name.trim()) {
      setError('Name is required.');
      return;
    }

    try {
      setSavingField(field);
      const response = await fetch('/api/profile', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: nextProfile.name,
          phone: nextProfile.phone,
          address: nextProfile.address,
          location: nextProfile.location,
        }),
      });
      const payload = await response.json().catch(() => ({}));
      if (!response.ok) {
        setError(payload?.error || 'Failed to update profile.');
        return;
      }
      setSuccess(`${field.charAt(0).toUpperCase()}${field.slice(1)} updated successfully.`);
      if (payload?.profile) {
        setProfile((prev) => ({
          ...prev,
          name: payload.profile.name || prev.name,
          phone: payload.profile.phone || '',
          address: payload.profile.address || '',
          location: payload.profile.location || '',
          subscriptionType: payload.profile.subscriptionType || prev.subscriptionType,
          subscriptionStart: payload.profile.subscriptionStart || prev.subscriptionStart,
          subscriptionEnd: payload.profile.subscriptionEnd || prev.subscriptionEnd,
        }));
      } else {
        setProfile(nextProfile);
      }
      setEditingField(null);
      setDraftValue('');
    } catch {
      setError('Failed to update profile.');
    } finally {
      setSavingField(null);
    }
  };

  if (isLoading) {
    return (
      <div className="flex min-h-screen items-center justify-center px-4">
        <p className="text-slate-300">Loading profile...</p>
      </div>
    );
  }

  const formatPlanDate = (value: string) => {
    if (!value) return 'Not set';
    const date = new Date(value);
    if (Number.isNaN(date.getTime())) return 'Not set';
    return date.toLocaleDateString('en-US', {
      year: 'numeric',
      month: 'long',
      day: 'numeric',
    });
  };

  const planName = profile.subscriptionType
    ? `${profile.subscriptionType.charAt(0).toUpperCase()}${profile.subscriptionType.slice(1)}
Plan`
    : 'No active plan';

  const renderEditableField = (
    label: string,
```

```
    field: EditableField,
    value: string,
    placeholder: string
  ) => {
    const isEditing = editingField === field;
    const isSaving = savingField === field;

    return (
      <div className="rounded-xl border border-white/10 bg-slate-900/60 p-4">
        <div className="mb-2 flex items-center justify-between">
          <p className="text-sm font-medium text-slate-200">{label}</p>
          {!isEditing && (
            <button
              type="button"
              onClick={() => startEditing(field)}
              className="rounded-lg border border-white/20 px-3 py-1 text-xs font-semibold
text-slate-200 transition hover:bg-white/10"
            >
              Edit
            </button>
          )}
        </div>
        {isEditing ? (
          <div className="space-y-2">
            <input
              type="text"
              value={draftValue}
              onChange={(event) => setDraftValue(event.target.value)}
              placeholder={placeholder}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-2.5
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
            />
            <div className="flex gap-2">
              <button
                type="button"
                onClick={() => saveField(field)}
                disabled={isSaving}
                className="rounded-lg bg-cyan-300 px-3 py-1.5 text-xs font-semibold text-
slate-900 transition hover:bg-cyan-200 disabled:cursor-not-allowed disabled:opacity-60"
              >
                {isSaving ? 'Saving...' : 'Save'}
              </button>
              <button
                type="button"
                onClick={cancelEditing}
                disabled={isSaving}
                className="rounded-lg border border-white/20 px-3 py-1.5 text-xs font-semibold
text-slate-200 transition hover:bg-white/10 disabled:cursor-not-allowed disabled:opacity-60"
              >
                Cancel
              </button>
            </div>
          </div>
        ) : (
          <p className="text-sm text-slate-100">{value || 'Not set'}</p>
        )}
      </div>
    );
  };

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-3xl rounded-3xl border border-white/10 bg-slate-900/80 p-
8">
        <p className="inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10 px-3
py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
          Account
        </p>
        <h1 className="mt-4 text-3xl font-bold text-white">Profile Details</h1>
        <p className="mt-2 text-slate-300">View and update your account details.</p>

        {profile.role === 'pharmacist' && (
          <div className="mt-6 rounded-2xl border border-white/10 bg-slate-950/60 p-5">
            <h2 className="text-lg font-semibold text-white">Plan Details</h2>
            <div className="mt-3 space-y-1 text-sm text-slate-200">
              <p><strong>Current Plan:</strong> {planName}</p>
              <p><strong>Start Date:</strong> {formatPlanDate(profile.subscriptionStart)}</p>
              <p><strong>End Date:</strong> {formatPlanDate(profile.subscriptionEnd)}</p>
            </div>
          </div>
        )}

        <div className="mt-6 space-y-4 rounded-2xl border border-white/10 bg-slate-950/60 p-
5">
          <div>
            <label className="mb-2 block text-sm font-medium text-slate-200">Email</label>
            <input
```

```
              type="email"
              value={profile.email}
              disabled
              className="w-full rounded-xl border border-slate-700 bg-slate-900/70 px-3 py-3
text-slate-400"
            />
          </div>

          {renderEditableField('Name', 'name', profile.name, 'Enter your full name')}
          {renderEditableField('Phone', 'phone', profile.phone, 'Enter phone number')}
          {renderEditableField('Address', 'address', profile.address, 'Enter address')}
          {profile.role === 'pharmacist' && (
            renderEditableField('Service Location (City/ZIP)', 'location', profile.location,
'Enter service location')
          )}

          {error && <p className="rounded-xl bg-rose-300/15 px-4 py-3 text-sm text-rose-100">
{error}</p>}
          {success && <p className="rounded-xl bg-emerald-300/15 px-4 py-3 text-sm text-
emerald-100">{success}</p>}
        </div>

        <div className="mt-6">
          <Link
            href="/"
            className="inline-flex rounded-full border border-white/20 px-5 py-2 text-sm font-
semibold text-slate-100 transition hover:bg-white/10"
          >
            Back to Home
          </Link>
        </div>
      </div>
    </div>
  );
}
```

[29] app/register/page.tsx FILE

```
'use client';

import { importLibrary, setOptions } from '@googlemaps/js-api-loader';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { useEffect, useRef, useState } from 'react';

type RegistrationMode = 'manual' | 'map';
type Coordinates = { lat: number; lng: number };

export default function RegisterPage() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
    confirmPassword: '',
    role: 'patient',
    phone: '',
    address: '',
    location: '',
  });
  const [message, setMessage] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isResolvingLocation, setIsResolvingLocation] = useState(false);
  const [locationHint, setLocationHint] = useState('');
  const [registrationMode, setRegistrationMode] = useState<RegistrationMode>('manual');
  const [mapPincode, setMapPincode] = useState('');
  const [selectedCoordinates, setSelectedCoordinates] = useState<Coordinates | null>(null);
  const [mapError, setMapError] = useState('');
  const mapRef = useRef<HTMLDivElement>(null);
  const mapInstanceRef = useRef<google.maps.Map | null>(null);
  const markerRef = useRef<google.maps.Marker | null>(null);
  const router = useRouter();

  const resolveCityFromIndianPincode = async (rawLocation: string) => {
    const value = rawLocation.trim();
    if (!/^[1-9][0-9]{5}$/.test(value)) return value;

    try {
      setIsResolvingLocation(true);
      setLocationHint('');

      const response = await fetch(`/api/google/pincode-city?
pincode=${encodeURIComponent(value)}`);
      if (!response.ok) {
```

```
        setLocationHint('Could not resolve city from PIN code.');
        return value;
      }

      const payload = (await response.json()) as { city?: string };
      const city = (payload.city || '').trim();
      if (!city) {
        setLocationHint('Could not resolve city from PIN code.');
        return value;
      }

      setLocationHint(`City selected: ${city}`);
      return city;
    } catch {
      setLocationHint('Could not resolve city from PIN code.');
      return value;
    } finally {
      setIsResolvingLocation(false);
    }
  };

  const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
    if (e.target.name === 'location') {
      setLocationHint('');
    }
  };

  const applySelectedMapLocation = (
    coordinates: Coordinates,
    city: string,
    formattedAddress: string,
    sourceLabel: string
  ) => {
    setSelectedCoordinates(coordinates);
    if (markerRef.current && mapInstanceRef.current) {
      markerRef.current.setMap(null);
    }
    if (mapInstanceRef.current) {
      markerRef.current = new google.maps.Marker({
        map: mapInstanceRef.current,
        position: coordinates,
        title: 'Your pharmacy location',
      });
      mapInstanceRef.current.panTo(coordinates);
      mapInstanceRef.current.setZoom(14);
    }

    setFormData((prev) => ({
      ...prev,
      location: city || prev.location,
      address: formattedAddress || prev.address,
    }));
    setLocationHint(`Pointer set from ${sourceLabel}: ${city || 'selected area'}`);
  };

  const handleLocateByPincode = async () => {
    if (!formData.name.trim()) {
      window.alert('Please enter the pharmacy name first.');
      return;
    }
    if (!/^[1-9][0-9]{5}$/.test(mapPincode.trim())) {
      setMessage('Enter a valid 6-digit Indian PIN code.');
      return;
    }
    if (!mapInstanceRef.current) {
      setMessage('Map is still loading. Please try again.');
      return;
    }

    setMessage('');
    setIsResolvingLocation(true);
    setLocationHint('');
    try {
      const response = await fetch(`/api/google/pincode-city?
pincode=${encodeURIComponent(mapPincode.trim())}`);
      if (!response.ok) {
        setLocationHint('Could not find this PIN code on map.');
        return;
      }

      const payload = (await response.json()) as {
        city?: string;
        formattedAddress?: string;
        coordinates?: Coordinates;
      };
```

```
      const coordinates = payload.coordinates;
      if (!coordinates || typeof coordinates.lat !== 'number' || typeof coordinates.lng !==
'number') {
        setLocationHint('Could not find this PIN code on map.');
        return;
      }

      const city = (payload.city || '').trim();
      const formattedAddress = (payload.formattedAddress || '').trim();
      const searchResponse = await fetch(
        `/api/google/nearby-pharmacies?q=${encodeURIComponent(
          `${formData.name.trim()} ${mapPincode.trim()}`
        )}&limit=25`
      );
      if (!searchResponse.ok) {
        setLocationHint('Could not search pharmacy by name at this PIN code.');
        return;
      }

      const searchResults = (await searchResponse.json()) as Array<{
        name?: string;
        address?: string;
        coordinates?: Coordinates;
      }>;
      const targetName = formData.name.trim().toLowerCase();
      const pinDigits = mapPincode.trim();

      const strictMatch = searchResults.find((item) => {
        const resultName = (item.name || '').trim().toLowerCase();
        const resultAddressDigits = (item.address || '').replace(/\D/g, '');
        return resultName === targetName && resultAddressDigits.includes(pinDigits);
      });

      const looseMatch = searchResults.find((item) => {
        const resultName = (item.name || '').trim().toLowerCase();
        const resultAddressDigits = (item.address || '').replace(/\D/g, '');
        return resultName.includes(targetName) && resultAddressDigits.includes(pinDigits);
      });

      const matched = strictMatch || looseMatch;
      if (!matched?.coordinates) {
        setLocationHint('No pharmacy found with this name at the given PIN code.');
        return;
      }

      const matchedCoordinates = matched.coordinates;
      const matchedAddress = (matched.address || '').trim();
      const matchedName = (matched.name || formData.name).trim();
      mapInstanceRef.current.panTo(matchedCoordinates);
      mapInstanceRef.current.setZoom(16);

      const shouldSetPointer = window.confirm(
        `Found "${matchedName}" at PIN ${mapPincode.trim()}. Set pointer to this location?`
      );

      if (shouldSetPointer) {
        applySelectedMapLocation(
          matchedCoordinates,
          city || formData.location,
          matchedAddress || formattedAddress,
          `PIN ${mapPincode.trim()}`
        );
      } else {
        setLocationHint('Location previewed. Pointer was not set.');
      }
    } catch {
      setLocationHint('Could not find this PIN code on map.');
    } finally {
      setIsResolvingLocation(false);
    }
  };

  useEffect(() => {
    if (formData.role !== 'pharmacist' || registrationMode !== 'map') return;
    if (!mapRef.current || mapInstanceRef.current) return;

    const initMap = async () => {
      const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
      if (!apiKey) {
        setMapError('Google Maps API key is missing.');
        return;
      }

      try {
        setOptions({ apiKey, version: 'weekly' });
        await importLibrary('maps');
```

```
        const map = new google.maps.Map(mapRef.current as HTMLDivElement, {
          center: { lat: 28.6139, lng: 77.2090 },
          zoom: 11,
        });
        mapInstanceRef.current = map;

        map.addListener('click', async (event: google.maps.MapMouseEvent) => {
          if (!event.latLng) return;

          const nextCoordinates = {
            lat: event.latLng.lat(),
            lng: event.latLng.lng(),
          };

          try {
            setIsResolvingLocation(true);
            const response = await fetch(
              `/api/google/reverse-geocode?
lat=${nextCoordinates.lat}&lng=${nextCoordinates.lng}`
            );
            if (!response.ok) {
              setLocationHint('Unable to resolve city from selected map point.');
              return;
            }
            const payload = (await response.json()) as { city?: string; formattedAddress?:
string };
            const city = (payload.city || '').trim();
            const formattedAddress = (payload.formattedAddress || '').trim();
            const shouldSetPointer = window.confirm(
              `Use ${city || 'this map point'} as pharmacy location?`
            );
            if (shouldSetPointer) {
              applySelectedMapLocation(nextCoordinates, city, formattedAddress, 'map click');
            } else {
              setLocationHint('Map point previewed. Pointer was not set.');
            }
          } catch {
            setLocationHint('Unable to resolve city from selected map point.');
          } finally {
            setIsResolvingLocation(false);
          }
        });
      } catch {
        setMapError('Unable to load Google Maps.');
      }
    };

    initMap();
  }, [formData.role, registrationMode]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsLoading(true);
    setMessage('');

    if (formData.password !== formData.confirmPassword) {
      setMessage('Passwords do not match');
      setIsLoading(false);
      return;
    }

    if (formData.password.length < 6) {
      setMessage('Password must be at least 6 characters long');
      setIsLoading(false);
      return;
    }

    try {
      if (formData.role === 'pharmacist' && registrationMode === 'map' &&
!selectedCoordinates) {
        setMessage('Please select your pharmacy location on the map.');
        setIsLoading(false);
        return;
      }

      const resolvedLocation =
        formData.role === 'pharmacist' && registrationMode === 'manual'
          ? await resolveCityFromIndianPincode(formData.location)
          : formData.location;

      if (formData.role === 'pharmacist' && resolvedLocation !== formData.location) {
        setFormData((prev) => ({ ...prev, location: resolvedLocation }));
      }

      const response = await fetch('/api/users', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
```

```
        body: JSON.stringify({
          ...formData,
          location: resolvedLocation,
          pharmacyCoordinates: formData.role === 'pharmacist' ? selectedCoordinates : null,
          subscriptionType: formData.role === 'pharmacist' ? null : undefined,
        }),
      });

      const result = await response.json();

      if (response.ok) {
        setMessage('Registration successful! You can now login.');
        setTimeout(() => {
          router.push('/login');
        }, 2000);
      } else {
        setMessage(result.error || 'Registration failed');
      }
    } catch {
      setMessage('An error occurred. Please try again.');
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className="min-h-screen px-4 py-14 sm:px-6 lg:px-8">
      <div className="mx-auto w-full max-w-3xl rounded-3xl border border-white/10 bg-slate-
900/85 p-8 shadow-2xl">
        <div className="mb-6">
          <p className="inline-flex rounded-full border border-emerald-300/30 bg-emerald-
300/10 px-3 py-1 text-xs font-semibold uppercase tracking-[0.14em] text-emerald-100">
            Create Account
          </p>
          <h1 className="mt-4 text-3xl font-bold text-white">Join E-Pharmacy</h1>
          <p className="mt-2 text-slate-300">
            Register as a patient or pharmacy and start using the platform immediately.
          </p>
        </div>

        <form onSubmit={handleSubmit} className="grid grid-cols-1 gap-5 md:grid-cols-2">
          <div className="md:col-span-2">
            <label className="mb-2 block text-sm font-medium text-slate-200">I am a</label>
            <select
              name="role"
              value={formData.role}
              onChange={(e) => {
                handleChange(e);
                if (e.target.value !== 'pharmacist') {
                  setRegistrationMode('manual');
                  setSelectedCoordinates(null);
                  setLocationHint('');
                }
              }}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
            >
              <option value="patient">Patient</option>
              <option value="pharmacist">Pharmacist</option>
            </select>
          </div>

          <div className="md:col-span-2">
            <label className="mb-2 block text-sm font-medium text-slate-200">
              {formData.role === 'patient' ? 'Full Name' : 'Pharmacy Name'}
            </label>
            <input
              type="text"
              name="name"
              value={formData.name}
              onChange={handleChange}
              required
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder={formData.role === 'patient' ? 'Enter your full name' : 'Enter
pharmacy name'}
            />
          </div>

          <div className="md:col-span-2">
            <label className="mb-2 block text-sm font-medium text-slate-200">Email</label>
            <input
              type="email"
              name="email"
              value={formData.email}
              onChange={handleChange}
              required
```

```
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your email"
            />
          </div>

          <div>
            <label className="mb-2 block text-sm font-medium text-slate-200">Password</label>
            <input
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
              required
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your password"
            />
          </div>

          <div>
            <label className="mb-2 block text-sm font-medium text-slate-200">Confirm
Password</label>
            <input
              type="password"
              name="confirmPassword"
              value={formData.confirmPassword}
              onChange={handleChange}
              required
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Confirm your password"
            />
          </div>

          <div>
            <label className="mb-2 block text-sm font-medium text-slate-200">Phone</label>
            <input
              type="tel"
              name="phone"
              value={formData.phone}
              onChange={handleChange}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your phone number"
            />
          </div>

          <div>
            <label className="mb-2 block text-sm font-medium text-slate-200">Address</label>
            <textarea
              name="address"
              value={formData.address}
              onChange={handleChange}
              rows={3}
              className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
              placeholder="Enter your address"
            />
          </div>

          {formData.role === 'pharmacist' && (
            <>
              <div className="md:col-span-2">
                <label className="mb-2 block text-sm font-medium text-slate-200">
                  Pharmacy Setup Method
                </label>
                <div className="flex gap-3">
                  <button
                    type="button"
                    onClick={() => setRegistrationMode('map')}
                    className={`rounded-lg px-4 py-2 text-sm font-semibold transition ${
                      registrationMode === 'map'
                        ? 'bg-cyan-300 text-slate-900'
                        : 'border border-white/20 text-slate-200 hover:bg-white/10'
                    }`}
                  >
                    Add on Map
                  </button>
                  <button
                    type="button"
                    onClick={() => setRegistrationMode('manual')}
                    className={`rounded-lg px-4 py-2 text-sm font-semibold transition ${
                      registrationMode === 'manual'
                        ? 'bg-cyan-300 text-slate-900'
                        : 'border border-white/20 text-slate-200 hover:bg-white/10'
                    }`}
```

```
                >
                  Manual Details
                </button>
              </div>
            </div>

            {registrationMode === 'map' && (
              <div className="md:col-span-2">
                <div className="mb-3 grid grid-cols-1 gap-2 sm:grid-cols-[1fr_auto]">
                  <input
                    type="text"
                    value={mapPincode}
                    onChange={(e) => setMapPincode(e.target.value.replace(/\D/g,
'').slice(0, 6))}
                    className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3
py-3 text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
                    placeholder="Enter 6-digit PIN code"
                  />
                  <button
                    type="button"
                    onClick={handleLocateByPincode}
                    className="rounded-xl bg-cyan-300 px-4 py-3 font-semibold text-slate-900
transition hover:bg-cyan-200 disabled:cursor-not-allowed disabled:opacity-60"
                    disabled={isResolvingLocation}
                  >
                    Find on Map
                  </button>
                </div>
                <p className="mb-2 text-sm text-slate-300">
                  Enter PIN, find location on map, then confirm to set pointer. You can also
click map directly.
                </p>
                <div className="overflow-hidden rounded-xl border border-white/10 bg-slate-
950/70">
                  <div ref={mapRef} className="h-72 w-full" />
                </div>
                {mapError && <p className="mt-2 text-xs text-rose-200">{mapError}</p>}
                {selectedCoordinates && (
                  <p className="mt-2 text-xs text-cyan-200">
                    Selected: {selectedCoordinates.lat.toFixed(5)},
{selectedCoordinates.lng.toFixed(5)}
                  </p>
                )}
              </div>
            )}

            {registrationMode === 'manual' && (
              <div className="md:col-span-2">
                <label className="mb-2 block text-sm font-medium text-slate-200">
                  Service Location (Indian City/PIN)
                </label>
                <input
                  type="text"
                  name="location"
                  value={formData.location}
                  onChange={handleChange}
                  onBlur={async (e) => {
                    const city = await resolveCityFromIndianPincode(e.target.value);
                    if (city !== e.target.value) {
                      setFormData((prev) => ({ ...prev, location: city }));
                    }
                  }}
                  required
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-
3 text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
                  placeholder="Enter Indian city or 6-digit PIN"
                />
              </div>
            )}

            {isResolvingLocation && (
              <div className="md:col-span-2">
                <p className="mt-2 text-xs text-cyan-200">Resolving location...</p>
              </div>
            )}
            {!isResolvingLocation && locationHint && (
              <div className="md:col-span-2">
                <p className="mt-2 text-xs text-cyan-200">{locationHint}</p>
              </div>
            )}
          </>
        )}

        <div className="md:col-span-2">
          <button
            type="submit"
            disabled={isLoading}
```

```
              className="w-full rounded-xl bg-cyan-300 px-4 py-3 font-semibold text-slate-900
transition hover:bg-cyan-200 disabled:cursor-not-allowed disabled:opacity-60"
            >
              {isLoading ? 'Registering...' : 'Register'}
            </button>
          </div>
        </form>

        {message && (
          <p
            className={`mt-4 rounded-xl px-4 py-3 text-center text-sm ${
              message.includes('successful')
                ? 'bg-emerald-300/15 text-emerald-100'
                : 'bg-rose-300/15 text-rose-100'
            }`}
          >
            {message}
          </p>
        )}

        <p className="mt-6 text-sm text-slate-300">
          Already have an account?{' '}
          <Link href="/login" className="font-semibold text-cyan-200 hover:text-cyan-100">
            Login here
          </Link>
        </p>
      </div>
    </div>
  );
}
```

[30] app/settings/page.tsx FILE

```
'use client';

import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { FormEvent, useEffect, useState } from 'react';

interface AuthUser {
  id: string;
  name: string;
  email: string;
  role: string;
}

export default function SettingsPage() {
  const router = useRouter();
  const [user, setUser] = useState<AuthUser | null>(null);
  const [isCheckingAuth, setIsCheckingAuth] = useState(true);
  const [isSaving, setIsSaving] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
  const [currentPassword, setCurrentPassword] = useState('');
  const [newPassword, setNewPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');

  useEffect(() => {
    const checkAuth = async () => {
      try {
        const response = await fetch('/api/auth/me');
        if (!response.ok) {
          router.push('/login');
          return;
        }

        const data = await response.json();
        setUser(data.user);
      } catch {
        router.push('/login');
      } finally {
        setIsCheckingAuth(false);
      }
    };

    checkAuth();
  }, [router]);

  const handleSubmit = async (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    setError('');
    setSuccess('');
```

```
    if (!currentPassword || !newPassword || !confirmPassword) {
      setError('Please fill in all password fields.');
      return;
    }
    if (newPassword.length < 8) {
      setError('New password must be at least 8 characters.');
      return;
    }
    if (newPassword !== confirmPassword) {
      setError('New password and confirm password do not match.');
      return;
    }
    if (currentPassword === newPassword) {
      setError('New password must be different from current password.');
      return;
    }

    try {
      setIsSaving(true);
      const response = await fetch('/api/auth/change-password', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          currentPassword,
          newPassword,
          confirmPassword,
        }),
      });
      const payload = await response.json().catch(() => ({}));

      if (!response.ok) {
        setError(payload?.error || 'Failed to change password.');
        return;
      }

      setSuccess('Password changed successfully.');
      setCurrentPassword('');
      setNewPassword('');
      setConfirmPassword('');
    } catch {
      setError('Failed to change password.');
    } finally {
      setIsSaving(false);
    }
  };

  if (isCheckingAuth) {
    return (
      <div className="flex min-h-screen items-center justify-center px-4">
        <p className="text-slate-300">Loading settings...</p>
      </div>
    );
  }

  return (
    <div className="min-h-screen px-4 py-10 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-3xl rounded-3xl border border-white/10 bg-slate-900/80 p-
8">
        <p className="inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10 px-3
py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
          Preferences
        </p>
        <h1 className="mt-4 text-3xl font-bold text-white">Settings</h1>
        <p className="mt-2 text-slate-300">
          Signed in as {user?.name || 'User'} ({user?.role || 'account'}). You can change your
password below.
        </p>

        <div className="mt-6 rounded-2xl border border-white/10 bg-slate-950/60 p-5">
          <h2 className="text-lg font-semibold text-white">Change Password</h2>
          <form onSubmit={handleSubmit} className="mt-4 space-y-4">
            <div>
              <label className="mb-2 block text-sm font-medium text-slate-200">Current
Password</label>
              <input
                type="password"
                value={currentPassword}
                onChange={(event) => setCurrentPassword(event.target.value)}
                className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
                placeholder="Enter current password"
              />
            </div>
            <div>
              <label className="mb-2 block text-sm font-medium text-slate-200">New
Password</label>
              <input
```

```
                type="password"
                value={newPassword}
                onChange={(event) => setNewPassword(event.target.value)}
                className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
                placeholder="Enter new password"
              />
            </div>
            <div>
              <label className="mb-2 block text-sm font-medium text-slate-200">Confirm New
Password</label>
              <input
                type="password"
                value={confirmPassword}
                onChange={(event) => setConfirmPassword(event.target.value)}
                className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none"
                placeholder="Confirm new password"
              />
            </div>
            <button
              type="submit"
              disabled={isSaving}
              className="w-full rounded-xl bg-cyan-300 px-4 py-3 font-semibold text-slate-900
transition hover:bg-cyan-200 disabled:cursor-not-allowed disabled:opacity-60"
            >
              {isSaving ? 'Updating Password...' : 'Update Password'}
            </button>
          </form>

          {error && <p className="mt-4 rounded-xl bg-rose-300/15 px-4 py-3 text-sm text-rose-
100">{error}</p>}
          {success && <p className="mt-4 rounded-xl bg-emerald-300/15 px-4 py-3 text-sm text-
emerald-100">{success}</p>}
        </div>

        <div className="mt-6">
          <Link
            href="/"
            className="inline-flex rounded-full border border-white/20 px-5 py-2 text-sm font-
semibold text-slate-100 transition hover:bg-white/10"
          >
            Back to Home
          </Link>
        </div>
      </div>
    </div>
  );
}
```

[31] app/subscribe/page.tsx FILE

```
'use client';

import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { useState } from 'react';
import { useEffect } from 'react';

interface AuthUser {
  id: string;
  name: string;
  email: string;
  role: string;
}

export default function SubscribePage() {
  const router = useRouter();
  const [authUser, setAuthUser] = useState<AuthUser | null>(null);
  const [isCheckingAuth, setIsCheckingAuth] = useState(true);
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
    phone: '',
    address: '',
    location: '',
    subscriptionType: '',
  });
  const [message, setMessage] = useState('');
  const [selectedPlan, setSelectedPlan] = useState('');

  useEffect(() => {
```

```
    const checkAuth = async () => {
      try {
        const response = await fetch('/api/auth/me');
        if (!response.ok) {
          setAuthUser(null);
          return;
        }
        const data = await response.json();
        setAuthUser(data?.user || null);
      } catch {
        setAuthUser(null);
      } finally {
        setIsCheckingAuth(false);
      }
    };

    checkAuth();
  }, []);

  const plans = [
    {
      id: 'monthly',
      name: 'Monthly Plan',
      price: '₹299/month',
      features: ['Access to prescriptions', 'Basic dashboard', 'Email support'],
      popular: false,
    },
    {
      id: 'yearly',
      name: 'Yearly Plan',
      price: '₹999/year',
      features: ['All monthly features', 'Priority matching', 'Phone support', 'Save 20%'],
      popular: true,
    },
    {
      id: 'premium',
      name: 'Premium Plan',
      price: '₹2999/year',
      features: ['All yearly features', 'Advanced analytics', 'Dedicated account manager',
'Custom integrations'],
      popular: false,
    },
  ];

  const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handlePlanSelect = (planId: string) => {
    const plan = plans.find((item) => item.id === planId);
    const planName = plan?.name || 'this plan';
    const shouldProceed = window.confirm(
      `You selected ${planName}. Do you want to continue and join this plan?`
    );

    if (!shouldProceed) {
      setMessage('Plan selection cancelled.');
      return;
    }

    setMessage('Payment gateway will be integrated soon.');
    setSelectedPlan(planId);
    setFormData({ ...formData, subscriptionType: planId });
  };

  const activateLoggedInPharmacistPlan = async () => {
    if (!authUser || authUser.role !== 'pharmacist' || !selectedPlan) return;
    try {
      const response = await fetch('/api/subscription/activate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ planId: selectedPlan }),
      });
      const payload = await response.json().catch(() => ({}));
      if (!response.ok) {
        setMessage(payload?.error || 'Failed to activate plan.');
        return;
      }
      const selectedPlanName = plans.find((plan) => plan.id === selectedPlan)?.name ||
'selected plan';
      setMessage(`${selectedPlanName} activated successfully. You can view it in Profile
Details.`);
    } catch {
      setMessage('Failed to activate plan.');
    }
  };
```

```
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (authUser) {
      setMessage('Please logout to create a new pharmacist subscription account.');
      return;
    }
    if (!selectedPlan) {
      setMessage('Please select a subscription plan.');
      return;
    }
    try {
      const response = await fetch('/api/users', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ ...formData, role: 'pharmacist' }),
      });
      const result = await response.json();
      if (response.ok) {
        setMessage('Subscription successful! You can now access the dashboard.');
        setFormData({
          name: '',
          email: '',
          password: '',
          phone: '',
          address: '',
          location: '',
          subscriptionType: '',
        });
        setSelectedPlan('');
      } else {
        setMessage(result.error || 'Failed to subscribe');
      }
    } catch {
      setMessage('Error subscribing');
    }
  };

  return (
    <div className="min-h-screen px-4 py-14 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-7xl">
        <div className="mb-10 text-center">
          <p className="inline-flex rounded-full border border-cyan-300/30 bg-cyan-300/10 px-3
py-1 text-xs font-semibold uppercase tracking-[0.14em] text-cyan-200">
            Pharmacy Subscription
          </p>
          <h1 className="mt-4 text-4xl font-bold text-white">Grow with Local Prescription
Demand</h1>
          <p className="mt-2 text-slate-300">
            Pick a plan, activate your profile, and start receiving nearby prescription
requests.
          </p>
        </div>

        <div className="grid grid-cols-1 gap-6 md:grid-cols-3">
          {plans.map((plan) => (
            <article
              key={plan.id}
              className={`rounded-3xl border p-6 transition ${
                selectedPlan === plan.id
                  ? 'border-cyan-300 bg-cyan-300/10'
                  : 'border-white/10 bg-slate-900/80 hover:border-white/25'
              }`}
            >
              {plan.popular && (
                <p className="mb-4 inline-flex rounded-full bg-emerald-300/20 px-3 py-1 text-
xs font-semibold text-emerald-100">
                  Most Popular
                </p>
              )}
              <h2 className="text-2xl font-bold text-white">{plan.name}</h2>
              <p className="mt-2 text-3xl font-bold text-cyan-200">{plan.price}</p>
              <ul className="mt-5 space-y-2 text-sm text-slate-200">
                {plan.features.map((feature) => (
                  <li key={feature}>• {feature}</li>
                ))}
              </ul>
              <button
                className={`mt-6 w-full rounded-xl px-4 py-3 font-semibold transition ${
                  selectedPlan === plan.id
                    ? 'bg-cyan-300 text-slate-900'
                    : 'border border-white/20 text-slate-100 hover:bg-white/10'
                }`}
                onClick={() => handlePlanSelect(plan.id)}
              >
                {selectedPlan === plan.id ? 'Selected' : 'Select Plan'}
              </button>
            </article>
```

```
          ))}
        </div>

        {!isCheckingAuth && !authUser && selectedPlan && (
          <div className="mx-auto mt-10 max-w-2xl rounded-3xl border border-white/10 bg-slate-
900/85 p-8 shadow-2xl">
            <h2 className="text-2xl font-bold text-white">Complete Your Subscription</h2>
            <p className="mt-2 text-slate-300">
              Fill in your details to subscribe to the {plans.find((plan) => plan.id ===
selectedPlan)?.name}.
            </p>
            <form onSubmit={handleSubmit} className="mt-6 grid grid-cols-1 gap-4 md:grid-cols-
2">
              <div className="md:col-span-2">
                <label className="mb-2 block text-sm text-slate-200">Pharmacy Name</label>
                <input
                  type="text"
                  name="name"
                  value={formData.name}
                  onChange={handleChange}
                  required
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div>
                <label className="mb-2 block text-sm text-slate-200">Email</label>
                <input
                  type="email"
                  name="email"
                  value={formData.email}
                  onChange={handleChange}
                  required
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div>
                <label className="mb-2 block text-sm text-slate-200">Password</label>
                <input
                  type="password"
                  name="password"
                  value={formData.password}
                  onChange={handleChange}
                  required
                  minLength={6}
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div>
                <label className="mb-2 block text-sm text-slate-200">Phone</label>
                <input
                  type="tel"
                  name="phone"
                  value={formData.phone}
                  onChange={handleChange}
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div>
                <label className="mb-2 block text-sm text-slate-200">Address</label>
                <textarea
                  name="address"
                  value={formData.address}
                  onChange={handleChange}
                  rows={3}
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div>
                <label className="mb-2 block text-sm text-slate-200">Service Location
(City/ZIP)</label>
                <input
                  type="text"
                  name="location"
                  value={formData.location}
                  onChange={handleChange}
                  required
                  className="w-full rounded-xl border border-slate-700 bg-slate-950 px-3 py-3
text-slate-100 focus:border-cyan-300 focus:outline-none"
                />
              </div>
              <div className="md:col-span-2">
                <button
```

```
                  type="submit"
                  className="w-full rounded-xl bg-cyan-300 px-4 py-3 font-semibold text-slate-900 transition hover:bg-cyan-200"
                >
                  Subscribe to {plans.find((plan) => plan.id === selectedPlan)?.name}
                </button>
              </div>
            </form>
            {message && (
              <p className="mt-4 rounded-xl bg-white/10 px-4 py-3 text-center text-sm text-slate-100">{message}</p>
            )}
          </div>
        )}

        {!isCheckingAuth && authUser?.role === 'pharmacist' && selectedPlan && (
          <div className="mx-auto mt-10 max-w-2xl rounded-3xl border border-white/10 bg-slate-900/85 p-8 shadow-2xl">
            <h2 className="text-2xl font-bold text-white">You are already logged in</h2>
            <p className="mt-2 text-slate-300">
              Payment flow is pending integration. For now, you can directly activate the selected plan.
            </p>
            <div className="mt-6 flex flex-wrap gap-3">
              <button
                type="button"
                onClick={activateLoggedInPharmacistPlan}
                className="rounded-xl bg-cyan-300 px-4 py-2 font-semibold text-slate-900 transition hover:bg-cyan-200"
              >
                Activate {plans.find((plan) => plan.id === selectedPlan)?.name}
              </button>
              <button
                type="button"
                onClick={() => router.push('/dashboard')}
                className="rounded-xl border border-white/20 px-4 py-2 font-semibold text-slate-100 transition hover:bg-white/10"
              >
                Go to Dashboard
              </button>
              <Link
                href="/pharmacies"
                className="rounded-xl border border-white/20 px-4 py-2 font-semibold text-slate-100 transition hover:bg-white/10"
              >
                Browse Pharmacies
              </Link>
            </div>
          </div>
        )}

        {!isCheckingAuth && authUser?.role === 'patient' && selectedPlan && (
          <div className="mx-auto mt-10 max-w-2xl rounded-3xl border border-white/10 bg-slate-900/85 p-8 shadow-2xl">
            <h2 className="text-2xl font-bold text-white">Pharmacist-only signup</h2>
            <p className="mt-2 text-slate-300">
              Complete subscription is for pharmacist account registration. Logout first to create a pharmacist account.
            </p>
          </div>
        )}
      </div>
    </div>
  );
}
```

[32] app/upload/page.tsx FILE

```
'use client';

import { useState, useEffect, useRef } from 'react';
import Link from 'next/link';
import PharmacyMap from '@/components/PharmacyMap';

interface Pharmacy {
  _id: string;
  name: string;
  email: string;
  phone: string;
  address: string;
  location: string;
  coordinates: {
    lat: number;
```

```
    lng: number;
  };
  supportsPrescriptionUpload: boolean;
  isUsingService: boolean;
  subscriptionType: string | null;
  rating: number;
  reviewCount: number;
  operatingHours?: Record<string, unknown>;
  services?: string[];
  distanceKm?: number;
}

interface User {
  id: string;
  name: string;
  email: string;
  role: string;
}

const STORAGE_PATIENT_NOTIFICATIONS_KEY = 'patient_notifications';
const STORAGE_PATIENT_UNREAD_KEY = 'patient_notifications_unread';

export default function UploadPage() {
  const [user, setUser] = useState<User | null>(null);
  const [formData, setFormData] = useState({
    patientName: '',
    patientEmail: '',
    patientPhone: '',
    patientAddress: '',
    location: '',
    prescriptionImages: [] as File[],
    notes: '',
  });
  const [selectedPharmacyIds, setSelectedPharmacyIds] = useState<string[]>([]);
  const [availablePharmacies, setAvailablePharmacies] = useState<Pharmacy[]>([]);
  const [activePharmacyId, setActivePharmacyId] = useState<string | null>(null);
  const [isSearchingPharmacies, setIsSearchingPharmacies] = useState(false);
  const [message, setMessage] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [dragActive, setDragActive] = useState(false);
  const [previews, setPreviews] = useState<string[]>([]);
  const [uploadProgress, setUploadProgress] = useState(0);
  const [userCoordinates, setUserCoordinates] = useState<{ lat: number; lng: number } | null>
(null);
  const fileInputRef = useRef<HTMLInputElement>(null);
  const searchTimeoutRef = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    checkAuthStatus();
    loadDefaultPharmacies();
  }, []);

  useEffect(() => {
    return () => {
      if (searchTimeoutRef.current) {
        clearTimeout(searchTimeoutRef.current);
      }
    };
  }, []);

  const checkAuthStatus = async () => {
    try {
      const response = await fetch('/api/auth/me?optional=1');
      if (!response.ok) return;

      const userData = await response.json();
      if (!userData?.user) return;

      setUser(userData.user);

      // Auto-fill user information if logged in as patient.
      if (userData.user.role === 'patient') {
        let phone = '';
        let address = '';

        // Prefer richer profile data when available.
        try {
          const profileResponse = await fetch('/api/profile');
          if (profileResponse.ok) {
            const profileData = await profileResponse.json();
            phone = profileData?.profile?.phone || '';
            address = profileData?.profile?.address || '';
          }
        } catch {
          // Fall back to basic auth data below.
        }
```

```
      setFormData((prev) => ({
        ...prev,
        patientName: userData.user.name || '',
        patientEmail: userData.user.email || '',
        patientPhone: phone,
        patientAddress: address,
      }));
    }
  } catch (error) {
    // User not authenticated, but that's okay for anonymous uploads
  }
};

const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};

const loadDefaultPharmacies = async () => {
  setIsSearchingPharmacies(true);
  try {
    const response = await fetch('/api/pharmacies?limit=30');
    if (response.ok) {
      const pharmacies = (await response.json()) as Pharmacy[];
      setAvailablePharmacies(pharmacies);
    } else {
      setAvailablePharmacies([]);
    }
  } catch (error) {
    console.error('Error loading default pharmacies:', error);
    setAvailablePharmacies([]);
  } finally {
    setIsSearchingPharmacies(false);
  }
};

const getDistanceInKm = (lat1: number, lng1: number, lat2: number, lng2: number) => {
  const toRad = (value: number) => (value * Math.PI) / 180;
  const earthRadiusKm = 6371;
  const dLat = toRad(lat2 - lat1);
  const dLng = toRad(lng2 - lng1);
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLng / 2) * Math.sin(dLng / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return earthRadiusKm * c;
};

const searchPharmacies = async (
  params: { location?: string; lat?: number; lng?: number; radius?: number; fetchAll?:
boolean } = {}
) => {
  const { location, lat, lng, radius = 100000, fetchAll = false } = params;
  const hasLocationText = !!location?.trim();
  const hasCoordinates = typeof lat === 'number' && typeof lng === 'number';

  if (!fetchAll && !hasLocationText && !hasCoordinates) return [] as Pharmacy[];

  setIsSearchingPharmacies(true);
  try {
    let url = '/api/pharmacies?limit=50';
    if (fetchAll) {
      // keep base URL
    } else if (hasCoordinates) {
      url += `&lat=${lat}&lng=${lng}&radius=${radius}`;
    } else if (hasLocationText) {
      url += `&location=${encodeURIComponent(location as string)}`;
    }

    const response = await fetch(url);
    if (response.ok) {
      const pharmacies = (await response.json()) as Pharmacy[];
      if (hasCoordinates) {
        const withDistance = pharmacies
          .map((pharmacy) => ({
            ...pharmacy,
            distanceKm: getDistanceInKm(
              lat as number,
              lng as number,
              pharmacy.coordinates.lat,
              pharmacy.coordinates.lng
            ),
          }))
          .sort((a, b) => (a.distanceKm || 0) - (b.distanceKm || 0));
        setAvailablePharmacies(withDistance);
        return withDistance;
      } else {
        setAvailablePharmacies(pharmacies);
```

```
          return pharmacies;
        }
      } else {
        setAvailablePharmacies([]);
        return [] as Pharmacy[];
      }
    } catch (error) {
      console.error('Error searching pharmacies:', error);
      setAvailablePharmacies([]);
      return [] as Pharmacy[];
    } finally {
      setIsSearchingPharmacies(false);
    }
    return [] as Pharmacy[];
  };

  const getCurrentLocationAndNearestPharmacies = () => {
    if (!navigator.geolocation) {
      setMessage('Geolocation is not supported by this browser.');
      return;
    }

    navigator.geolocation.getCurrentPosition(
      async (position) => {
        const { latitude, longitude } = position.coords;
        setUserCoordinates({ lat: latitude, lng: longitude });
        setSelectedPharmacyIds([]);
        setActivePharmacyId(null);

        let resolvedCity = '';
        try {
          const cityResponse = await fetch(`/api/google/reverse-geocode?
lat=${latitude}&lng=${longitude}`);
          if (cityResponse.ok) {
            const payload = await cityResponse.json();
            resolvedCity = payload?.city || '';
          }
        } catch (error) {
          console.error('Failed to resolve city from coordinates:', error);
        }

        if (resolvedCity) {
          setFormData((prev) => ({ ...prev, location: resolvedCity }));
        }

        let found = await searchPharmacies({ lat: latitude, lng: longitude, radius: 100000 });
        if (found.length === 0 && resolvedCity) {
          found = await searchPharmacies({ location: resolvedCity });
        }
        if (found.length === 0) {
          found = await searchPharmacies({ fetchAll: true });
          if (found.length > 0) {
            setMessage('Showing closest available pharmacies.');
          } else {
            setMessage('No pharmacies are currently available. Please try again later.');
          }
        }
      },
      () => {
        setMessage('Unable to get your location. Please allow location access.');
      }
    );
  };

  const handleLocationChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const newLocation = e.target.value;
    setFormData({ ...formData, location: newLocation });
    setUserCoordinates(null);

    // Clear previous selections when location changes
    setSelectedPharmacyIds([]);
    setActivePharmacyId(null);
    setAvailablePharmacies([]);

    // Clear previous timeout
    if (searchTimeoutRef.current) {
      clearTimeout(searchTimeoutRef.current);
    }

    if (!newLocation.trim()) {
      loadDefaultPharmacies();
      return;
    }

    // Trigger search immediately if location is at least 3 characters, otherwise debounce
    if (newLocation.trim().length >= 3) {
      searchTimeoutRef.current = setTimeout(() => {
```

```
        searchPharmacies({ location: newLocation });
        searchTimeoutRef.current = null;
      }, 150); // Quick response for longer searches
    } else if (newLocation.trim()) {
      searchTimeoutRef.current = setTimeout(() => {
        searchPharmacies({ location: newLocation });
        searchTimeoutRef.current = null;
      }, 500); // Longer debounce for short inputs
    }
  };

  const togglePharmacySelection = (pharmacyId: string) => {
    setActivePharmacyId(pharmacyId);
    setSelectedPharmacyIds(prev =>
      prev.includes(pharmacyId)
        ? prev.filter(id => id !== pharmacyId)
        : [...prev, pharmacyId]
    );
  };

  const handleMapPharmacySelect = (pharmacy: Pharmacy) => {
    setActivePharmacyId(pharmacy._id);
    setSelectedPharmacyIds((prev) =>
      prev.includes(pharmacy._id) ? prev : [...prev, pharmacy._id]
    );
  };

  const toggleSelectAllPharmacies = () => {
    const allIds = availablePharmacies.map((pharmacy) => pharmacy._id);
    const allSelected = allIds.length > 0 && allIds.every((id) =>
selectedPharmacyIds.includes(id));
    setSelectedPharmacyIds(allSelected ? [] : allIds);
    setActivePharmacyId(allSelected ? null : allIds[0] || null);
  };

  const validateFile = (file: File): string | null => {
    const maxSize = 5 * 1024 * 1024; // 5MB
    const allowedTypes = ['image/jpeg', 'image/jpg', 'image/png', 'image/gif', 'image/webp'];

    if (!allowedTypes.includes(file.type)) {
      return 'Please upload only image files (JPEG, PNG, GIF, WebP)';
    }

    if (file.size > maxSize) {
      return 'File size must be less than 5MB';
    }

    return null;
  };

  const handleFileChange = (files: FileList | null) => {
    if (!files) return;

    const fileArray = Array.from(files);
    const validFiles: File[] = [];
    const newPreviews: string[] = [];

    for (const file of fileArray) {
      const error = validateFile(file);
      if (error) {
        setMessage(error);
        return;
      }

      validFiles.push(file);

      // Create preview
      const reader = new FileReader();
      reader.onloadend = () => {
        newPreviews.push(reader.result as string);
        if (newPreviews.length === validFiles.length) {
          setPreviews(prev => [...prev, ...newPreviews]);
        }
      };
      reader.readAsDataURL(file);
    }

    setFormData(prev => ({
      ...prev,
      prescriptionImages: [...prev.prescriptionImages, ...validFiles]
    }));
    setMessage('');
  };

  const handleDrag = (e: React.DragEvent) => {
    e.preventDefault();
    e.stopPropagation();
```

```
    if (e.type === 'dragenter' || e.type === 'dragover') {
      setDragActive(true);
    } else if (e.type === 'dragleave') {
      setDragActive(false);
    }
  };

  const handleDrop = (e: React.DragEvent) => {
    e.preventDefault();
    e.stopPropagation();
    setDragActive(false);

    if (e.dataTransfer.files && e.dataTransfer.files[0]) {
      handleFileChange(e.dataTransfer.files);
    }
  };

  const removeImage = (index: number) => {
    setFormData(prev => ({
      ...prev,
      prescriptionImages: prev.prescriptionImages.filter((_, i) => i !== index)
    }));
    setPreviews(prev => prev.filter((_, i) => i !== index));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    if (formData.prescriptionImages.length === 0) {
      setMessage('Please upload at least one prescription image');
      return;
    }

    if (selectedPharmacyIds.length === 0) {
      setMessage('Please select at least one pharmacy');
      return;
    }

    setIsLoading(true);
    setUploadProgress(0);

    try {
      // Convert images to base64
      const imagePromises = formData.prescriptionImages.map((file, index) => {
        return new Promise<string>((resolve) => {
          const reader = new FileReader();
          reader.onloadend = () => {
            setUploadProgress(((index + 1) / formData.prescriptionImages.length) * 50);
            resolve(reader.result as string);
          };
          reader.readAsDataURL(file);
        });
      });

      const prescriptionImages = await Promise.all(imagePromises);

      const submitData = {
        ...formData,
        prescriptionImages,
        selectedPharmacyIds,
      };

      setUploadProgress(75);

      const response = await fetch('/api/prescriptions', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(submitData),
      });

      const result = await response.json();

      setUploadProgress(100);

      if (response.ok) {
        setMessage('Prescription uploaded successfully! Selected pharmacies will be
notified.');
        if (user?.role === 'patient') {
          try {
            const notification = {
              id: `patient-upload-${Date.now()}`,
              title: 'Prescription Uploaded',
              message: 'Your prescription was submitted and sent to selected pharmacies.',
              createdAt: new Date().toISOString(),
            };
            const existingRaw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
            const existing = existingRaw ? (JSON.parse(existingRaw) as typeof notification[])
```

```
: [];
            const merged = [notification, ...existing].slice(0, 100);
            localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(merged));
            const unreadCount = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) ||
'0');
            localStorage.setItem(STORAGE_PATIENT_UNREAD_KEY, String(unreadCount + 1));
            window.dispatchEvent(new Event('notification-updated'));
          } catch {
            // Do not fail successful upload if local storage writes fail.
          }
        }
        setFormData({
          patientName: user?.name || '',
          patientEmail: user?.email || '',
          patientPhone: '',
          patientAddress: '',
          location: '',
          prescriptionImages: [],
          notes: '',
        });
        setPreviews([]);
        setSelectedPharmacyIds([]);
        setActivePharmacyId(null);
        setAvailablePharmacies([]);
        loadDefaultPharmacies();
        setTimeout(() => {
          setMessage('');
        }, 5000);
      } else {
        setMessage(result.error || 'Failed to upload prescription');
      }
    } catch (error) {
      setMessage('Error uploading prescription. Please try again.');
    } finally {
      setIsLoading(false);
      setUploadProgress(0);
    }
  };

  return (
    <div className="min-h-screen px-4 py-12 sm:px-6 lg:px-8">
      <div className="mx-auto max-w-4xl">
        <div className="glass-panel overflow-hidden rounded-3xl border border-white/10 shadow-
2xl">
          <div className="bg-slate-900 px-6 py-6">
            <h1 className="text-3xl font-bold text-white text-center">Upload Prescription</h1>
            <p className="text-slate-300 text-center mt-2">Securely upload your prescription
for pharmacy fulfillment</p>
          </div>

          <div className="p-8">
            {!user && (
              <div className="mb-6 rounded-lg border border-cyan-300/30 bg-cyan-300/10 p-4">
                <div className="flex items-center justify-between">
                  <div>
                    <h3 className="text-sm font-medium text-cyan-100">Already have an account?
</h3>
                    <p className="mt-1 text-sm text-cyan-200">Login to auto-fill your
information and track your prescriptions.</p>
                  </div>
                  <Link href="/login" className="rounded-lg bg-cyan-300 px-4 py-2 text-sm
font-medium text-slate-900 transition duration-300 hover:bg-cyan-200">
                    Login
                  </Link>
                </div>
              </div>
            )}

            <form onSubmit={handleSubmit} className="space-y-8">
              {/* Patient Information Section */}
              <div className="rounded-lg border border-white/10 bg-slate-900/60 p-6">
                <h2 className="mb-4 flex items-center text-xl font-semibold text-slate-100">
                  <svg className="h-6 w-6 mr-2 text-blue-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M16
7a4 4 0 11-8 0 4 4 0 018 0zM12 14a7 7 0 00-7 7h14a7 7 0 00-7-7z" />
                  </svg>
                  Patient Information
                </h2>

                <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
                  <div>
                    <label className="mb-2 block text-sm font-medium text-slate-300">Full Name
*</label>
                    <input
                      type="text"
                      name="patientName"
```

```
                      value={formData.patientName}
                      onChange={handleChange}
                      required
                      className="w-full rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                      placeholder="Enter your full name"
                    />
                  </div>

                  <div>
                    <label className="mb-2 block text-sm font-medium text-slate-300">Email
Address *</label>
                    <input
                      type="email"
                      name="patientEmail"
                      value={formData.patientEmail}
                      onChange={handleChange}
                      required
                      className="w-full rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                      placeholder="Enter your email"
                    />
                  </div>

                  <div>
                    <label className="mb-2 block text-sm font-medium text-slate-300">Phone
Number</label>
                    <input
                      type="tel"
                      name="patientPhone"
                      value={formData.patientPhone}
                      onChange={handleChange}
                      className="w-full rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                      placeholder="Enter your phone number"
                    />
                  </div>

                  <div>
                    <label className="mb-2 block text-sm font-medium text-slate-300">Location
(City/ZIP) *</label>
                    <div className="flex space-x-2">
                      <input
                        type="text"
                        name="location"
                        value={formData.location}
                        onChange={handleLocationChange}
                        required
                        className="flex-1 rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                        placeholder="Enter your city or ZIP code"
                      />
                      <button
                        type="button"
                        onClick={() => formData.location.trim() && searchPharmacies({
location: formData.location })}
                        disabled={!formData.location.trim() || isSearchingPharmacies}
                        className="rounded-lg bg-blue-600 px-4 py-3 text-white hover:bg-blue-
700 disabled:cursor-not-allowed disabled:bg-slate-500 disabled:text-slate-200"
                        aria-label="Search pharmacies"
                      >
                        <svg
                          className={`h-5 w-5 ${isSearchingPharmacies ? 'animate-pulse' :
''}`}

                          fill="none"
                          stroke="currentColor"
                          viewBox="0 0 24 24"
                        >
                          <path
                            strokeLinecap="round"
                            strokeLinejoin="round"
                            strokeWidth={2}
                            d="m21 21-4.35-4.35m0 0A7.5 7.5 0 1 0 6.04 6.04a7.5 7.5 0 0 0
10.61 10.61Z"
                          />
                        </svg>
                      </button>
                      <button
                        type="button"
                        onClick={getCurrentLocationAndNearestPharmacies}
                        disabled={isSearchingPharmacies}
                        className="rounded-lg bg-emerald-600 px-4 py-3 text-white hover:bg-
emerald-700 disabled:cursor-not-allowed disabled:bg-slate-500 disabled:text-slate-200"
```

```
                    >
                      Nearest
                    </button>
                  </div>
                </div>
              </div>

              <div className="mt-6">
                <label className="mb-2 block text-sm font-medium text-slate-
300">Address</label>
                <textarea
                  name="patientAddress"
                  value={formData.patientAddress}
                  onChange={handleChange}
                  className="w-full rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                  rows={3}
                  placeholder="Enter your complete address"
                />
              </div>
            </div>

            {/* Prescription Upload Section */}
            <div className="rounded-lg border border-white/10 bg-slate-900/60 p-6">
              <h2 className="mb-4 flex items-center text-xl font-semibold text-slate-100">
                <svg className="h-6 w-6 mr-2 text-green-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9
12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0
01.293.707V19a2 2 0 01-2 2z" />
                </svg>
                Prescription Images
              </h2>

              {/* Drag and Drop Area */}
              <div
                className={`border-2 border-dashed rounded-lg p-8 text-center transition-
colors ${
                  dragActive
                    ? 'border-cyan-300 bg-cyan-300/10'
                    : 'border-slate-600 hover:border-slate-400'
                }`}
                onDragEnter={handleDrag}
                onDragLeave={handleDrag}
                onDragOver={handleDrag}
                onDrop={handleDrop}
                onClick={() => fileInputRef.current?.click()}
              >
                <svg className="mx-auto mb-4 h-12 w-12 text-slate-400" stroke="currentColor"
fill="none" viewBox="0 0 48 48">
                  <path d="M28 8H12a4 4 0 00-4 4v20m32-12v8m0 0v8a4 4 0 01-4 4H12a4 4 0 01-
4-4v-4m32-4l-3.172-3.172a4 4 0 00-5.656 0L28 28M8 32l9.172-9.172a4 4 0 015.656 0L28 28m0 0l4
4m4-24h8m-4-4v8m-12 4h.02" strokeWidth={2} strokeLinecap="round" strokeLinejoin="round" />
                </svg>
                <p className="mb-2 text-lg font-medium text-slate-100">
                  {dragActive ? 'Drop your prescription images here' : 'Click to upload or
drag and drop'}
                </p>
                <p className="text-sm text-slate-400">
                  PNG, JPG, GIF, WebP up to 5MB each
                </p>
                <input
                  ref={fileInputRef}
                  type="file"
                  multiple
                  accept="image/*"
                  onChange={(e) => handleFileChange(e.target.files)}
                  className="hidden"
                />
              </div>

              {/* Image Previews */}
              {previews.length > 0 && (
                <div className="mt-6">
                  <h3 className="mb-4 text-lg font-medium text-slate-100">Uploaded Images
({previews.length})</h3>
                  <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-
cols-4 gap-3 sm:gap-4">
                    {previews.map((preview, index) => (
                      <div key={index} className="relative group">
                        <img
                          src={preview}
                          alt={`Prescription ${index + 1}`}
                          className="w-full h-32 sm:h-36 object-cover rounded-lg border"
                        />
                        <button
```

```
                              type="button"
                              onClick={() => removeImage(index)}
                              className="absolute top-2 right-2 bg-red-500 text-white rounded-
full p-1 opacity-0 group-hover:opacity-100 transition-opacity hover:bg-red-600"
                            >
                              <svg className="h-4 w-4" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth=
{2} d="M6 18L18 6M6 6l12 12" />
                              </svg>
                            </button>
                          </div>
                        ))}
                      </div>
                    </div>
                  )}

                  {/* Additional Notes */}
                  <div className="mt-6">
                    <label className="mb-2 block text-sm font-medium text-slate-300">Additional
Notes (Optional)</label>
                    <textarea
                      name="notes"
                      value={formData.notes}
                      onChange={handleChange}
                      className="w-full rounded-lg border border-slate-700 bg-slate-950 p-3
text-slate-100 placeholder:text-slate-500 focus:border-cyan-300 focus:outline-none focus:ring-
2 focus:ring-cyan-300/30"
                      rows={3}
                      placeholder="Any special instructions or notes for the pharmacy"
                    />
                  </div>
                </div>

                {/* Pharmacy Selection Section */}
                <div className="rounded-lg border border-white/10 bg-slate-900/60 p-6">
                  <h2 className="mb-4 flex items-center text-xl font-semibold text-slate-100">
                    <svg className="h-6 w-6 mr-2 text-purple-600" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M19
21V5a2 2 0 00-2-2H7a2 2 0 00-2 2v16m14 0h2m-2 0h-5m-9 0H3m2 0h5M9 7h1m-1 4h1m4-4h1m-1 4h1m-5
10v-5a1 1 0 011-1h2a1 1 0 011 1v5m-4 0h4" />
                    </svg>
                    Select Pharmacies
                  </h2>

                  {formData.location && (
                    <div className="mb-4">
                      <p className="text-sm text-slate-300">
                        {isSearchingPharmacies ? (
                          <>🔍 Searching pharmacies...</>
                        ) : formData.location.trim().length < 3 ? (
                          <>Type at least 3 characters to search pharmacies</>
                        ) : availablePharmacies.length > 0 ? (
                          <>
                            Found {availablePharmacies.length} pharmacies near &quot;
{formData.location}&quot;
                            {userCoordinates ? ' (nearest first)' : ''}
                          </>
                        ) : (
                          <>No pharmacies found near &quot;{formData.location}&quot;. Try a
different location.</>
                        )}
                      </p>
                    </div>
                  )}

                  {!formData.location && (
                    <div className="mb-4">
                      <p className="text-sm text-slate-300">
                        Showing registered pharmacies from our platform. You can still type a
city/ZIP or use Nearest.
                      </p>
                    </div>
                  )}

                  {availablePharmacies.length > 0 && (
                    <div>
                      <div className="mb-3 flex justify-end">
                        <button
                          type="button"
                          onClick={toggleSelectAllPharmacies}
                          className="text-sm font-medium text-cyan-200 hover:text-cyan-100"
                        >
                          {availablePharmacies.every((pharmacy) =>
selectedPharmacyIds.includes(pharmacy._id))
                            ? 'Clear all'
```

```
                          : 'Select all'}
                      </button>
                    </div>
                    <div className="grid grid-cols-1 gap-4 lg:grid-cols-2">
                      <div className="space-y-3 max-h-96 overflow-y-auto pr-1">
                        {availablePharmacies.map((pharmacy) => (
                          <div
                            key={pharmacy._id}
                            className={`border rounded-lg p-4 cursor-pointer transition-colors
${

                              selectedPharmacyIds.includes(pharmacy._id)
                                ? 'border-cyan-300 bg-cyan-300/10'
                                : activePharmacyId === pharmacy._id
                                  ? 'border-indigo-400 bg-indigo-300/10'
                                  : 'border-slate-700 hover:border-slate-500'
                            }`}
                            onClick={() => togglePharmacySelection(pharmacy._id)}
                          >
                            <div className="flex items-start justify-between">
                              <div className="flex-1">
                                <div className="flex items-center">
                                  <input
                                    type="checkbox"
                                    checked={selectedPharmacyIds.includes(pharmacy._id)}
                                    onClick={(e) => e.stopPropagation()}
                                    onChange={() => togglePharmacySelection(pharmacy._id)}
                                    className="mr-3 h-4 w-4 rounded border-slate-600 bg-slate-
950 text-cyan-300 focus:ring-cyan-300"
                                  />
                                  <h3 className="font-medium text-slate-100">{pharmacy.name}
</h3>
                                </div>
                                <p className="mt-1 text-sm text-slate-300">{pharmacy.address}
</p>
                                {typeof pharmacy.distanceKm === 'number' && (
                                  <p className="text-xs text-emerald-700 mt-1">
                                    Approx. {pharmacy.distanceKm.toFixed(1)} km away
                                  </p>
                                )}
                                <div className="flex items-center mt-2 space-x-2">
                                {pharmacy.isUsingService && (
                                  <span className="inline-flex items-center rounded-full bg-
emerald-300/20 px-2 py-1 text-xs font-medium text-emerald-100">
                                    ⭐ Service Partner
                                  </span>
                                )}
                                {!pharmacy.isUsingService && !pharmacy.subscriptionType && (
                                  <span className="inline-flex items-center rounded-full bg-
slate-700/70 px-2 py-1 text-xs font-medium text-slate-200">
                                    Registered Pharmacy
                                  </span>
                                )}
                                {pharmacy.supportsPrescriptionUpload && (
                                  <span className="inline-flex items-center rounded-full bg-
cyan-300/20 px-2 py-1 text-xs font-medium text-cyan-100">
                                    📄 Upload Support
                                  </span>
                                )}
                                  <div className="flex items-center">
                                    <span className="text-yellow-400 text-sm">⭐</span>
                                    <span className="ml-1 text-xs text-slate-300">
                                      {pharmacy.rating} ({pharmacy.reviewCount} reviews)
                                    </span>
                                  </div>
                                </div>
                              </div>
                            </div>
                          </div>
                        ))}
                      </div>
                      <div className="min-h-[20rem] rounded-lg border border-white/10 bg-
slate-950/80 p-2">
                        <PharmacyMap
                          pharmacies={availablePharmacies}
                          userLocation={userCoordinates || undefined}
                          selectedPharmacyId={activePharmacyId || selectedPharmacyIds[0] ||
null}

                          onPharmacySelect={handleMapPharmacySelect}
                          showInfoCard={false}
                        />
                      </div>
                    </div>
                  </div>
                )}

                {(formData.location || userCoordinates) && !isSearchingPharmacies &&
availablePharmacies.length === 0 && (
```

```
                  <div className="py-8 text-center text-slate-400">
                    <svg className="mx-auto mb-4 h-12 w-12 text-slate-500" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M19 21V5a2 2 0 00-2-2H7a2 2 0 00-2 2v16m14 0h2m-2 0h-5m-9 0H3m2 0h5M9 7h1m-1 4h1m4-4h1m-1
4h1m-5 10v-5a1 1 0 011-1h2a1 1 0 011 1v5m-4 0h4" />
                    </svg>
                    <p>No pharmacies found in this area.</p>
                    <p className="text-sm mt-1">Try a different location, click Nearest again,
or contact support.</p>
                  </div>
                )}

                {!formData.location && availablePharmacies.length === 0 && (
                  <div className="py-8 text-center text-slate-400">
                    <p>No registered pharmacies available right now.</p>
                  </div>
                )}

                {selectedPharmacyIds.length > 0 && (
                  <div className="mt-4 rounded-lg border border-cyan-300/30 bg-cyan-300/10 p-
3">
                    <p className="text-sm text-cyan-100">
                      <strong>{selectedPharmacyIds.length} pharmacy(ies) selected</strong> -
Your prescription will be sent to these pharmacies.
                    </p>
                    <div className="mt-2 text-xs text-cyan-200">
                      Active details: {availablePharmacies.find((pharmacy) => pharmacy._id ===
(activePharmacyId || selectedPharmacyIds[0]))?.name || 'Select a pharmacy from the list'}
                    </div>
                  </div>
                )}
              </div>

              {/* Progress Bar */}
              {isLoading && (
                <div className="rounded-lg border border-white/10 bg-slate-900/60 p-6">
                  <div className="flex items-center justify-between mb-2">
                    <span className="text-sm font-medium text-slate-300">Uploading...</span>
                    <span className="text-sm font-medium text-slate-300">{uploadProgress}%
</span>
                  </div>
                  <div className="h-2 w-full rounded-full bg-slate-700">
                    <div
                      className="h-2 rounded-full bg-cyan-300 transition-all duration-300"
                      style={{ width: `${uploadProgress}%` }}
                    ></div>
                  </div>
                </div>
              )}

              {/* Submit Button */}
              <button
                type="submit"
                disabled={isLoading || formData.prescriptionImages.length === 0 ||
selectedPharmacyIds.length === 0}
                className="flex w-full items-center justify-center rounded-lg bg-gradient-to-r
from-cyan-400 to-emerald-400 p-4 text-lg font-semibold text-slate-900 transition duration-300
hover:from-cyan-300 hover:to-emerald-300 disabled:cursor-not-allowed disabled:from-slate-500
disabled:to-slate-500 disabled:text-slate-200"
              >
                {isLoading ? (
                  <>
                    <svg className="animate-spin -ml-1 mr-3 h-5 w-5 text-white" fill="none"
viewBox="0 0 24 24">
                      <circle className="opacity-25" cx="12" cy="12" r="10"
stroke="currentColor" strokeWidth="4"></circle>
                      <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-
8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3 7.938l3-
2.647z"></path>
                    </svg>
                    Uploading...
                  </>
                ) : (
                  <>
                    <svg className="mr-2 h-5 w-5" fill="none" stroke="currentColor" viewBox="0
0 24 24">
                      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M7
16a4 4 0 01-.88-7.903A5 5 0 1115.9 6L16 6a5 5 0 011 9.9M15 13l-3-3m0 0l-3 3m3-3v12" />
                    </svg>
                    Upload Prescription
                  </>
                )}
              </button>
            </form>

            {/* Success/Error Messages */}
```

```
          {message && (
            <div className={`mt-6 p-4 rounded-lg ${
              message.includes('successfully')
                ? 'border border-emerald-300/30 bg-emerald-300/10 text-emerald-100'
                : 'border border-rose-300/30 bg-rose-300/10 text-rose-100'
            }`}>
              <div className="flex items-center">
                {message.includes('successfully') ? (
                  <svg className="h-5 w-5 mr-2" fill="none" stroke="currentColor" viewBox="0
0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9
12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" />
                  </svg>
                ) : (
                  <svg className="h-5 w-5 mr-2" fill="none" stroke="currentColor" viewBox="0
0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 8v4m0 4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
                  </svg>
                )}
                {message}
              </div>
            </div>
          )}

          {/* Help Text */}
          <div className="mt-8 rounded-lg border border-cyan-300/30 bg-cyan-300/10 p-4">
            <h3 className="mb-2 text-sm font-medium text-cyan-100">Need Help?</h3>
            <ul className="space-y-1 text-sm text-cyan-200">
              <li>• Ensure your prescription is clearly visible and readable</li>
              <li>• Include all pages of your prescription</li>
              <li>• Make sure the image is well-lit and in focus</li>
              <li>• Contact support if you have any questions</li>
            </ul>
          </div>
        </div>
      </div>
    </div>
  </div>
  );
}
```

[33] components/Footer.tsx FILE

```
import Link from "next/link";

const quickLinks = [
  { href: "/", label: "Home" },
  { href: "/upload", label: "Upload" },
  { href: "/pharmacies", label: "Pharmacies" },
  { href: "/subscribe", label: "Plans" },
];

export default function Footer() {
  const year = new Date().getFullYear();

  return (
    <footer className="border-t border-white/10 bg-slate-950/95">
      <div className="mx-auto grid max-w-7xl gap-8 px-4 py-10 sm:px-6 lg:grid-cols-3 lg:px-8">
        <div>
          <h3 className="text-lg font-semibold text-white">E-Pharmacy</h3>
          <p className="mt-2 max-w-sm text-sm text-slate-300">
            Connecting patients and pharmacies for faster prescription fulfillment.
          </p>
        </div>

        <div>
          <h4 className="text-sm font-semibold uppercase tracking-[0.12em] text-slate-200">
            Quick Links
          </h4>
          <ul className="mt-3 space-y-2 text-sm text-slate-300">
            {quickLinks.map((item) => (
              <li key={item.href}>
                <Link href={item.href} className="transition hover:text-white">
                  {item.label}
                </Link>
              </li>
            ))}
          </ul>
        </div>

        <div>
          <h4 className="text-sm font-semibold uppercase tracking-[0.12em] text-slate-200">
```

```
            Contact
          </h4>
          <p className="mt-3 text-sm text-slate-300">
            support@epharmacy.com
          </p>
          <p className="mt-1 text-sm text-slate-300">Mon to Sat, 9:00 AM - 7:00 PM</p>
        </div>
      </div>

      <div className="border-t border-white/10 px-4 py-4 text-center text-xs text-slate-400
sm:px-6 lg:px-8">
        Copyright {year} E-Pharmacy. All rights reserved.
      </div>
    </footer>
  );
}
```

[34] components/Navbar.tsx FILE

```
'use client';

import Link from 'next/link';
import { usePathname, useRouter } from 'next/navigation';
import { useEffect, useMemo, useRef, useState } from 'react';

interface User {
  id: string;
  name: string;
  email: string;
  role: string;
}

interface NavItem {
  href: string;
  label: string;
}

interface PharmacistNotificationPreview {
  prescription: {
    id: string;
    patientName: string;
    location: string;
    createdAt: string;
    status?: string;
    selectedPharmacyIds?: string[];
  };
}

interface PrescriptionApiItem {
  _id: string;
  patientName: string;
  location: string;
  createdAt: string;
  status?: string;
  pharmacyStatuses?: Array<{
    pharmacyId?: string;
    status?: string;
    assignedAt?: string | null;
    completedAt?: string | null;
  }>;
}

interface PatientNotificationPreview {
  id: string;
  title: string;
  message: string;
  createdAt: string;
  actionType?: 'confirm_fulfillment';
  prescriptionId?: string;
  pharmacyId?: string;
}

interface PatientHistoryItem {
  _id: string;
  createdAt: string;
  pharmacyDetails?: Array<{
    pharmacyId?: string;
    name?: string;
    status?: string;
    availabilityResponse?: 'not_available' | 'same_medicine_available' |
'same_salt_different_company' | null;
    pharmacistMessage?: string;
    assignedAt?: string | null;
```

```
    completedAt?: string | null;
  }>;
}

interface NavNotificationItem {
  id: string;
  title: string;
  subtitle: string;
  createdAt: string;
  actionType?: 'confirm_fulfillment';
  prescriptionId?: string;
  pharmacyId?: string;
}

interface ProfileMenuItem {
  href: string;
  label: string;
}

const STORAGE_NOTIFICATIONS_KEY = 'pharmacy_notifications';
const STORAGE_UNREAD_KEY = 'pharmacy_notifications_unread';
const STORAGE_PATIENT_NOTIFICATIONS_KEY = 'patient_notifications';
const STORAGE_PATIENT_UNREAD_KEY = 'patient_notifications_unread';
const STORAGE_THEME_KEY = 'pharmacy_theme';
const MAX_NOTIFICATION_ITEMS = 100;
const getAvailabilityLabel = (
  value?: 'not_available' | 'same_medicine_available' | 'same_salt_different_company' | null
) => {
  switch (value) {
    case 'not_available':
      return 'Medicine not available';
    case 'same_medicine_available':
      return 'Same medicine available';
    case 'same_salt_different_company':
      return 'Same salt, different company available';
    default:
      return '';
  }
};

export default function Navbar() {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);
  const [isNotificationOpen, setIsNotificationOpen] = useState(false);
  const [unreadCount, setUnreadCount] = useState(0);
  const [notificationPreview, setNotificationPreview] = useState<NavNotificationItem[]>([]);
  const [confirmingNotificationId, setConfirmingNotificationId] = useState<string | null>
(null);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [isDarkMode, setIsDarkMode] = useState(true);
  const [hasInitializedTheme, setHasInitializedTheme] = useState(false);
  const router = useRouter();
  const pathname = usePathname();
  const navRef = useRef<HTMLElement>(null);
  const profileRef = useRef<HTMLDivElement>(null);
  const notificationRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    checkAuthStatus();
  }, []);

  useEffect(() => {
    const syncTheme = () => {
      try {
        const savedTheme = localStorage.getItem(STORAGE_THEME_KEY);
        if (savedTheme === 'dark' || savedTheme === 'light') {
          setIsDarkMode(savedTheme === 'dark');
        }
      } catch {
        // Ignore storage/read issues and preserve current in-memory state.
      } finally {
        setHasInitializedTheme(true);
      }
    };

    syncTheme();
    window.addEventListener('theme-updated', syncTheme);
    window.addEventListener('storage', syncTheme);
    return () => {
      window.removeEventListener('theme-updated', syncTheme);
      window.removeEventListener('storage', syncTheme);
    };
  }, []);

  useEffect(() => {
    if (!hasInitializedTheme) return;
```

```
    const root = document.documentElement;
    const nextTheme = isDarkMode ? 'dark' : 'light';
    root.setAttribute('data-theme', nextTheme);
    localStorage.setItem(STORAGE_THEME_KEY, nextTheme);
    window.dispatchEvent(new Event('theme-updated'));
  }, [hasInitializedTheme, isDarkMode]);

  useEffect(() => {
    if (!isMobileMenuOpen) {
      return;
    }

    const handleClickOutside = (event: MouseEvent) => {
      const target = event.target as Node;
      if (navRef.current && !navRef.current.contains(target)) {
        setIsMobileMenuOpen(false);
      }
    };

    document.addEventListener('mousedown', handleClickOutside);
    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, [isMobileMenuOpen]);

  useEffect(() => {
    if (!isProfileOpen) {
      return;
    }

    const handleClickOutsideProfile = (event: MouseEvent) => {
      const target = event.target as Node;
      if (profileRef.current && !profileRef.current.contains(target)) {
        setIsProfileOpen(false);
      }
    };

    document.addEventListener('mousedown', handleClickOutsideProfile);
    return () => document.removeEventListener('mousedown', handleClickOutsideProfile);
  }, [isProfileOpen]);

  useEffect(() => {
    if (!isNotificationOpen) {
      return;
    }

    const handleClickOutsideNotification = (event: MouseEvent) => {
      const target = event.target as Node;
      if (notificationRef.current && !notificationRef.current.contains(target)) {
        setIsNotificationOpen(false);
      }
    };

    document.addEventListener('mousedown', handleClickOutsideNotification);
    return () => document.removeEventListener('mousedown', handleClickOutsideNotification);
  }, [isNotificationOpen]);

  useEffect(() => {
    const loadNotificationState = () => {
      try {
        if (user?.role === 'patient') {
          const unread = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) || '0');
          const raw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
          const parsed = raw ? (JSON.parse(raw) as PatientNotificationPreview[]) : [];
          const items = Array.isArray(parsed)
            ? parsed.slice(0, 5).map((item) => ({
                id: item.id || `${item.title}-${item.createdAt}`,
                title: item.title || 'Notification',
                subtitle: item.message || '',
                createdAt: item.createdAt || '',
                actionType: item.actionType,
                prescriptionId: item.prescriptionId,
                pharmacyId: item.pharmacyId,
              }))
            : [];
          setUnreadCount(Number.isNaN(unread) ? 0 : unread);
          setNotificationPreview(items);
          return;
        }

        const unread = Number(localStorage.getItem(STORAGE_UNREAD_KEY) || '0');
        const raw = localStorage.getItem(STORAGE_NOTIFICATIONS_KEY);
        const parsed = raw ? (JSON.parse(raw) as PharmacistNotificationPreview[]) : [];
        const items = Array.isArray(parsed)
          ? parsed.slice(0, 5).map((item) => ({
              id: `${item.prescription.id}-${item.prescription.createdAt}`,
              title: item.prescription.patientName,
              subtitle: `${(item.prescription.status || 'pending').replace('_', ' ')} •
```

```
${item.prescription.location}`,
              createdAt: item.prescription.createdAt,
            }))
          : [];
        setUnreadCount(Number.isNaN(unread) ? 0 : unread);
        setNotificationPreview(items);
      } catch (error) {
        console.error('Failed to read notification badge state:', error);
      }
    };

    loadNotificationState();
    window.addEventListener('notification-updated', loadNotificationState);
    window.addEventListener('storage', loadNotificationState);

    return () => {
      window.removeEventListener('notification-updated', loadNotificationState);
      window.removeEventListener('storage', loadNotificationState);
    };
  }, [user?.role]);

  useEffect(() => {
    if (user?.role !== 'pharmacist') return;
    if (pathname === '/dashboard' || pathname === '/notifications') return;

    let eventSource: EventSource | null = null;
    let reconnectTimer: ReturnType<typeof setTimeout> | null = null;

    const connect = () => {
      eventSource = new EventSource('/api/notifications/stream');

      eventSource.onmessage = (event) => {
        try {
          const data = JSON.parse(event.data) as PharmacistNotificationPreview & { type?:
string };
          if (data.type !== 'new_prescription' || !data?.prescription?.id) return;

          const raw = localStorage.getItem(STORAGE_NOTIFICATIONS_KEY);
          const existing = raw ? (JSON.parse(raw) as PharmacistNotificationPreview[]) : [];
          const exists = existing.some(
            (item) =>
              item.prescription.id === data.prescription.id &&
              item.prescription.createdAt === data.prescription.createdAt
          );

          if (!exists) {
            const merged = [data, ...existing].slice(0, MAX_NOTIFICATION_ITEMS);
            localStorage.setItem(STORAGE_NOTIFICATIONS_KEY, JSON.stringify(merged));
            const unreadCount = Number(localStorage.getItem(STORAGE_UNREAD_KEY) || '0');
            localStorage.setItem(STORAGE_UNREAD_KEY, String(unreadCount + 1));
            window.dispatchEvent(new Event('notification-updated'));
          }
        } catch (error) {
          console.error('Failed to process navbar notification stream event:', error);
        }
      };

      eventSource.onerror = () => {
        if (eventSource) {
          eventSource.close();
          eventSource = null;
        }
        reconnectTimer = setTimeout(connect, 5000);
      };
    };

    connect();

    return () => {
      if (reconnectTimer) clearTimeout(reconnectTimer);
      if (eventSource) eventSource.close();
    };
  }, [pathname, user?.role]);

  const checkAuthStatus = async () => {
    try {
      const response = await fetch('/api/auth/me?optional=1');
      if (response.ok) {
        const userData = await response.json();
        setUser(userData.user);
      }
    } catch {
      // Guest state is valid.
    } finally {
      setIsLoading(false);
    }
  };
```

```
  const handleLogout = async () => {
    try {
      await fetch('/api/auth/logout', { method: 'POST' });
      setUser(null);
      setIsMobileMenuOpen(false);
      setIsProfileOpen(false);
      router.push('/');
    } catch (error) {
      console.error('Logout error:', error);
    }
  };

  const navItems = useMemo(() => {
    const common: NavItem[] = [{ href: '/', label: 'Home' }];

    if (user?.role === 'patient') {
      return [
        ...common,
        { href: '/pharmacies', label: 'Nearby Pharmacies' },
        { href: '/upload', label: 'Upload' },
        { href: '/history', label: 'History' },
      ];
    }

    if (user?.role === 'pharmacist') {
      return [
        ...common,
        { href: '/pharmacies', label: 'Nearby Pharmacies' },
        { href: '/subscribe', label: 'Plans' },
        { href: '/dashboard', label: 'Dashboard' },
      ];
    }

    return [
      ...common,
      { href: '/pharmacies', label: 'Nearby Pharmacies' },
      { href: '/subscribe', label: 'Plans' },
    ];
  }, [user?.role]);

  const profileMenuItems = useMemo<ProfileMenuItem[]>(
    () => [
      { href: '/profile', label: 'Profile Details' },
      { href: '/settings', label: 'Settings' },
    ],
    []
  );

  const navLinkClass = (href: string) =>
    `rounded-full px-4 py-2 text-sm font-semibold transition ${
      pathname === href
        ? 'bg-white text-slate-900'
        : 'text-slate-200 hover:bg-white/10 hover:text-white'
    }`;

  const mobileNavLinkClass = (href: string) =>
    `block rounded-xl px-4 py-3 text-sm font-medium transition ${
      pathname === href
        ? 'bg-cyan-300 text-slate-900'
        : 'text-slate-200 hover:bg-white/10 hover:text-white'
    }`;

  const handleNotificationToggle = () => {
    const next = !isNotificationOpen;
    setIsNotificationOpen(next);
    if (next) {
      const unreadKey = user?.role === 'patient' ? STORAGE_PATIENT_UNREAD_KEY :
STORAGE_UNREAD_KEY;
      localStorage.setItem(unreadKey, '0');
      setUnreadCount(0);
      window.dispatchEvent(new Event('notification-updated'));
    }
  };

  const handleConfirmFulfillmentFromNotification = async (item: NavNotificationItem) => {
    if (user?.role !== 'patient') return;
    if (item.actionType !== 'confirm_fulfillment' || !item.prescriptionId || !item.pharmacyId)
return;

    try {
      setConfirmingNotificationId(item.id);
      const response = await fetch(`/api/prescriptions/${item.prescriptionId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ status: 'confirm_fulfillment', pharmacyId: item.pharmacyId }),
      });
```

```
      const payload = await response.json().catch(() => ({}));
      if (!response.ok) {
        throw new Error(payload?.error || 'Failed to confirm fulfillment');
      }

      const nowIso = new Date().toISOString();
      const confirmationNotice: PatientNotificationPreview = {
        id: `${item.prescriptionId}-${item.pharmacyId}-fulfilled`,
        title: 'Fulfillment confirmed',
        message: 'Prescription status updated to fulfilled.',
        createdAt: nowIso,
      };
      const raw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
      const existing = raw ? (JSON.parse(raw) as PatientNotificationPreview[]) : [];
      const filtered = Array.isArray(existing)
        ? existing.filter(
            (entry) =>
              !(
                entry.actionType === 'confirm_fulfillment' &&
                entry.prescriptionId === item.prescriptionId &&
                entry.pharmacyId === item.pharmacyId
              ) && entry.id !== item.id
          )
        : [];
      const merged = [confirmationNotice, ...filtered].slice(0, MAX_NOTIFICATION_ITEMS);
      localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(merged));

      const unreadCount = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) || '0');
      localStorage.setItem(STORAGE_PATIENT_UNREAD_KEY, String(unreadCount + 1));
      window.dispatchEvent(new Event('notification-updated'));
    } catch (error) {
      const message = error instanceof Error ? error.message : 'Unable to confirm fulfillment
right now.';
      if (message.toLowerCase().includes('no pending fulfillment confirmation request')) {
        // Request already resolved earlier; replace action item with a non-action status
item.
        const resolvedNotice: PatientNotificationPreview = {
          id: `${item.prescriptionId}-${item.pharmacyId}-fulfilled`,
          title: 'Fulfillment confirmed',
          message: 'Prescription status is already fulfilled.',
          createdAt: new Date().toISOString(),
        };
        const raw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
        const existing = raw ? (JSON.parse(raw) as PatientNotificationPreview[]) : [];
        const filtered = Array.isArray(existing)
          ? existing.filter(
              (entry) =>
                !(
                  entry.actionType === 'confirm_fulfillment' &&
                  entry.prescriptionId === item.prescriptionId &&
                  entry.pharmacyId === item.pharmacyId
                ) && entry.id !== item.id
            )
          : [];
        const merged = [resolvedNotice, ...filtered].slice(0, MAX_NOTIFICATION_ITEMS);
        localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(merged));
        window.dispatchEvent(new Event('notification-updated'));
        return;
      }

      const failNotice: PatientNotificationPreview = {
        id: `patient-confirm-failed-${Date.now()}`,
        title: 'Confirmation failed',
        message,
        createdAt: new Date().toISOString(),
      };
      const raw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
      const existing = raw ? (JSON.parse(raw) as PatientNotificationPreview[]) : [];
      const merged = [failNotice, ...(Array.isArray(existing) ? existing : [])].slice(0,
MAX_NOTIFICATION_ITEMS);
      localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(merged));
      const unreadCount = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) || '0');
      localStorage.setItem(STORAGE_PATIENT_UNREAD_KEY, String(unreadCount + 1));
      window.dispatchEvent(new Event('notification-updated'));
    } finally {
      setConfirmingNotificationId(null);
    }
  };

  useEffect(() => {
    if (user?.role !== 'pharmacist') return;

    const hydrateFromRecentActivity = async () => {
      try {
        const raw = localStorage.getItem(STORAGE_NOTIFICATIONS_KEY);
        const existing = raw ? (JSON.parse(raw) as PharmacistNotificationPreview[]) : [];
        if (Array.isArray(existing) && existing.length > 0) return;
```

```
        const meRes = await fetch('/api/auth/me');
        if (!meRes.ok) return;
        const mePayload = await meRes.json();
        const location = typeof mePayload?.user?.location === 'string' ?
mePayload.user.location : '';
        const pharmacyId = typeof mePayload?.user?.pharmacyId === 'string' ?
mePayload.user.pharmacyId : '';
        if (!location || !pharmacyId) return;

        const rxRes = await fetch(`/api/prescriptions?
location=${encodeURIComponent(location)}&status=all`);
        if (!rxRes.ok) return;
        const rxPayload = (await rxRes.json()) as PrescriptionApiItem[];
        if (!Array.isArray(rxPayload) || rxPayload.length === 0) return;

        const own = rxPayload
          .filter((item) =>
            item.pharmacyStatuses?.some((entry) => String(entry?.pharmacyId || '') ===
pharmacyId)
          )
          .sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime())
          .slice(0, MAX_NOTIFICATION_ITEMS)
          .map((item) => ({
            prescription: {
              id: item._id,
              patientName: item.patientName,
              location: item.location,
              createdAt: item.createdAt,
              status:
                item.pharmacyStatuses?.find((entry) => String(entry?.pharmacyId || '') ===
pharmacyId)?.status ||
                item.status ||
                'pending',
            },
          }));

        if (own.length > 0) {
          localStorage.setItem(STORAGE_NOTIFICATIONS_KEY, JSON.stringify(own));
          window.dispatchEvent(new Event('notification-updated'));
        }
      } catch {
        // Keep navbar functional if fallback hydration fails.
      }
    };

    hydrateFromRecentActivity();
  }, [user?.role]);

  useEffect(() => {
    if (user?.role !== 'patient') return;

    const hydratePatientNotificationsFromHistory = async () => {
      try {
        const response = await fetch('/api/patient/history');
        if (!response.ok) return;

        const prescriptions = (await response.json()) as PatientHistoryItem[];
        if (!Array.isArray(prescriptions) || prescriptions.length === 0) return;

        const mapped: PatientNotificationPreview[] = [];
        for (const prescription of prescriptions) {
          const pharmacyDetails = Array.isArray(prescription.pharmacyDetails) ?
prescription.pharmacyDetails : [];
          for (const detail of pharmacyDetails) {
            const normalizedStatus = typeof detail?.status === 'string' ? detail.status :
'pending';
            if (!['accepted', 'fulfillment_requested', 'fulfilled',
'rejected'].includes(normalizedStatus)) continue;

            const statusLabel = normalizedStatus.replace('_', ' ');
            const timeSource = detail?.completedAt || detail?.assignedAt ||
prescription.createdAt || '';
            const isFulfillmentRequest = normalizedStatus === 'fulfillment_requested';
            const availabilityText = getAvailabilityLabel(detail?.availabilityResponse);
            const customMessage = (detail?.pharmacistMessage || '').trim();
            const confirmationMessage = [
              `${detail?.name || 'Pharmacy'} requested your fulfillment confirmation.`,
              availabilityText ? `Availability: ${availabilityText}.` : '',
              customMessage ? `Message: ${customMessage}` : '',
            ]
              .filter(Boolean)
              .join(' ');
            mapped.push({
              id: `${prescription._id}-${detail?.pharmacyId ||
'unknown'}-${normalizedStatus}`,
              title: `Prescription ${statusLabel}`,
```

```
            message: isFulfillmentRequest
              ? confirmationMessage
              : [
                  `${detail?.name || 'Pharmacy'} updated your prescription status.`,
                  availabilityText ? `Availability: ${availabilityText}.` : '',
                  customMessage ? `Message: ${customMessage}` : '',
                ]
                  .filter(Boolean)
                  .join(' '),
            createdAt: timeSource || new Date().toISOString(),
            actionType: isFulfillmentRequest ? 'confirm_fulfillment' : undefined,
            prescriptionId: isFulfillmentRequest ? prescription._id : undefined,
            pharmacyId: isFulfillmentRequest ? detail?.pharmacyId : undefined,
          });
        }
      }

      if (mapped.length === 0) return;

      mapped.sort((a, b) => new Date(b.createdAt).getTime() - new
Date(a.createdAt).getTime());
      const clipped = mapped.slice(0, MAX_NOTIFICATION_ITEMS);
      const existingRaw = localStorage.getItem(STORAGE_PATIENT_NOTIFICATIONS_KEY);
      const existing = existingRaw ? (JSON.parse(existingRaw) as
PatientNotificationPreview[]) : [];
      const existingIds = new Set(Array.isArray(existing) ? existing.map((item) => item.id)
: []);
      const hasNew = clipped.some((item) => !existingIds.has(item.id));

      localStorage.setItem(STORAGE_PATIENT_NOTIFICATIONS_KEY, JSON.stringify(clipped));
      if (hasNew) {
        const unreadCount = Number(localStorage.getItem(STORAGE_PATIENT_UNREAD_KEY) || '0');
        localStorage.setItem(STORAGE_PATIENT_UNREAD_KEY, String(unreadCount + 1));
      }
      window.dispatchEvent(new Event('notification-updated'));
    } catch {
      // Ignore hydration failures and keep navbar usable.
    }
  };

  hydratePatientNotificationsFromHistory();
}, [user?.role]);

const formatActivityTime = (dateString: string) => {
  if (!dateString) return '';
  const date = new Date(dateString);
  if (Number.isNaN(date.getTime())) return '';
  return date.toLocaleString();
};

return (
  <header
    ref={navRef}
    className="sticky top-0 z-50 border-b border-white/10 bg-slate-950/85 backdrop-blur-xl"
  >
    <div className="mx-auto flex h-16 max-w-7xl items-center justify-between px-4 sm:px-6
lg:px-8">
      <Link href="/" className="flex items-center gap-3">
        <span className="flex h-9 w-9 items-center justify-center rounded-xl bg-cyan-300/20
text-cyan-100">
          ✚
        </span>
        <div>
          <p className="text-sm font-semibold text-white">E-Pharmacy</p>
          <p className="text-[11px] text-slate-300">Patient & Pharmacy Network</p>
        </div>
      </Link>

      <nav className="hidden items-center gap-2 md:flex">
        {navItems.map((item) => (
          <Link key={item.href} href={item.href} className={navLinkClass(item.href)}>
            {item.label}
          </Link>
        ))}
      </nav>

      <div className="hidden items-center gap-3 md:flex">
        <button
          type="button"
          onClick={() => setIsDarkMode((prev) => !prev)}
          className="inline-flex h-10 w-10 items-center justify-center rounded-full border
border-white/15 bg-white/5 text-slate-200 transition hover:bg-white/10"
          aria-label={isDarkMode ? 'Switch to light mode' : 'Switch to dark mode'}
          title={isDarkMode ? 'Switch to light mode' : 'Switch to dark mode'}
        >
          {isDarkMode ? (
            <svg className="h-5 w-5" viewBox="0 0 24 24" fill="none" stroke="currentColor"
```

```
aria-hidden="true">
                <path
                  strokeLinecap="round"
                  strokeLinejoin="round"
                  strokeWidth={2}
                  d="M12 3v2m0 14v2m9-9h-2M5 12H3m15.364 6.364-1.414-1.414M7.05 7.05 5.636
5.636m12.728 0L16.95 7.05M7.05 16.95l-1.414 1.414M12 8a4 4 0 100 8 4 4 0 000-8z"
                />
              </svg>
            ) : (
              <svg className="h-5 w-5" viewBox="0 0 24 24" fill="none" stroke="currentColor"
aria-hidden="true">
                <path
                  strokeLinecap="round"
                  strokeLinejoin="round"
                  strokeWidth={2}
                  d="M21 12.8A9 9 0 1111.2 3a7 7 0 009.8 9.8z"
                />
              </svg>
            )}
          </button>
          {isLoading ? (
            <span className="text-sm text-slate-300">Checking session...</span>
          ) : user ? (
            <>
              {(user.role === 'pharmacist' || user.role === 'patient') && (
                <div ref={notificationRef} className="relative">
                  <button
                    onClick={handleNotificationToggle}
                    className="relative rounded-full border border-white/15 bg-white/5 p-2
text-slate-100 hover:bg-white/10"
                    aria-label="Toggle notifications"
                  >
                    <svg className="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24
24">
                      <path
                        strokeLinecap="round"
                        strokeLinejoin="round"
                        strokeWidth={2}
                        d="M15 17h5l-1.4-1.4A2 2 0 0118 14.17V11a6 6 0 10-12 0v3.17a2 2 0
01-.6 1.43L4 17h5m6 0a3 3 0 11-6 0m6 0H9"
                      />
                    </svg>
                    {unreadCount > 0 && (
                      <span className="absolute -right-1 -top-1 inline-flex h-5 min-w-5 items-
center justify-center rounded-full bg-rose-500 px-1 text-[10px] font-bold text-white">
                        {unreadCount > 99 ? '99+' : unreadCount}
                      </span>
                    )}
                  </button>
                  {isNotificationOpen && (
                    <div className="absolute right-0 mt-2 w-80 rounded-2xl border border-
white/10 bg-slate-950/95 p-3 shadow-2xl">
                      <div className="mb-2 flex items-center justify-between">
                        <p className="text-sm font-semibold text-white">Recent Activity</p>
                        <button
                          onClick={() => setIsNotificationOpen(false)}
                          className="text-xs text-slate-300 hover:text-white"
                        >
                          Close
                        </button>
                      </div>
                      {notificationPreview.length === 0 ? (
                        <p className="text-sm text-slate-300">No recent notifications.</p>
                      ) : (
                        <div className="space-y-2">
                          {notificationPreview.map((item) => (
                            <div key={item.id} className="rounded-lg border border-white/10
bg-slate-900/70 p-2">
                              <p className="text-sm text-white">{item.title}</p>
                              {item.subtitle && <p className="text-xs text-slate-300">
{item.subtitle}</p>}
                              <p className="mt-1 text-[11px] text-slate-400">
{formatActivityTime(item.createdAt)}</p>
                              {user?.role === 'patient' && item.actionType ===
'confirm_fulfillment' && (
                                <button
                                  type="button"
                                  onClick={() =>
handleConfirmFulfillmentFromNotification(item)}
                                  disabled={confirmingNotificationId === item.id}
                                  className="mt-2 rounded-lg bg-emerald-300 px-3 py-1.5 text-
xs font-semibold text-slate-900 transition hover:bg-emerald-200 disabled:cursor-not-allowed
disabled:opacity-60"
                                >
                                  {confirmingNotificationId === item.id ? 'Confirming...' :
'Confirm Fulfillment'}
```

```
                              </button>
                            )}
                          </div>
                        ))}
                      </div>
                    )}
                  </div>
                )}
              </div>
            )}
            <div ref={profileRef} className="relative">
              <button
                type="button"
                onClick={() => setIsProfileOpen((prev) => !prev)}
                className="inline-flex h-10 w-10 items-center justify-center rounded-full
border border-white/15 bg-white/5 text-sm font-semibold text-slate-100 transition hover:bg-
white/10"
                aria-label="Open profile menu"
                title="Profile"
              >
                {user.name?.trim()?.charAt(0)?.toUpperCase() || 'U'}
              </button>
              {isProfileOpen && (
                <div className="absolute right-0 mt-2 w-72 rounded-2xl border border-
white/10 bg-slate-950/95 p-4 shadow-2xl">
                  <p className="text-xs font-semibold uppercase tracking-[0.12em] text-
slate-300">Profile</p>
                  <p className="mt-2 text-sm font-semibold text-white">{user.name}</p>
                  <p className="text-xs text-slate-300">{user.email}</p>
                  <p className="mt-1 text-xs text-cyan-100">{user.role}</p>
                  <div className="mt-4 space-y-2">
                    {profileMenuItems.map((item) => (
                      <Link
                        key={item.href}
                        href={item.href}
                        onClick={() => setIsProfileOpen(false)}
                        className="block rounded-lg border border-white/10 bg-white/5 px-3
py-2 text-sm text-slate-200 transition hover:bg-white/10"
                      >
                        {item.label}
                      </Link>
                    ))}
                  </div>
                  <button
                    onClick={handleLogout}
                    className="mt-4 w-full rounded-xl border border-rose-300/30 px-4 py-2
text-sm font-semibold text-rose-100 transition hover:bg-rose-300/20"
                  >
                    Logout
                  </button>
                </div>
              )}
            </div>
          </>
        ) : (
          <>
            <Link href="/login" className="rounded-full px-4 py-2 text-sm font-semibold
text-slate-200 hover:bg-white/10">
              Login
            </Link>
            <Link
              href="/register"
              className="rounded-full bg-cyan-300 px-4 py-2 text-sm font-semibold text-
slate-900 transition hover:bg-cyan-200"
            >
              Register
            </Link>
          </>
        )}
      </div>

      <button
        onClick={() => setIsMobileMenuOpen((open) => !open)}
        className="rounded-lg border border-white/15 p-2 text-slate-200 md:hidden"
        aria-label="Toggle navigation menu"
      >
        <svg className="h-5 w-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
          {isMobileMenuOpen ? (
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18
6M6 6l12 12" />
          ) : (
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M4 6h16M4
12h16M4 18h16" />
          )}
        </svg>
      </button>
    </div>
```

```
      {isMobileMenuOpen && (
        <div className="border-t border-white/10 bg-slate-950/95 px-4 pb-4 pt-3 md:hidden">
          <nav className="space-y-2">
            {navItems.map((item) => (
              <Link
                key={item.href}
                href={item.href}
                className={mobileNavLinkClass(item.href)}
                onClick={() => setIsMobileMenuOpen(false)}
              >
                {item.label}
              </Link>
            ))}
          </nav>

          <div className="mt-4 border-t border-white/10 pt-4">
            <button
              type="button"
              onClick={() => setIsDarkMode((prev) => !prev)}
              className="mb-3 inline-flex h-10 w-10 items-center justify-center rounded-full
border border-white/15 bg-white/5 text-slate-200 transition hover:bg-white/10"
              aria-label={isDarkMode ? 'Switch to light mode' : 'Switch to dark mode'}
              title={isDarkMode ? 'Switch to light mode' : 'Switch to dark mode'}
            >
              {isDarkMode ? (
                <svg className="h-5 w-5" viewBox="0 0 24 24" fill="none" stroke="currentColor"
aria-hidden="true">
                  <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    strokeWidth={2}
                    d="M12 3v2m0 14v2m9-9h-2M5 12H3m15.364 6.364-1.414-1.414M7.05 7.05 5.636
5.636m12.728 0L16.95 7.05M7.05 16.95l-1.414 1.414M12 8a4 4 0 100 8 4 4 0 000-8z"
                  />
                </svg>
              ) : (
                <svg className="h-5 w-5" viewBox="0 0 24 24" fill="none" stroke="currentColor"
aria-hidden="true">
                  <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    strokeWidth={2}
                    d="M21 12.8A9 9 0 1111.2 3a7 7 0 009.8 9.8z"
                  />
                </svg>
              )}
            </button>
            {isLoading ? (
              <p className="text-sm text-slate-300">Checking session...</p>
            ) : user ? (
              <div className="space-y-2">
                <div className="rounded-xl border border-white/15 bg-white/5 px-4 py-3 text-xs
text-slate-200">
                  <p className="text-[11px] uppercase tracking-[0.1em] text-slate-
300">Profile</p>
                  <p className="mt-1 font-semibold text-slate-100">{user.name}</p>
                  <p className="text-slate-300">{user.email}</p>
                  <p className="mt-1 text-cyan-100">{user.role}</p>
                </div>
                <div className="space-y-2">
                  {profileMenuItems.map((item) => (
                    <Link
                      key={item.href}
                      href={item.href}
                      className="block w-full rounded-xl border border-white/10 bg-white/5 px-
4 py-3 text-sm font-semibold text-slate-200 transition hover:bg-white/10"
                      onClick={() => setIsMobileMenuOpen(false)}
                    >
                      {item.label}
                    </Link>
                  ))}
                </div>
                <button
                  onClick={handleLogout}
                  className="w-full rounded-xl border border-rose-300/30 px-4 py-3 text-sm
font-semibold text-rose-100 transition hover:bg-rose-300/20"
                >
                  Logout
                </button>
              </div>
            ) : (
              <div className="grid grid-cols-2 gap-2">
                <Link
                  href="/login"
                  className="rounded-xl border border-white/15 px-4 py-3 text-center text-sm
font-semibold text-slate-200"
```

```
                  onClick={() => setIsMobileMenuOpen(false)}
                >
                  Login
                </Link>
                <Link
                  href="/register"
                  className="rounded-xl bg-cyan-300 px-4 py-3 text-center text-sm font-
semibold text-slate-900"
                  onClick={() => setIsMobileMenuOpen(false)}
                >
                  Register
                </Link>
              </div>
            )}
          </div>
        </div>
      )}
    </header>
  );
}
```

[35] components/NotificationComponent.tsx FILE

```
'use client';

import { useEffect, useState } from 'react';

interface NotificationData {
  type: 'new_prescription';
  prescription: {
    id: string;
    patientName: string;
    location: string;
    createdAt: string;
    status: string;
    selectedPharmacyIds?: string[];
  };
}

interface NotificationComponentProps {
  currentLocation: string;
  onNewPrescription?: () => void;
  maxItems?: number;
  title?: string;
}

const STORAGE_NOTIFICATIONS_KEY = 'pharmacy_notifications';
const STORAGE_UNREAD_KEY = 'pharmacy_notifications_unread';
const STORAGE_THEME_KEY = 'pharmacy_theme';

export default function NotificationComponent({
  currentLocation: _currentLocation,
  onNewPrescription = () => {},
  maxItems = 5,
  title = 'Recent Notifications',
}: NotificationComponentProps) {
  const [notifications, setNotifications] = useState<NotificationData[]>(() => {
    if (typeof window === 'undefined') return [];
    try {
      const raw = localStorage.getItem(STORAGE_NOTIFICATIONS_KEY);
      if (!raw) return [];
      const parsed = JSON.parse(raw) as NotificationData[];
      return Array.isArray(parsed) ? parsed : [];
    } catch {
      return [];
    }
  });
  const [isConnected, setIsConnected] = useState(false);
  const [showNotifications, setShowNotifications] = useState(false);
  const [isDarkMode, setIsDarkMode] = useState<boolean>(() => {
    if (typeof window === 'undefined') return true;
    try {
      const savedTheme = localStorage.getItem(STORAGE_THEME_KEY);
      return savedTheme ? savedTheme === 'dark' : true;
    } catch {
      return true;
    }
  });

  useEffect(() => {
    const syncTheme = () => {
      try {
        const savedTheme = localStorage.getItem(STORAGE_THEME_KEY);
```

```
      if (savedTheme === 'dark' || savedTheme === 'light') {
        setIsDarkMode(savedTheme === 'dark');
      }
    } catch {
      // Ignore storage parse/read issues and keep current in-memory theme.
    }
  };

  syncTheme();
  window.addEventListener('theme-updated', syncTheme);
  window.addEventListener('storage', syncTheme);
  return () => {
    window.removeEventListener('theme-updated', syncTheme);
    window.removeEventListener('storage', syncTheme);
  };
}, []);

useEffect(() => {
  let eventSource: EventSource | null = null;
  let reconnectTimer: ReturnType<typeof setTimeout> | null = null;

  const connect = () => {
    eventSource = new EventSource('/api/notifications/stream');

    eventSource.onopen = () => {
      setIsConnected(true);
    };

    eventSource.onmessage = (event) => {
      try {
        const data: NotificationData = JSON.parse(event.data);

        if (data.type !== 'new_prescription') {
          return;
        }

        setNotifications((prev) => {
          const existed = prev.some(
            (item) =>
              item.prescription.id === data.prescription.id &&
              item.prescription.createdAt === data.prescription.createdAt
          );
          const merged = [data, ...prev].filter(
            (item, index, arr) =>
              arr.findIndex(
                (candidate) =>
                  candidate.prescription.id === item.prescription.id &&
                  candidate.prescription.createdAt === item.prescription.createdAt
              ) === index
          );
          const clipped = merged.slice(0, 100);
          localStorage.setItem(STORAGE_NOTIFICATIONS_KEY, JSON.stringify(clipped));

          if (!existed) {
            const unreadCount = Number(localStorage.getItem(STORAGE_UNREAD_KEY) || '0');
            localStorage.setItem(STORAGE_UNREAD_KEY, String(unreadCount + 1));
          }
          window.dispatchEvent(new Event('notification-updated'));
          return clipped;
        });
        onNewPrescription();
        setShowNotifications(true);
        setTimeout(() => setShowNotifications(false), 5000);
      } catch (error) {
        console.error('Error parsing notification:', error);
      }
    };

    eventSource.onerror = (error) => {
      console.error('SSE connection error:', error);
      setIsConnected(false);
      if (eventSource) {
        eventSource.close();
        eventSource = null;
      }
      reconnectTimer = setTimeout(connect, 5000);
    };
  };

  connect();

  return () => {
    if (reconnectTimer) clearTimeout(reconnectTimer);
    if (eventSource) eventSource.close();
  };
}, [_currentLocation, onNewPrescription]);
```

```
  const clearNotifications = () => {
    setNotifications([]);
    setShowNotifications(false);
    localStorage.setItem(STORAGE_NOTIFICATIONS_KEY, JSON.stringify([]));
    localStorage.setItem(STORAGE_UNREAD_KEY, '0');
    window.dispatchEvent(new Event('notification-updated'));
  };

  const formatTime = (dateString: string) =>
    new Date(dateString).toLocaleTimeString('en-US', {
      hour: '2-digit',
      minute: '2-digit',
    });

  const statusTextClass = isDarkMode ? 'text-slate-300' : 'text-slate-700';
  const clearButtonClass = isDarkMode ? 'text-cyan-200 hover:text-cyan-100' : 'text-cyan-700
hover:text-cyan-800';
  const panelClass = isDarkMode
    ? 'rounded-2xl border border-white/10 bg-slate-950/70 p-4'
    : 'rounded-2xl border border-slate-200 bg-white p-4 shadow-sm';
  const panelTitleClass = isDarkMode ? 'text-white' : 'text-slate-900';
  const itemClass = isDarkMode
    ? 'flex items-center justify-between rounded-xl border border-white/10 bg-slate-900/70 p-
3'
    : 'flex items-center justify-between rounded-xl border border-slate-200 bg-slate-50 p-3';
  const itemTitleClass = isDarkMode ? 'text-white' : 'text-slate-900';
  const itemMetaClass = isDarkMode ? 'text-slate-300' : 'text-slate-600';

  return (
    <div className="mb-6">
      <div className="mb-4 flex items-center justify-between">
        <div className="flex items-center space-x-2">
          <div className={`h-3 w-3 rounded-full ${isConnected ? 'bg-emerald-500' : 'bg-rose-
500'}`} />
          <span className={`text-sm ${statusTextClass}`}>{isConnected ? 'Live Updates Active'
: 'Connecting...'}</span>
        </div>

        {notifications.length > 0 && (
          <button onClick={clearNotifications} className={`text-sm ${clearButtonClass}`}>
            Clear All ({notifications.length})
          </button>
        )}
      </div>

      {showNotifications && notifications.length > 0 && (
        <div
          className={`mb-4 rounded-xl border p-4 ${
            isDarkMode ? 'border-cyan-300/30 bg-cyan-300/10' : 'border-cyan-300/50 bg-cyan-50'
          }`}
        >
          <p className={`text-sm ${isDarkMode ? 'text-cyan-100' : 'text-cyan-800'}`}>
            <strong>New prescription alert:</strong>
{notifications[0].prescription.patientName} from{' '}
            {notifications[0].prescription.location}
          </p>
        </div>
      )}

      {notifications.length > 0 && (
        <div className={panelClass}>
          <h3 className={`mb-3 text-lg font-medium ${panelTitleClass}`}>{title}</h3>
          <div className="space-y-3">
            {notifications.slice(0, maxItems).map((notification, index) => (
              <div key={index} className={itemClass}>
                <div>
                  <p className={`text-sm font-medium ${itemTitleClass}`}>
                    New prescription from {notification.prescription.patientName}
                  </p>
                  <p className={`text-sm ${itemMetaClass}`}>
                    Location: {notification.prescription.location} •
{formatTime(notification.prescription.createdAt)}
                  </p>
                </div>
                <span
                  className={`rounded-full px-2.5 py-0.5 text-xs font-medium ${
                    isDarkMode ? 'bg-cyan-300/20 text-cyan-100' : 'bg-cyan-100 text-cyan-800'
                  }`}
                >
                  New
                </span>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
```

```
  );
}
```

[36] components/PharmacyMap.tsx FILE

```
'use client';

import { importLibrary, setOptions } from '@googlemaps/js-api-loader';
import { useEffect, useRef, useState } from 'react';

interface Pharmacy {
  _id: string;
  name: string;
  email: string;
  phone: string;
  address: string;
  location: string;
  coordinates: {
    lat: number;
    lng: number;
  };
  supportsPrescriptionUpload: boolean;
  isUsingService: boolean;
  subscriptionType: string | null;
  rating: number;
  reviewCount: number;
  operatingHours?: Record<string, unknown>;
  services?: string[];
}

interface PharmacyMapProps {
  pharmacies: Pharmacy[];
  userLocation?: { lat: number; lng: number };
  onPharmacySelect?: (pharmacy: Pharmacy) => void;
  selectedPharmacyId?: string | null;
  heightClassName?: string;
  showInfoCard?: boolean;
}

export default function PharmacyMap({
  pharmacies,
  userLocation,
  onPharmacySelect,
  selectedPharmacyId,
  heightClassName = 'h-96',
  showInfoCard = true,
}: PharmacyMapProps) {
  const mapRef = useRef<HTMLDivElement>(null);
  const markersRef = useRef<google.maps.Marker[]>([]);
  const markersByIdRef = useRef<Record<string, google.maps.Marker>>({});
  const [map, setMap] = useState<google.maps.Map | null>(null);
  const [mapError, setMapError] = useState('');
  const [internalSelectedPharmacyId, setInternalSelectedPharmacyId] = useState<string | null>
(null);
  const selectedPopupPharmacyId = selectedPharmacyId || internalSelectedPharmacyId;
  const selectedPharmacy =
    (selectedPopupPharmacyId
      ? pharmacies.find((pharmacy) => pharmacy._id === selectedPopupPharmacyId)
      : null) || null;

  const getMarkerIcon = (pharmacy: Pharmacy, isSelected = false) => ({
    url:
      pharmacy.isUsingService || pharmacy.subscriptionType
        ? 'data:image/svg+xml;charset=UTF-8,' +
          encodeURIComponent(`
            <svg width="40" height="40" viewBox="0 0 40 40"
xmlns="http://www.w3.org/2000/svg">
              <circle cx="20" cy="20" r="18" fill="${isSelected ? '#2563EB' : '#10B981'}"
stroke="white" stroke-width="3"/>
              <text x="20" y="25" text-anchor="middle" fill="white" font-size="16" font-
weight="bold">★</text>
            </svg>
          `)
        : 'data:image/svg+xml;charset=UTF-8,' +
          encodeURIComponent(`
            <svg width="40" height="40" viewBox="0 0 40 40"
xmlns="http://www.w3.org/2000/svg">
              <circle cx="20" cy="20" r="18" fill="${isSelected ? '#2563EB' : '#6B7280'}"
stroke="white" stroke-width="3"/>
              <text x="20" y="25" text-anchor="middle" fill="white" font-size="14">P</text>
            </svg>
          `),
    scaledSize: new google.maps.Size(isSelected ? 44 : 40, isSelected ? 44 : 40),
```

```
  });

  useEffect(() => {
    const initMap = async () => {
      if (!mapRef.current || map) return;
      const apiKey = process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY;
      if (!apiKey) {
        setMapError('Google Maps API key is missing. Add NEXT_PUBLIC_GOOGLE_MAPS_API_KEY and
restart the app.');
        return;
      }

      try {
        setOptions({
          apiKey,
          version: 'weekly',
        });
        await importLibrary('maps');

        const center = userLocation || { lat: 28.6139, lng: 77.209 };
        const mapInstance = new google.maps.Map(mapRef.current, {
          center,
          zoom: 12,
        });

        setMap(mapInstance);
      } catch (error) {
        console.error('Error loading Google Maps:', error);
        setMapError('Unable to load Google Maps. Check API key restrictions/billing and
reload.');
      }
    };

    initMap();
  }, [map, userLocation]);

  useEffect(() => {
    if (!map) return;

    markersRef.current.forEach((marker) => marker.setMap(null));
    markersByIdRef.current = {};

    const newMarkers = pharmacies.map((pharmacy) => {
      const marker = new google.maps.Marker({
        position: pharmacy.coordinates,
        map,
        title: pharmacy.name,
        icon: getMarkerIcon(pharmacy, selectedPharmacyId === pharmacy._id),
      });

      marker.addListener('click', () => {
        setInternalSelectedPharmacyId(pharmacy._id);
        onPharmacySelect?.(pharmacy);
      });

      markersByIdRef.current[pharmacy._id] = marker;
      return marker;
    });

    markersRef.current = newMarkers;

    return () => {
      newMarkers.forEach((marker) => marker.setMap(null));
      markersRef.current = [];
      markersByIdRef.current = {};
    };
  }, [map, pharmacies, onPharmacySelect, selectedPharmacyId]);

  useEffect(() => {
    if (map && userLocation) {
      map.setCenter(userLocation);
      map.setZoom(13);
    }
  }, [map, userLocation]);

  useEffect(() => {
    if (!map || !selectedPharmacyId) return;
    const selected = pharmacies.find((pharmacy) => pharmacy._id === selectedPharmacyId);
    if (!selected) return;

    map.panTo(selected.coordinates);
    if ((map.getZoom() || 0) < 14) {
      map.setZoom(14);
    }
  }, [map, pharmacies, selectedPharmacyId]);

  useEffect(() => {
```

```
    if (!map || pharmacies.length === 0) return;
    const hasVisibleSelectedPharmacy = Boolean(
      selectedPharmacyId && pharmacies.some((pharmacy) => pharmacy._id === selectedPharmacyId)
    );
    if (hasVisibleSelectedPharmacy) return;

    const bounds = new google.maps.LatLngBounds();
    pharmacies.forEach((pharmacy) => {
      bounds.extend(pharmacy.coordinates);
    });

    if (pharmacies.length === 1) {
      map.setCenter(pharmacies[0].coordinates);
      map.setZoom(14);
      return;
    }

    map.fitBounds(bounds);
  }, [map, pharmacies, selectedPharmacyId]);

  return (
    <div className="space-y-3">
      <div className="relative overflow-hidden rounded-xl border border-white/10 bg-slate-
950/50">
        <div ref={mapRef} className={`${heightClassName} w-full`} />
        {mapError && (
          <div className="absolute inset-3 flex items-center justify-center rounded-lg border
border-amber-300/40 bg-amber-300/10 p-4 text-center text-sm text-amber-100">
            {mapError}
          </div>
        )}
      </div>
      {showInfoCard && selectedPharmacy && (
        <div className="rounded-xl border border-white/10 bg-slate-900/85 p-4">
          <div className="flex items-start justify-between">
            <div>
              <h3 className="text-lg font-semibold text-white">{selectedPharmacy.name}</h3>
              <p className="mt-1 text-sm text-slate-300">{selectedPharmacy.address}</p>
              <div className="mt-3 flex flex-wrap items-center gap-2">
                {selectedPharmacy.isUsingService && (
                  <span className="inline-flex items-center rounded-full bg-emerald-300/20 px-
2 py-1 text-xs font-medium text-emerald-100">
                    ★ Service Partner
                  </span>
                )}
                {selectedPharmacy.subscriptionType && (
                  <span className="inline-flex items-center rounded-full bg-violet-300/20 px-2
py-1 text-xs font-medium text-violet-100">
                    {selectedPharmacy.subscriptionType} subscriber
                  </span>
                )}
                {selectedPharmacy.supportsPrescriptionUpload && (
                  <span className="inline-flex items-center rounded-full bg-sky-300/20 px-2
py-1 text-xs font-medium text-sky-100">
                    Upload Support
                  </span>
                )}
              </div>
              <div className="mt-2 flex items-center">
                <span className="text-amber-300">★</span>
                <span className="ml-1 text-sm text-slate-300">
                  {selectedPharmacy.rating} ({selectedPharmacy.reviewCount} reviews)
                </span>
              </div>
            </div>
            <button
              onClick={() => setInternalSelectedPharmacyId(null)}
              className="rounded-md border border-white/10 px-2 py-1 text-slate-400 transition
hover:bg-white/5 hover:text-slate-200"
              aria-label="Close selected pharmacy details"
            >
              ×
            </button>
          </div>
        </div>
      )}
    </div>
  );
}
```

[37] eslint.config.mjs FILE

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

[38] lib/auth.ts FILE

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { getUserModel } from '@/models/User';

const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key';

export interface UserPayload {
  id: string;
  email: string;
  role: string;
  name: string;
}

export const hashPassword = async (password: string): Promise<string> => {
  return await bcrypt.hash(password, 12);
};

export const comparePassword = async (password: string, hashedPassword: string):
Promise<boolean> => {
  return await bcrypt.compare(password, hashedPassword);
};

export const generateToken = (payload: UserPayload): string => {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' });
};

export const verifyToken = (token: string): UserPayload | null => {
  try {
    return jwt.verify(token, JWT_SECRET) as UserPayload;
  } catch (error) {
    return null;
  }
};

export const getUserFromToken = async (token: string) => {
  try {
    const payload = verifyToken(token);
    if (!payload) return null;

    if (!['patient', 'pharmacist'].includes(payload.role)) {
      return null;
    }

    const UserModel = await getUserModel(payload.role as 'patient' | 'pharmacist');
    return await UserModel.findById(payload.id);
  } catch (error) {
    console.error('getUserFromToken error:', error);
    return null;
  }
};
```

[39] lib/mongodb.ts FILE

```
import mongoose from 'mongoose';

const PATIENT_MONGODB_URI = process.env.PATIENT_MONGODB_URI!;
const PHARMACIST_MONGODB_URI = process.env.PHARMACIST_MONGODB_URI!;
```

```
const MAIN_MONGODB_URI = process.env.MONGODB_URI!; // For prescriptions and shared data

if (!PATIENT_MONGODB_URI) {
  throw new Error('Please define the PATIENT_MONGODB_URI environment variable inside
.env.local');
}

if (!PHARMACIST_MONGODB_URI) {
  throw new Error('Please define the PHARMACIST_MONGODB_URI environment variable inside
.env.local');
}

if (!MAIN_MONGODB_URI) {
  throw new Error('Please define the MONGODB_URI environment variable inside .env.local');
}

/**
 * Global is used here to maintain cached connections across hot reloads
 * in development. This prevents connections growing exponentially
 * during API Route usage.
 */
let cached = (global as any).mongoose;

if (!cached) {
  cached = (global as any).mongoose = {
    patientConn: null,
    patientPromise: null,
    pharmacistConn: null,
    pharmacistPromise: null,
    mainConn: null,
    mainPromise: null
  };
}

async function dbConnectPatients() {
  if (cached.patientConn) {
    return cached.patientConn;
  }

  if (!cached.patientPromise) {
    const opts = {
      bufferCommands: true, // Allow buffering to prevent connection issues
    };

    cached.patientPromise = mongoose.createConnection(PATIENT_MONGODB_URI, opts);
    console.log('Connected to Patient Database');
  }

  cached.patientConn = cached.patientPromise;
  return cached.patientConn;
}

async function dbConnectPharmacists() {
  if (cached.pharmacistConn) {
    return cached.pharmacistConn;
  }

  if (!cached.pharmacistPromise) {
    const opts = {
      bufferCommands: true, // Allow buffering to prevent connection issues
    };

    cached.pharmacistPromise = mongoose.createConnection(PHARMACIST_MONGODB_URI, opts);
    console.log('Connected to Pharmacist Database');
  }

  cached.pharmacistConn = cached.pharmacistPromise;
  return cached.pharmacistConn;
}

async function dbConnectMain() {
  if (cached.mainConn) {
    return cached.mainConn;
  }

  if (!cached.mainPromise) {
    const opts = {
      bufferCommands: true, // Allow buffering to prevent connection issues
    };

    cached.mainPromise = mongoose.createConnection(MAIN_MONGODB_URI, opts);
    console.log('Connected to Main Database');
  }

  cached.mainConn = cached.mainPromise;
  return cached.mainConn;
}
```

```
// Legacy function for backward compatibility (use specific functions instead)
async function dbConnect() {
  console.warn('Warning: Using legacy dbConnect. Please use dbConnectPatients(),
dbConnectPharmacists(), or dbConnectMain() instead.');
  return dbConnectMain();
}

export { dbConnectPatients, dbConnectPharmacists, dbConnectMain };
export default dbConnect;
```

[40] lib/seedPharmacies.ts FILE

```
import getPharmacyModel from '@/models/Pharmacy';
import getPharmacistUserModel from '@/models/PharmacistUser';

const samplePharmacies = [
  {
    name: 'City Pharmacy Plus',
    email: 'citypharmacy@example.com',
    phone: '+91-9876543210',
    address: '123 MG Road, Connaught Place, New Delhi, Delhi 110001',
    location: 'New Delhi',
    coordinates: { lat: 28.6304, lng: 77.2177 },
    supportsPrescriptionUpload: true,
    isUsingService: true,
    subscriptionType: 'premium',
    rating: 4.8,
    reviewCount: 156,
    services: ['24/7 Service', 'Home Delivery', 'Online Consultation'],
    operatingHours: {
      monday: { open: '09:00', close: '22:00' },
      tuesday: { open: '09:00', close: '22:00' },
      wednesday: { open: '09:00', close: '22:00' },
      thursday: { open: '09:00', close: '22:00' },
      friday: { open: '09:00', close: '22:00' },
      saturday: { open: '10:00', close: '20:00' },
      sunday: { open: '10:00', close: '18:00' }
    }
  },
  {
    name: 'HealthCare Pharmacy',
    email: 'healthcare@example.com',
    phone: '+91-9876543211',
    address: '456 Karol Bagh, New Delhi, Delhi 110005',
    location: 'New Delhi',
    coordinates: { lat: 28.6517, lng: 77.1925 },
    supportsPrescriptionUpload: true,
    isUsingService: true,
    subscriptionType: 'yearly',
    rating: 4.6,
    reviewCount: 89,
    services: ['Home Delivery', 'Insurance Claims'],
    operatingHours: {
      monday: { open: '08:00', close: '21:00' },
      tuesday: { open: '08:00', close: '21:00' },
      wednesday: { open: '08:00', close: '21:00' },
      thursday: { open: '08:00', close: '21:00' },
      friday: { open: '08:00', close: '21:00' },
      saturday: { open: '09:00', close: '19:00' },
      sunday: { open: '10:00', close: '17:00' }
    }
  },
  {
    name: 'MediCare Pharmacy',
    email: 'medicare@example.com',
    phone: '+91-9876543212',
    address: '789 Lajpat Nagar, New Delhi, Delhi 110024',
    location: 'New Delhi',
    coordinates: { lat: 28.5789, lng: 77.2401 },
    supportsPrescriptionUpload: false,
    isUsingService: false,
    subscriptionType: null,
    rating: 4.2,
    reviewCount: 45,
    services: ['Basic Pharmacy Services'],
    operatingHours: {
      monday: { open: '09:00', close: '20:00' },
      tuesday: { open: '09:00', close: '20:00' },
      wednesday: { open: '09:00', close: '20:00' },
      thursday: { open: '09:00', close: '20:00' },
      friday: { open: '09:00', close: '20:00' },
      saturday: { open: '10:00', close: '18:00' },
```

```
      sunday: { open: 'Closed', close: 'Closed' }
    }
  },
  {
    name: 'Wellness Pharmacy',
    email: 'wellness@example.com',
    phone: '+91-9876543213',
    address: '321 South Extension, New Delhi, Delhi 110049',
    location: 'New Delhi',
    coordinates: { lat: 28.5689, lng: 77.2201 },
    supportsPrescriptionUpload: true,
    isUsingService: false,
    subscriptionType: null,
    rating: 4.4,
    reviewCount: 67,
    services: ['Home Delivery', 'Health Check-ups'],
    operatingHours: {
      monday: { open: '08:30', close: '22:00' },
      tuesday: { open: '08:30', close: '22:00' },
      wednesday: { open: '08:30', close: '22:00' },
      thursday: { open: '08:30', close: '22:00' },
      friday: { open: '08:30', close: '22:00' },
      saturday: { open: '09:00', close: '21:00' },
      sunday: { open: '10:00', close: '19:00' }
    }
  },
  {
    name: 'QuickMeds Pharmacy',
    email: 'quickmeds@example.com',
    phone: '+91-9876543214',
    address: '654 Rajouri Garden, New Delhi, Delhi 110027',
    location: 'New Delhi',
    coordinates: { lat: 28.6475, lng: 77.1234 },
    supportsPrescriptionUpload: false,
    isUsingService: true,
    subscriptionType: 'monthly',
    rating: 4.0,
    reviewCount: 23,
    services: ['Express Delivery', 'Online Ordering'],
    operatingHours: {
      monday: { open: '07:00', close: '23:00' },
      tuesday: { open: '07:00', close: '23:00' },
      wednesday: { open: '07:00', close: '23:00' },
      thursday: { open: '07:00', close: '23:00' },
      friday: { open: '07:00', close: '23:00' },
      saturday: { open: '08:00', close: '22:00' },
      sunday: { open: '09:00', close: '20:00' }
    }
  }
];

export async function seedPharmacies() {
  try {
    const PharmacyModel = await getPharmacyModel();
    const PharmacistUserModel = await getPharmacistUserModel();

    for (const pharmacyData of samplePharmacies) {
      // Create a corresponding pharmacist user
      const pharmacistUser = new PharmacistUserModel({
        name: pharmacyData.name,
        email: pharmacyData.email,
        password: '$2a$10$hashedpassword', // This would be properly hashed in real
implementation
        role: 'pharmacist',
        phone: pharmacyData.phone,
        address: pharmacyData.address,
        location: pharmacyData.location,
        subscriptionType: pharmacyData.subscriptionType,
        subscriptionStart: pharmacyData.isUsingService ? new Date() : null,
        subscriptionEnd: pharmacyData.isUsingService ? new Date(Date.now() + 365 * 24 * 60 *
60 * 1000) : null,
      });

      await pharmacistUser.save();

      // Create pharmacy entry
      const pharmacy = new PharmacyModel({
        ...pharmacyData,
        pharmacistId: pharmacistUser._id,
      });

      await pharmacy.save();
      console.log(`Created pharmacy: ${pharmacyData.name}`);
    }

    console.log('Sample pharmacies seeded successfully!');
  } catch (error) {
```

```
    console.error('Error seeding pharmacies:', error);
  }
}
```

[41] middleware.ts FILE

```
import { NextRequest, NextResponse } from 'next/server';

type JwtPayload = {
  role?: string;
};

const decodeJwtPayload = (token: string): JwtPayload | null => {
  try {
    const parts = token.split('.');
    if (parts.length !== 3) return null;
    const base64Url = parts[1];
    const base64 = base64Url.replace(/-/g, '+').replace(/_/g, '/');
    const padded = base64.padEnd(base64.length + ((4 - (base64.length % 4)) % 4), '=');
    const payload = JSON.parse(atob(padded)) as JwtPayload;
    return payload;
  } catch {
    return null;
  }
};

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value;
  const path = request.nextUrl.pathname;

  if (!path.startsWith('/dashboard')) {
    return NextResponse.next();
  }

  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  const payload = decodeJwtPayload(token);
  if (!payload) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  if (payload.role !== 'pharmacist') {
    return NextResponse.redirect(new URL('/upload', request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

[42] models/PatientUser.ts FILE

```
import mongoose from 'mongoose';
import { dbConnectPatients } from '@/lib/mongodb';

// Patient User Schema
const PatientUserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['patient'], required: true, default: 'patient' },
  phone: { type: String },
  address: { type: String },
  createdAt: { type: Date, default: Date.now },
});

// Export a function that returns the model after ensuring connection
const getPatientUserModel = async () => {
  const conn = await dbConnectPatients();
  return conn.models.PatientUser || conn.model('PatientUser', PatientUserSchema);
};

export default getPatientUserModel;
```

[43] models/PharmacistUser.ts FILE

```
import mongoose from 'mongoose';
import { dbConnectPharmacists } from '@/lib/mongodb';

// Pharmacist User Schema
const PharmacistUserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['pharmacist'], required: true, default: 'pharmacist' },
  phone: { type: String },
  address: { type: String },
  location: { type: String }, // pharmacy store location
  subscriptionType: { type: String, enum: ['monthly', 'yearly', 'premium'], default: null },
  subscriptionStart: { type: Date },
  subscriptionEnd: { type: Date },
  createdAt: { type: Date, default: Date.now },
});

// Export a function that returns the model after ensuring connection
const getPharmacistUserModel = async () => {
  const conn = await dbConnectPharmacists();
  return conn.models.PharmacistUser || conn.model('PharmacistUser', PharmacistUserSchema);
};

export default getPharmacistUserModel;
```

[44] models/Pharmacy.ts FILE

```
import mongoose from 'mongoose';
import { dbConnectMain } from '@/lib/mongodb';

// Pharmacy Schema for enhanced pharmacy finder
const PharmacySchema = new mongoose.Schema({
  pharmacistId: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'PharmacistUser'
},
  name: { type: String, required: true },
  email: { type: String, required: true },
  phone: { type: String },
  address: { type: String, required: true },
  location: { type: String, required: true }, // City/area
  coordinates: {
    lat: { type: Number, required: true },
    lng: { type: Number, required: true }
  },
  supportsPrescriptionUpload: { type: Boolean, default: false },
  isUsingService: { type: Boolean, default: false }, // Whether they use our e-pharmacy
service
  subscriptionType: { type: String, enum: ['monthly', 'yearly', 'premium'], default: null },
  rating: { type: Number, min: 0, max: 5, default: 0 },
  reviewCount: { type: Number, default: 0 },
  operatingHours: {
    monday: { open: String, close: String },
    tuesday: { open: String, close: String },
    wednesday: { open: String, close: String },
    thursday: { open: String, close: String },
    friday: { open: String, close: String },
    saturday: { open: String, close: String },
    sunday: { open: String, close: String }
  },
  services: [{ type: String }], // Additional services offered
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
});

// Index for geospatial queries
PharmacySchema.index({ coordinates: '2dsphere' });

// Export a function that returns the model after ensuring connection
const getPharmacyModel = async () => {
  const conn = await dbConnectMain();
  return conn.models.Pharmacy || conn.model('Pharmacy', PharmacySchema);
};

export default getPharmacyModel;
```

[45] models/Prescription.ts FILE

```
import mongoose from 'mongoose';
import { dbConnectMain } from '@/lib/mongodb';

// Prescription Schema for main database
const PrescriptionSchema = new mongoose.Schema({
  patientName: { type: String, required: true },
  patientEmail: { type: String, required: true },
  patientPhone: { type: String },
  patientAddress: { type: String },
  prescriptionImages: [{ type: String }], // Array of base64 images
  notes: { type: String },
  status: { type: String, enum: ['pending', 'assigned', 'fulfilled'], default: 'pending' },
  pharmacistIds: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }], // Array of assigned
pharmacy IDs
  pharmacyStatuses: [{
    pharmacyId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    status: { type: String, enum: ['pending', 'accepted', 'rejected', 'fulfilled',
'fulfillment_requested'], default: 'pending' },
    availabilityResponse: {
      type: String,
      enum: ['not_available', 'same_medicine_available', 'same_salt_different_company'],
      default: null,
    },
    pharmacistMessage: { type: String, default: '' },
    assignedAt: { type: Date, default: Date.now },
    completedAt: { type: Date }
  }],
  createdAt: { type: Date, default: Date.now },
  location: { type: String, required: true }, // city or zip for nearby matching
});

// Create and export the Prescription model for main database
let Prescription: mongoose.Model<unknown> | null = null;

const getPrescriptionModel = async () => {
  if (Prescription) return Prescription;

  try {
    const mainConn = await dbConnectMain();
    Prescription = mainConn.models.Prescription || mainConn.model('Prescription',
PrescriptionSchema);
    return Prescription;
  } catch (error) {
    console.error('Error creating Prescription model:', error);
    throw error;
  }
};

export default getPrescriptionModel;
```

[46] models/User.ts FILE

```
import getPatientUserModel from './PatientUser';
import getPharmacistUserModel from './PharmacistUser';

// Function to get the appropriate User model based on role
export const getUserModel = async (role: 'patient' | 'pharmacist') => {
  if (role === 'patient') {
    return await getPatientUserModel();
  } else if (role === 'pharmacist') {
    return await getPharmacistUserModel();
  } else {
    throw new Error('Invalid user role');
  }
};

// Legacy User model for backward compatibility (uses patient database)
import mongoose from 'mongoose';

const LegacyUserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['patient', 'pharmacist'], required: true },
  phone: { type: String },
  address: { type: String },
  location: { type: String }, // for pharmacists, their store location
  subscriptionType: { type: String, enum: ['monthly', 'yearly', 'premium'], default: null },
  subscriptionStart: { type: Date },
  subscriptionEnd: { type: Date },
  createdAt: { type: Date, default: Date.now },
});
```

```
const LegacyUser = mongoose.models.User || mongoose.model('User', LegacyUserSchema);

export default LegacyUser;
```

[47] next.config.ts FILE

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

[48] package.json FILE

```
{
  "name": "e-pharmacy",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@googlemaps/js-api-loader": "^2.0.2",
    "@types/bcryptjs": "^2.4.6",
    "@types/google.maps": "^3.58.1",
    "@types/jsonwebtoken": "^9.0.10",
    "bcryptjs": "^3.0.3",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.1.5",
    "next": "16.1.4",
    "react": "19.2.3",
    "react-dom": "19.2.3"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.1.4",
    "tailwindcss": "^4.1.18",
    "typescript": "^5"
  }
}
```

[49] postcss.config.mjs FILE

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

[50] tailwind.config.js FILE

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    "./lib/**/*.{js,ts,jsx,tsx,mdx}",
  ],
```

```
  theme: {
    extend: {},
  },
  plugins: [],
};
```

[51] tsconfig.json FILE

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```